



**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Федеральное государственное автономное
образовательное учреждение высшего образования
«Дальневосточный федеральный университет»**

ИНСТИТУТ ШКОЛА ЭКОНОМИКИ И МЕНЕДЖМЕНТА

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

**по работе с учебными файлами с использованием нейронных сетей
для студентов Дальневосточного федерального университета**

**г. Владивосток
2026**

Содержание

Введение	4
Работа с оформлением файла через макросы в Word	11
Подготовка перед использованием макросов	11
Самостоятельная запись макросов	14
Оформление текста	16
Оформление таблиц	17
Добавление нумерации	18
Оформление заголовков	19
Оформление содержания	20
Работа с сервисом «Яндекс Алиса Про»	21
Создание проекта	21
Загрузка материалов	23
Работа с чатом (запросы)	24
Лайфхаки и возможности	25
Как поделиться проектом или чатом	26
Работа с ИИ-ассистентом QWEN	27
Базовые принципы взаимодействия	27
Выбор режима генерации	29
Выбор языковой модели	30
Структура эффективного запроса	31
Работа с учебными материалами	32
Оптимизация запросов	34
Модели организации сессий	35
Совместный доступ	37
Работа через Visual Studio Code	39
Установка Visual Studio Code	39
Привязка VS Code к GitHub	43
Регистрация на GitHub	44
Установка Git на компьютер	45
Привязка GitHub к VS Code	45
Установка расширений в Visual Studio Code	47

Рекомендуемые расширения	47
Создание папки для работы с файлами проекта	50
Работа с файлами в Visual Studio Code	53
Открытие файлов	53
Организация учебных материалов	54
Работа с несколькими файлами	54
Поиск по файлам	55
Использование нейросети в Visual Studio Code	56
Работа с расширением Qwen.....	56
Работа с GitHub Copilot	57
Настройка AI Tunnel и интеграция с VS Code	59
Регистрация в сервисе AI Tunnel.....	59
Просмотр стоимости моделей и тарифов.....	61
Пополнение баланса	64
Получение и настройка API-ключа	66
Установка расширения Cline и настройка подключения (Cline + AI Tunnel)....	68
Основные элементы интерфейса Cline	72
Настройка агентов: системные файлы AGENTS.md, QWEN.md, Project.md и другие	76
Практическое применение Visual Studio Code	80
Подготовка отчётов и рефератов	80
Создание презентации в Visual Studio Code с помощью HTML	84
Анализ информации.....	93
Работа с таблицами и данными	97
Подготовка к экзаменам и зачётам	99
Возможные проблемы.....	106
Заключение.....	115

Введение

Современный этап развития высшего образования в Российской Федерации характеризуется активным внедрением цифровых технологий и интеллектуальных систем во все сферы учебной и научно-исследовательской деятельности. Федеральные государственные образовательные стандарты последнего поколения предполагают не только освоение предметных знаний, но и формирование у студентов универсальных компетенций в области работы с информацией, включая её поиск, критический анализ, структурирование, обработку и представление. В этих условиях традиционные методы подготовки курсовых работ, рефератов, отчётов и презентаций, основанные преимущественно на ручном труде и последовательном просмотре больших объёмов литературы, перестают быть достаточными. Возникает объективная потребность в освоении современных инструментов автоматизации, позволяющих сократить временные затраты, минимизировать технические ошибки и сосредоточиться на содержательной стороне учебной работы.

Одним из наиболее значимых трендов последних лет стало широкое распространение доступных нейросетевых моделей и интеллектуальных ассистентов (таких как «Яндекс Алиса Про», Qwen, GitHub Copilot и других). Эти инструменты способны выполнять широкий спектр задач: от генерации структурированных планов и кратких конспектов до анализа табличных данных, переформулирования текста в академическом стиле и создания работающих HTML-презентаций по текстовому описанию. Однако, как показывает практика, простое наличие доступа к нейросети не гарантирует высокого качества результатов. Эффективное использование искусственного интеллекта в учебных целях требует понимания принципов работы таких систем, умения корректно формулировать запросы (промты), выстраивать диалог, контролировать достоверность ответов и интегрировать полученные результаты в итоговые учебные документы. Без системного методического сопровождения студент рискует либо недооценить потенциал новых технологий, либо, напротив,

столкнуться с неверными или поверхностными ответами, что может негативно сказаться на качестве обучения.

Настоящее методическое руководство разработано для студентов Дальневосточного федерального университета всех направлений подготовки и призвано восполнить указанный пробел. Его основная цель – сформировать у обучающихся целостное, практически ориентированное представление о современных цифровых инструментах, которые могут быть эффективно использованы на всех этапах учебной деятельности: от подготовки черновиков и оформления по требованиям до анализа сложных источников и систематизации материалов для экзаменов. В отличие от многих существующих пособий, которые фокусируются только на одном инструменте или подходе, данная работа предлагает комплексный взгляд, последовательно рассматривая три категории средств автоматизации, каждая из которых решает свои задачи и обладает определёнными преимуществами.

Первая часть руководства посвящена макросам в текстовом редакторе Microsoft Word. Этот инструмент остаётся актуальным благодаря своей автономности (не требует доступа к интернету), полной совместимости с типовыми шаблонами образовательных организаций и возможности мгновенного применения жёстких требований к оформлению (поля, шрифты, отступы, нумерация, оглавление). В пособии подробно описаны способы записи макросов без программирования, а также готовые листинги кода для типовых операций: форматирование основного текста, таблиц, заголовков, автоматическое создание содержания. Подчёркиваются ограничения макросов (например, невозможность скрыть номер на первых двух страницах без ручного добавления разделов), что помогает студенту реалистично оценивать область их применения.

Вторая часть методических указаний знакомит читателя с облачными ИИ-сервисами, прежде всего с «Яндекс Алиса Про» и ассистентом Qwen. Эти сервисы позволяют загружать до 25 файлов (лекции, методички, ГОСТы, научные статьи) и получать ответы, строго основанные на предоставленных источниках, что критически важно для учебной работы, где ценятся достоверность и проверяемость

информации. В разделе детально разбираются: создание проектов, загрузка материалов, структура эффективного запроса (контекст – задача – ограничения – формат), сравнение режимов генерации («Автоматический», «Мышление», «Быстрый»), а также лайфхаки вроде запроса прямой цитаты или генерации вопросов для самопроверки. Отдельное внимание уделяется модели организации сессий (единый контекст или разделение по задачам), возможности совместного доступа через ссылку и сравнению различных языковых моделей семейства Qwen.

Третья, наиболее объёмная часть пособия посвящена интеграции нейросетевых технологий в профессиональную среду разработки Visual Studio Code (VS Code). Обосновывается выбор VS Code как единой рабочей среды, объединяющей файловый менеджер, текстовый редактор, поддержку Git, просмотрщики PDF и Excel, а также расширения для работы с ИИ. Даются пошаговые инструкции по установке VS Code, регистрации на GitHub, установке Git, привязке аккаунта и установке рекомендуемых расширений. Особое место занимает раздел по настройке системных файлов агентов (AGENTS.md, copilot-instructions.md, CLAUDE.md), которые позволяют один раз задать правила поведения ИИ в проекте – например, отвечать на русском языке, использовать академический стиль, не придумывать факты, сохранять создаваемые файлы в определённые папки. Это существенно экономит время при многократных сессиях работы с одним проектом.

Практическая значимость VS Code раскрывается через конкретные кейсы: подготовка отчётов и рефератов с генерацией структуры и черновиков, создание полноценных HTML-презентаций с навигацией (кнопки «Вперёд/Назад», стрелки клавиатуры) и возможностью вставки изображений, анализ информации из нескольких документов, сравнение табличных данных, подготовка к экзаменам через генерацию кратких конспектов, терминов и вопросов для самопроверки. Каждый кейс сопровождается примерами эффективных запросов и скриншотами интерфейса, что позволяет студенту сразу применить изученное на практике.

Отдельный блок методических указаний посвящён настройке доступа к платным языковым моделям через сервис AI Tunnel с последующей интеграцией в

VS Code через расширение Cline. Здесь подробно описываются: регистрация, пополнение баланса, получение API-ключа, выбор модели из каталога с фильтрацией по стоимости и контекстному окну, а также настройка подключения. Такой подход открывает перед студентом доступ к широкому спектру моделей (включая продвинутые варианты OpenAI и Anthropic) без необходимости использования VPN, что особенно актуально в текущих геополитических условиях.

Завершается методическое пособие разделом, посвящённым типичным проблемам и способам их решения (расширение не открывается, нейросеть не отвечает, потерян чат, проблемы с Python, поиск сохранённых файлов и др.). Это превращает руководство из простого описания в практический инструмент, к которому студент может обращаться в процессе реальной работы. В заключении намечаются дальнейшие перспективы: использование навыков (Skills), под-агентов (Subagents) и протокола MCP для подключения к внешним базам данных и библиотекам, а также рекомендация по оформлению GitHub Student Developer Pack для бесплатного доступа к профессиональным инструментам.

Таким образом, настоящее методическое руководство не только знакомит студентов с конкретными программными продуктами, но и формирует системное понимание того, как выстроить персонализированную интеллектуальную среду для учебной деятельности, объединяющую локальные средства автоматизации, облачные ИИ-сервисы, профессиональные редакторы и системы контроля версий. Авторы выражают уверенность, что последовательное освоение предложенных материалов позволит студентам Дальневосточного федерального университета повысить эффективность самостоятельной работы, улучшить качество представляемых академических текстов и приобрести цифровые компетенции, востребованные как в продолжении образования, так и в будущей профессиональной карьере.

Перед дальнейшим изучением методического руководства рекомендуем обратиться к глоссарию. Также можете ознакомиться с нашим методическим руководством в электронном формате: <https://yutr-228.github.io/method-guide/>.

Глоссарий

1. API (Application Programming Interface) – набор правил, по которым программы обмениваются данными. «Посредник» между приложениями. Пример: AI Tunnel даёт API для подключения VS Code к моделям ИИ.

2. API-ключ – уникальный код, идентифицирующий пользователя при обращении к API. Нужно хранить в секрете. Пример: sk-xxxxxx – ключ, который получают в AI Tunnel для настройки Cline.

3. CI/CD (Continuous Integration / Continuous Delivery) – непрерывная интеграция и доставка – практика автоматической сборки, тестирования и развёртывания кода.

4. CSV (Comma-Separated Values) – формат табличных данных, где значения разделены запятыми или точками с запятой. Пример: Таблицы из Excel можно сохранить в CSV.

5. Git – система контроля версий, которая отслеживает изменения в файлах. Позволяет возвращать старые версии и работать в команде. Пример: Git хранит историю курсовой – всегда можно вернуться к черновику.

6. GitHub – облачная платформа для хранения проектов с использованием Git. Аналог «облака для кода». Пример: Студенты выкладывают курсовые на GitHub и делятся ссылкой с преподавателем.

7. HTML (HyperText Markup Language) – язык разметки для создания веб-страниц. Определяет структуру документа.

8. IDE (Integrated Development Environment) – среда разработки, объединяющая редактор кода, терминал, отладчик и другие инструменты. Пример: VS Code, PyCharm, IntelliJ IDEA – всё это IDE.

9. Jupyter Notebook – интерактивная среда для анализа данных, где можно писать код и сразу видеть результат. Пример: в VS Code через расширение Jupyter можно запускать ячейки с Python прямо в редакторе.

10. Markdown – лёгкий язык разметки для форматирования текста (заголовки, списки, таблицы) простыми символами.

11. MCP (Model Context Protocol) – универсальный протокол, позволяющий ИИ-агентам подключаться к внешним системам (базам данных, API). Пример: через MCP агент может сам найти статьи в университетской библиотеке.

12. Open Source – программы с открытым исходным кодом — их можно бесплатно использовать, изучать и изменять.

13. PDF (Portable Document Format) – формат для представления документов, не зависящий от устройства. Удобен для чтения, но труден для редактирования.

14. Python – популярный язык программирования. Используется для анализа данных, ИИ, веб-разработки.

15. Skill (навык) – многоразовая инструкция для агента, которую он загружает по требованию. Пример: навык «Академический рецензент» проверит курсовую на соответствие ГОСТ.

16. Subagent (подагент) – специализированный агент, который выполняет часть большой задачи. Пример: один подагент ищет источники, второй пишет текст, третий оформляет библиографию.

17. Автодополнение – функция, которая предлагает варианты продолжения слов, кода или команд по мере ввода.

18. Аннотация – краткая характеристика работы, раскрывающая тему, цель, методы и основные результаты.

19. Библиотека (library) – набор готовых функций и классов, которые можно использовать в своей программе, чтобы не писать всё с нуля.

20. Бэкенд (backend) – серверная часть приложения, которая обрабатывает данные, работает с базами данных и выполняет основную логику.

21. ИИ-агент – искусственный интеллект, который не только отвечает, но и выполняет действия (создаёт и редактирует файлы).

22. Код (исходный код) – текст программы на языке программирования (Python, HTML и др.). Пример: Весь код нашего сайта написан на HTML, CSS и JavaScript.

23. Контекстное окно – максимальный объём текста (в токенах), который модель может обработать за один запрос.

24. Макрос – записанная последовательность действий, выполняемая одной командой. Ускоряет рутинные операции.

25. Нейросеть – программа, обученная на больших данных распознавать и генерировать текст, изображения и т.д. Пример: ChatGPT, Qwen, Claude – нейросети для работы с текстом.

26. Промт (prompt) – инструкция, которую пользователь отправляет нейросети, чтобы получить нужный результат.

27. Расширение (extension) – дополнительный модуль, добавляющий новые функции в VS Code (подсветку, проверку орфографии, ИИ).

28. Репозиторий – папка, за которой следит Git. Хранит файлы проекта и всю историю изменений. Пример: репозиторий курсовой – место, где Git отслеживает правки.

29. Синтаксис – правила написания кода или разметки. Если нарушить, то программа не поймёт, что вы хотите. Пример: в Markdown # является заголовком, а не просто решёткой.

30. Терминал – окно для ввода текстовых команд. В VS Code находится внизу (CTRL+).

31. Токен – минимальная единица текста для нейросети. Может быть словом, частью слова или знаком. Пример: «Привет, мир!» — это 4 токена (Привет + , + мир + !).

32. Файловая система – способ организации папок и файлов на компьютере. Пример: проводник в Windows – это графическое представление файловой системы.

33. Фронтенд (frontend) – клиентская часть, которую видит пользователь (интерфейс).

Работа с оформлением файла через макросы в Word

Каждый студент сталкивается с необходимостью оформлять курсовые, отчёты, рефераты по строгим требованиям (поля, шрифты, отступы, нумерация, содержание). Делать это вручную для каждого документа – долго и утомительно. Макросы в Word позволяют одним нажатием кнопки применить все нужные настройки к любому файлу. Это экономит часы и исключает ошибки.

Макросы работают прямо внутри Word, не требуют установки дополнительного ПО и не зависят от интернета. Они идеальны для быстрой «одноразовой» автоматизации. Однако у них есть ограничения.

Подготовка перед использованием макросов

1. Прожимаем комбинацию клавиш Alt + F11 и получаем следующее диалоговое окно (Рисунок 1)

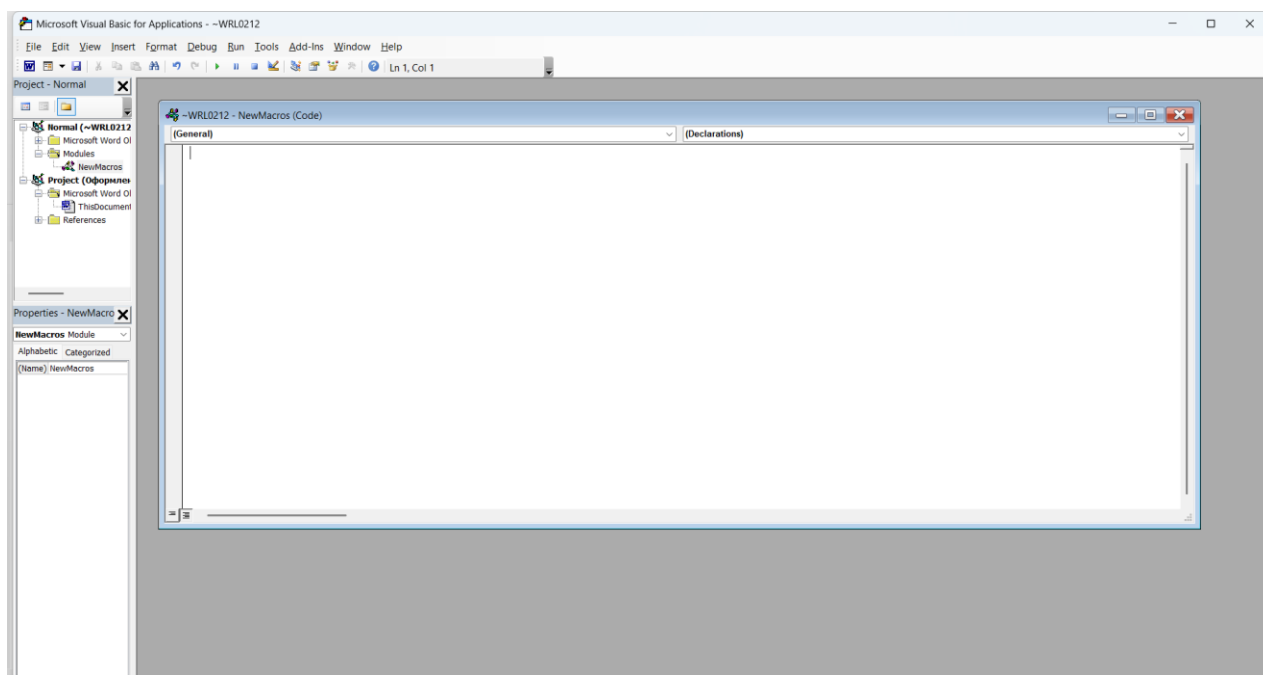


Рисунок 1 – Диалоговое окно

2. Вставляем в окно необходимый нам код и сохраняем (Рисунок 2).

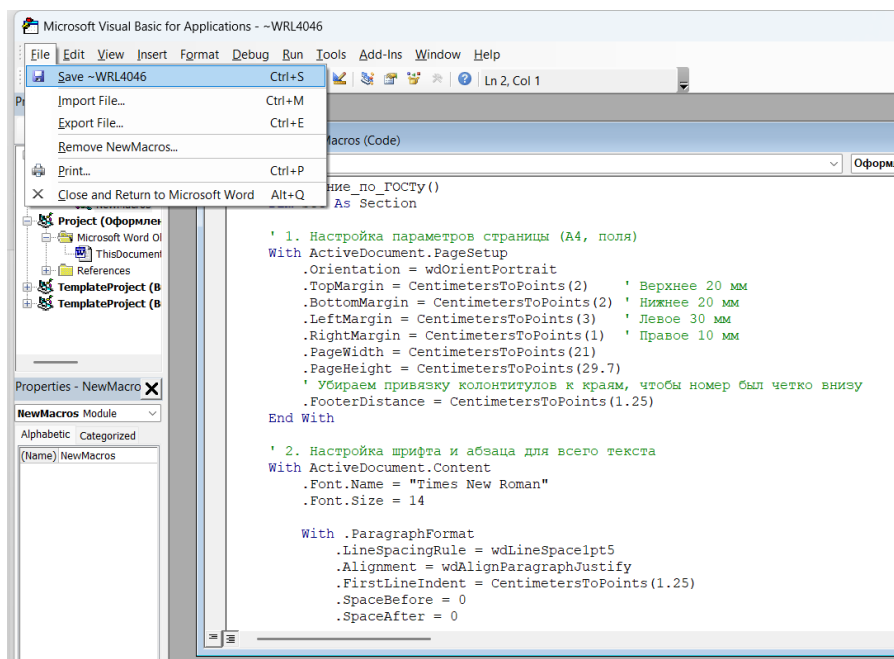


Рисунок 2 – Пример кода

3. Чтобы добавить еще макросы необходимо в этом же диалоговом окне (см. п. 1) сверху выбрать «Insert», далее нажать «Module» (Рисунок 3).

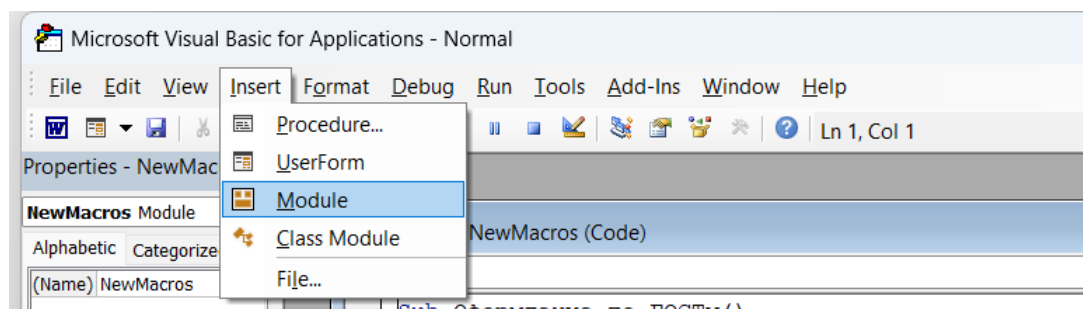


Рисунок 3 – Добавление макросов

4. Если правильно выполнен п. 3, то появится следующее окно (Рисунок. 4). В него вставляем нужный нам код. При необходимости добавления еще макросов повторяем п. 3.

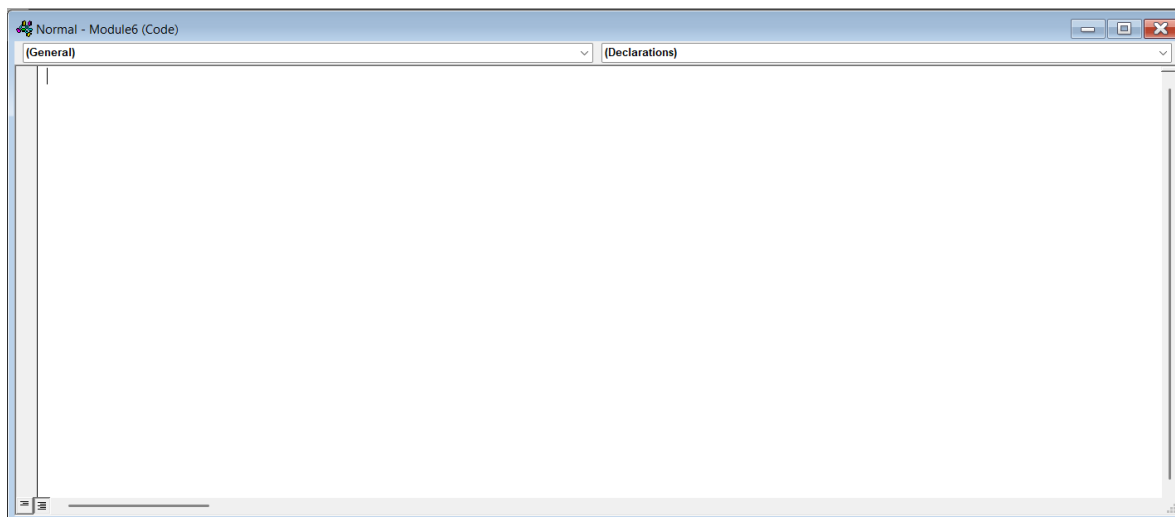


Рисунок 4 – Диалоговое окно

Самостоятельная запись макросов

Макросы можно записывать самостоятельно следующим способом:

1. Открываем на верхней вкладке «Вид», далее – «Макросы» (во втором случае необходимо нажать снизу на надпись, а не на значок макроса) (Рисунок 5)

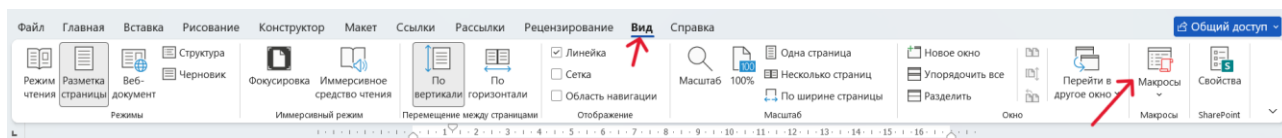


Рисунок 5 – Местонахождение вкладки «Макросы»

2. В появившемся окне нажимаем «Запись макроса...» (Рисунок 6).

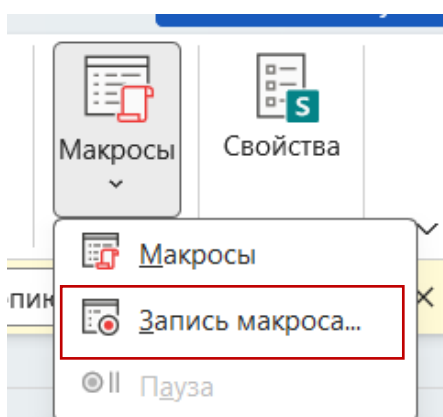


Рисунок 6 – Функционал вкладки «Макросы»

3. Далее вводим любое имя макроса и нажимаем «ОК» (Рисунок 7)

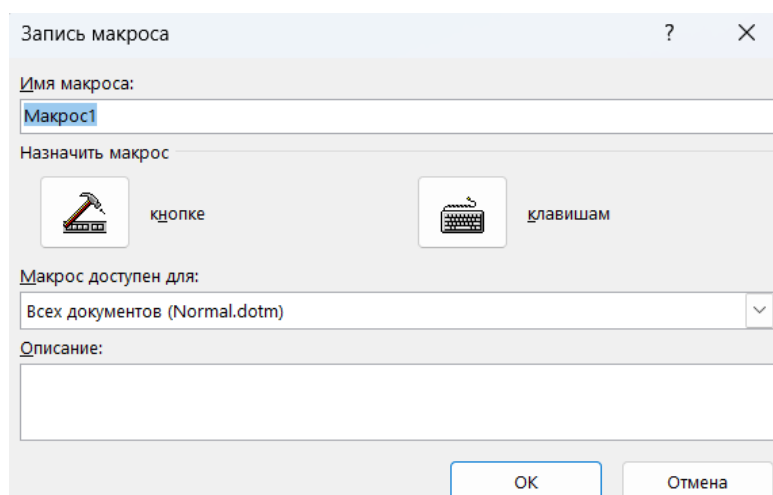


Рисунок 7 – Запись макроса

4. После этого начинается запись макроса. Выставляем вручную необходимое нам оформление документа.

5. Когда вы завершите оформление необходимо остановить запись макроса. Сделать это можно, нажав на ту же кнопку, что и в первом пункте. (Рисунок 8).

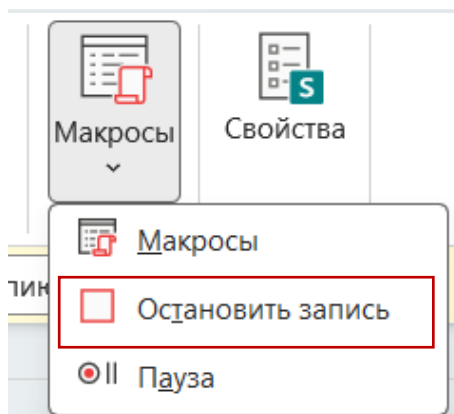


Рисунок 8 – Остановка записи макроса

6. Чтобы макрос, который вы записали выполнялся, нажмите на сам значок макроса и выберите тот макрос, который был записан вами. Нажмите «Выполнить». (Рисунок 9)

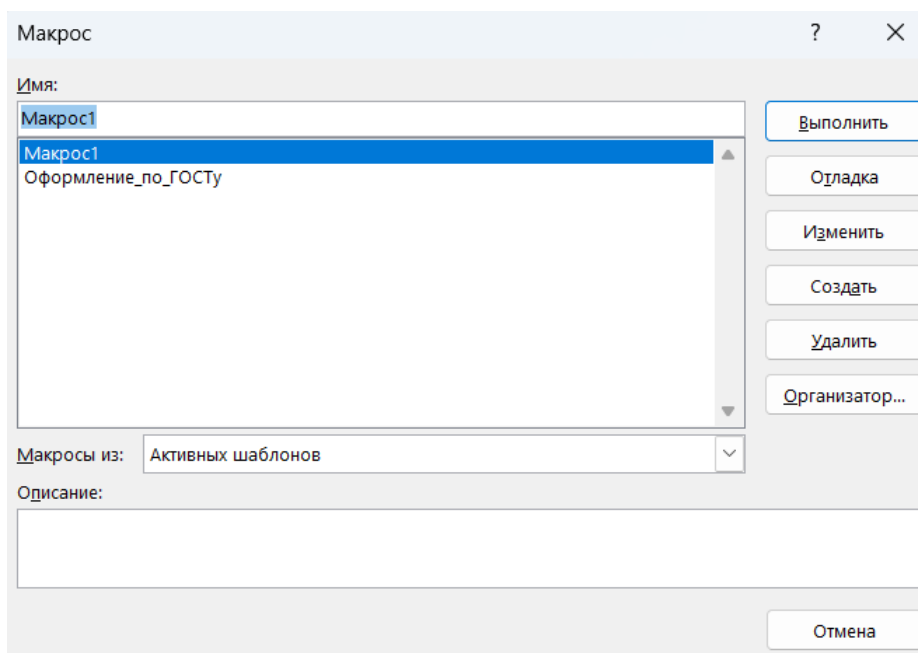


Рисунок 9 – Запись макроса

Оформление текста

Вставляем в окно следующий код и сохраняем

```
Sub FormatFullDocument()  
    With ActiveDocument.PageSetup  
        .TopMargin = CentimetersToPoints(2)  
        .BottomMargin = CentimetersToPoints(2)  
        .LeftMargin = CentimetersToPoints(3)  
        .RightMargin = CentimetersToPoints(1)  
        .HeaderDistance = CentimetersToPoints(1.25)  
        .FooterDistance = CentimetersToPoints(1.25)  
        .PageWidth = CentimetersToPoints(21)  
        .PageHeight = CentimetersToPoints(29.7)  
    End With  
  
    With ActiveDocument.Content  
        .Font.Name = "Times New Roman"  
        .Font.Size = 14  
        .Font.Bold = False  
        .Font.Italic = False  
        .Font.Underline = wdUnderlineNone  
  
        With .ParagraphFormat  
            .LineSpacingRule = wdLineSpacing1pt5  
            .Alignment = wdAlignParagraphJustify  
            .FirstLineIndent = CentimetersToPoints(1.25)  
            .SpaceBefore = 0  
            .SpaceAfter = 0  
            .LeftIndent = 0  
            .RightIndent = 0  
        End With  
    End With  
End Sub
```

Главное не прожимать макросы по несколько раз, иначе макрос будет накладывать оформление друг на друга (будет увеличиваться шрифт, сдвигаться текст и др.)

Оформление таблиц

Вставляем в окно следующий код и сохраняем

```
Sub FormatTables()  
    Dim tbl As Table  
    For Each tbl In ActiveDocument.Tables  
        With tbl.Range  
            .Font.Name = "Times New Roman"  
            .Font.Size = 12  
            With .ParagraphFormat  
                .LineSpacingRule = wdLineSpaceSingle  
                .Alignment = wdAlignParagraphLeft  
                .FirstLineIndent = 0  
            End With  
        End With  
    Next tbl  
End Sub
```

Для того чтобы оформить таблицу, нужно сначала ее создать и добавить в нее текст. Фактически данный код добавляет оформление для текста в таблице. Полностью автоматизировать создание таблиц не получится.

Добавление нумерации

Вставляем в окно следующий код и сохраняем

```
Sub SetupPageNumbers()  
Dim sec As Section  
Set sec = ActiveDocument.Sections(1)  
  
sec.Footers(wdHeaderFooterPrimary).Range.Delete  
  
With sec.Footers(wdHeaderFooterPrimary).Range  
.Fields.Add Range:=sec.Footers(wdHeaderFooterPrimary).Range,  
Type:=wdFieldPage  
.ParagraphFormat.Alignment = wdAlignParagraphCenter  
.Font.Name = "Times New Roman"  
.Font.Size = 14  
End With  
End Sub
```

Макрос добавляет нумерацию с 1-ой страницы. Скрыть номера первых двух страниц через код не получится, так как для этого нужно использовать добавление разделов в документе.

Оформление заголовков

Вставляем код и сохраняем

```
Sub FormatHeading()  
    With Selection.ParagraphFormat  
        .Alignment = wdAlignParagraphLeft  
        .LineSpacingRule = wdLineSpacing1pt5  
        .FirstLineIndent = 0  
        .OutlineLevel = 1  
        .SpaceBefore = 0  
        .SpaceAfter = 0  
    End With  
    With Selection.Font  
        .Name = "Times New Roman"  
        .Size = 14  
        .Bold = True  
    End With  
End Sub
```

Чтобы пользоваться кодом нужно выделить необходимый нам заголовок и выбрать нужный макрос для этого. Заголовок будет по левой стороне.

Оформление содержания

Вставляем следующий код и сохраняем

```
Sub CreateTableOfContents()  
    Dim toc As TableOfContents  
    Set toc = ActiveDocument.TablesOfContents.Add(Range:=Selection.Range, _  
        RightAlignPageNumbers:=True, _  
        UseHeadingStyles:=True, _  
        UpperHeadingLevel:=1, _  
        LowerHeadingLevel:=3)  
    With toc.Range.Font  
        .Name = "Times New Roman"  
        .Size = 14  
    End With  
End Sub
```

Код используем после применения кода из предыдущего пункта. Чтобы создать содержание, необходимо на пустой строке использовать макрос. Пример содержания можно увидеть на рисунке 10.

4	Оформление заголовков	8
5	Оформление содержания.....	9

Рисунок 10 – Пример оформления содержание

Работа с сервисом «Яндекс Алиса Про»

Сервис «Яндекс Алиса Про» представляет собой инновационный инструмент, призванный фундаментально изменить подход к организации самостоятельной учебной деятельности. Его ключевая особенность и главное преимущество для обучения заключается в способности формировать ответы строго на основе предоставленных пользователем материалов (до 25 файлов: лекции, ГОСТы, учебники), что минимизирует фактические ошибки. Вместо ручного поиска по страницам студент получает быстрый локализованный ответ на вопрос.

Сервис автоматически создаёт краткие конспекты, глоссарии или планы ответов, помогает сравнивать разные версии документов и генерировать вопросы для самопроверки. При подготовке курсовых работ он помогает структурировать введение и выделять тезисы, оставляя анализ за студентом. Сложные фрагменты можно попросить объяснить иначе, не выходя за рамки источника. Доступ к проекту можно открыть для коллективной работы. Главный эффект – экономия времени на поиск и обработку информации при сохранении достоверности, а также развитие навыков критической работы с источниками.

Создание проекта

1. Зайдите на alicepro.yandex.ru и выберите раздел «Проекты».
2. Нажмите «Создать проект», дайте ему понятное название (например, «Теория игр»). (Рисунок 11)
3. Внутри проекта вы увидите пустое хранилище материалов и окно чата. (Рисунок 12)

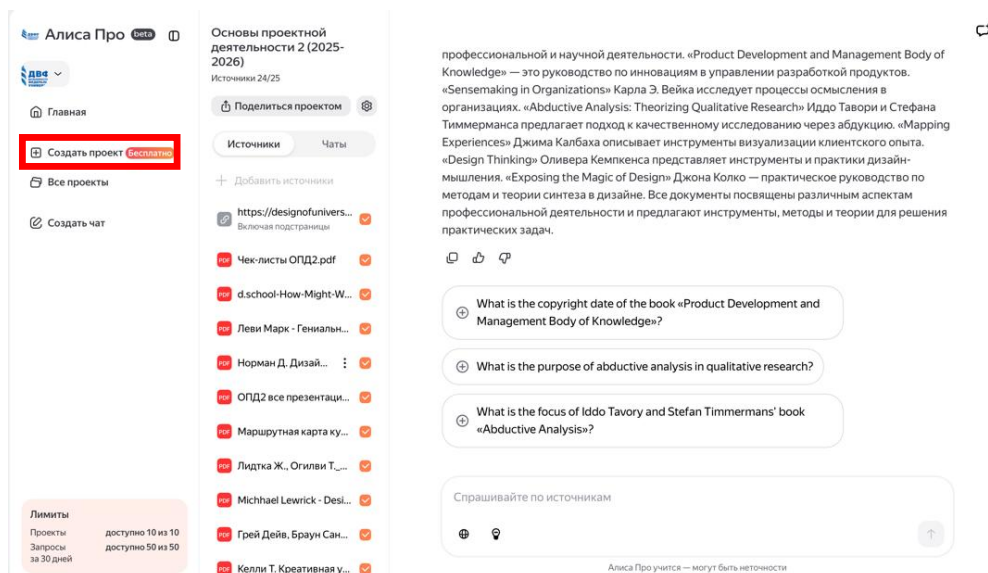


Рисунок 11 – Создание проекта

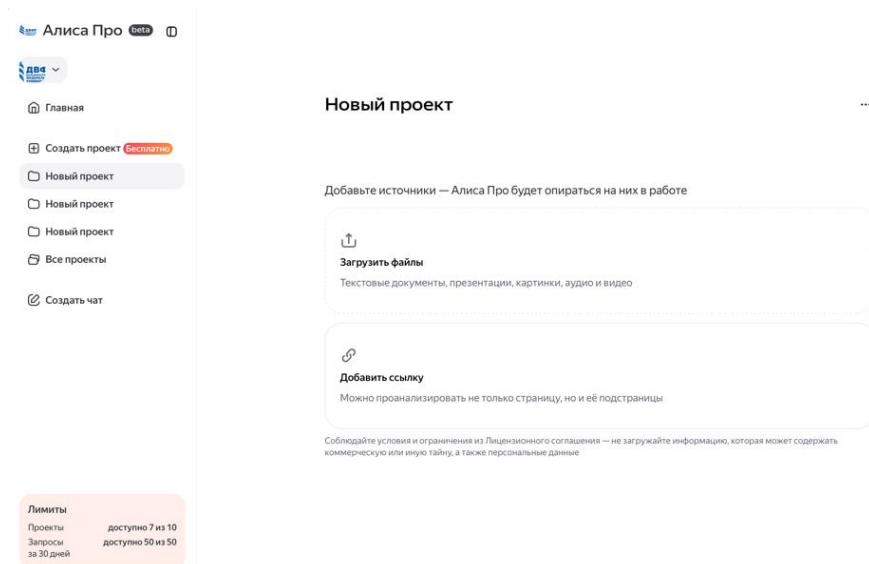


Рисунок 12 – Окно чата

Загрузка материалов

- в проект можно загрузить до 25 файлов (тексты, PDF, презентации, ссылки на веб-страницы);
- нажмите «Загрузить материал» и выберите файлы с компьютера или укажите URL. В процессе пользования также можно добавлять документы. (Рисунок 13).

Важно: нейросеть будет отвечать только по тем документам, которые вы добавили. Чем точнее подобран набор материалов, тем качественнее ответы.

Рекомендация: загружайте лекции, методички, ГОСТы, списки литературы, конспекты по одной теме – всё, что нужно для конкретной учебной задачи.

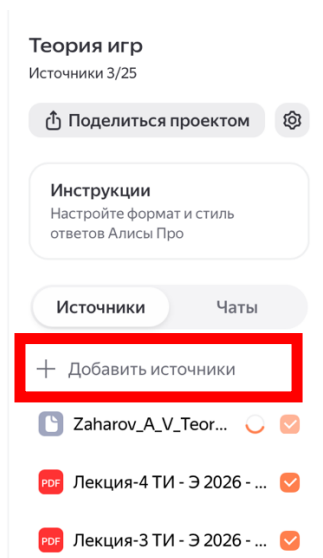


Рисунок 13 – Загруженные файлы

Работа с чатом (запросы)

- в чате пишите вопросы на русском языке;
- нейросеть отвечает, опираясь только на загруженные материалы. Если ответа в них нет, она сообщит об этом (а не будет придумывать);
- для получения более точных ответов используйте приёмы:
 1. Ссылайтесь на конкретный файл: «В файле „Лекция_3.pdf“ найди определение ...».
 2. Просите выжимку: «Составь краткий конспект по теме «...» из всех загруженных материалов».
 3. Сравнивайте: «Сравни требования из ГОСТа 2024 и старого стандарта».
 4. Генерируйте структуру: «Составь план курсовой по теме ..., используя введение из файла „методичка.pdf“».

Лайфхаки и возможности

1. Промпт «показать цитату»: если сомневаетесь в ответе, попросите нейросеть дать прямую цитату из загруженного документа с указанием источника.
2. Работа с большими документами: даже если файл объёмный, сервис ищет только по нему, не нужно вручную перелистывать.
3. Список вопросов: после загрузки материалов можно попросить: *«Сгенерируй 10 вопросов для самопроверки по всем лекциям»* - получите готовый набор.
4. Форматирование: в ответах часто можно попросить оформить результат в виде таблицы, списка или плана.
5. Уточняющие вопросы: если ответ слишком общий, пишите *«Подробнее»* или *«На примере из файла ...»*.

Как поделиться проектом или чатом

1. В интерфейсе проекта есть кнопка «Поделиться». (Рисунок 14)
2. Можно отправить ссылку на проект целиком (с материалами и историей диалогов) коллегам или преподавателю.
3. Уровень доступа настраивается: «только просмотр» или «редактирование» (добавление материалов, вопросов).

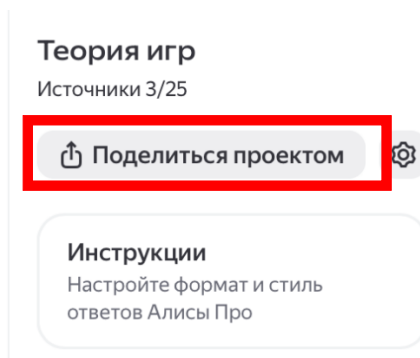


Рисунок 14 – Кнопка поделиться проектом

Работа с ИИ-ассистентом QWEN

QWEN – мощная языковая модель, которая доступна бесплатно через веб-интерфейс. Она умеет анализировать тексты, писать планы, переформулировать в академический стиль, работать с загруженными файлами. Это универсальный помощник для всех этапов учебной работы: от поиска идей до финальной редакции.

QWEN предлагает несколько режимов (быстрый, мышление, автоматический) и выбор моделей под разные задачи. Она хорошо понимает русскоязычные академические тексты и, в отличие от некоторых западных аналогов, стабильно работает. Кроме того, в неё можно загружать файлы прямо в чат – это удобнее, чем копировать текст. К сожалению, с недавнего времени QWEN ввел ограничение на 100 запросов в день.

Базовые принципы взаимодействия

Работа с ИИ-агентом осуществляется через веб-интерфейс (Рисунок 15). Основные элементы управления:

1. Левая панель навигации:

- «Новый чат» – создание новой сессии диалога;
- «Поиск в чатах» – поиск по истории взаимодействий;
- «Проекты» – организация тематических рабочих пространств;
- «Coder» – специализированный режим для работы с кодом.

2. Центральная область:

- поле ввода запросов – основная зона для формулирования задач;
- кнопка отправки – запуск генерации ответа.

Рекомендация: для каждого учебного проекта создавайте отдельный чат или используйте функцию «Проекты» для систематизации материалов.

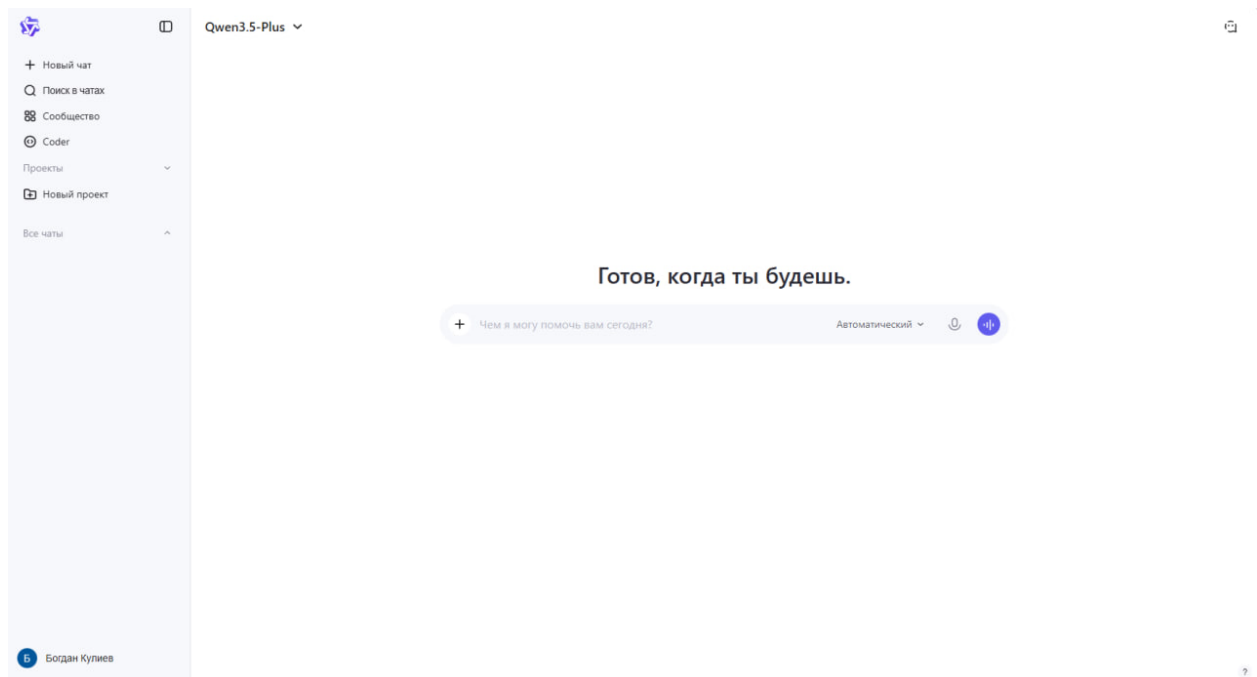


Рисунок 15 - Основной интерфейс ИИ-ассистента QWEN.

Выбор режима генерации

Интерфейс QWEN предоставляет возможность выбора режима работы (Рисунок 16). Доступны три опции: «Автоматический» (система самостоятельно определяет оптимальный режим), «Мышление» (расширенный анализ задачи с пошаговой аргументацией), «Быстрый» (оперативная генерация ответа без детализации)

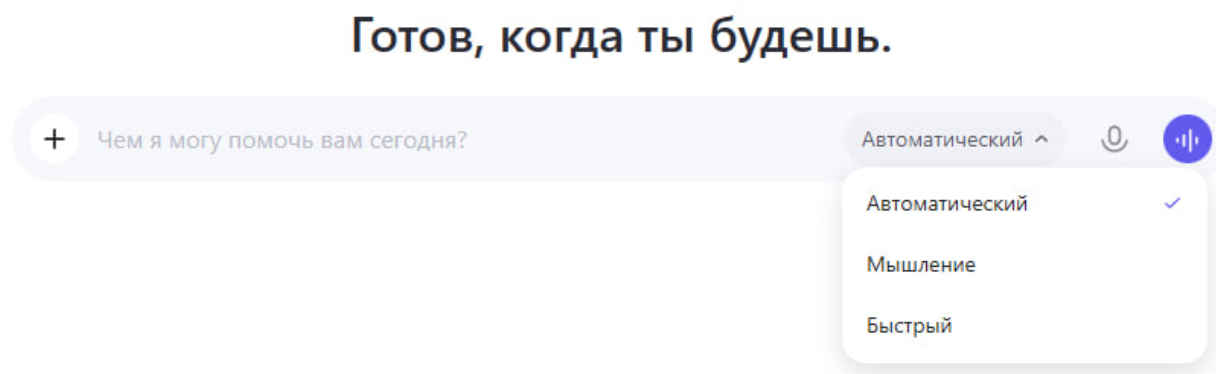


Рисунок 16 - Меню выбора режима генерации ответов.

Рекомендация: по умолчанию используйте «Автоматический» режим. Переключайтесь на «Мышление» при подготовке аналитических разделов курсовых работ. Режим «Быстрый» оптимален для оперативных уточнений.

Выбор языковой модели

QWEN функционирует на базе семейства языковых моделей Qwen3.5. Пользователю доступен выбор конкретной модели в зависимости от характера учебной задачи (Рисунок 17): Qwen3.5-Plus (универсальная модель для текстовых и мультимодальных задач), Qwen3.5-Flash (высокопроизводительная модель), Qwen3.5-Omni-Plus (наиболее мощная мультимодальная модель), а также специализированные версии с открытым исходным кодом (397B-A17B, 122B-A10B) и более старые модели.

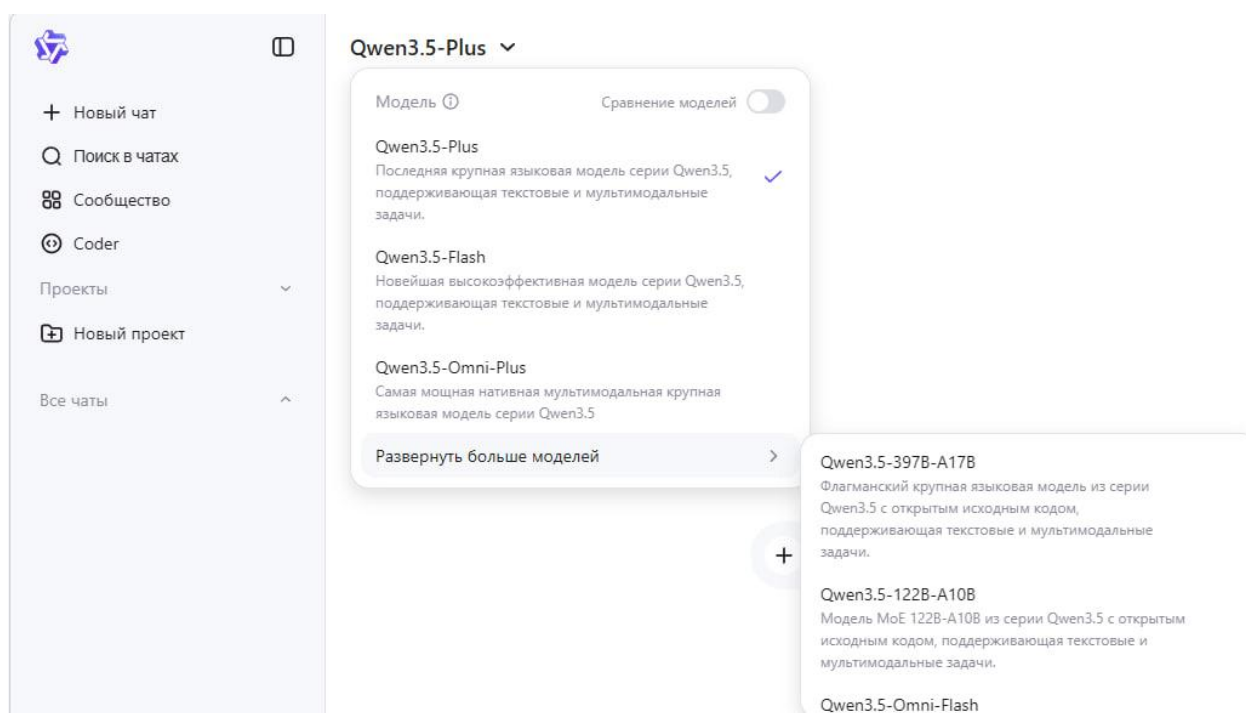


Рисунок 17 - Меню выбора языковой модели Qwen3.5.

Рекомендация: для стандартных образовательных целей достаточно модели Qwen3.5-Plus. Переключение на другие модели осуществляйте только при наличии специфических требований к задаче.

Структура эффективного запроса

Для получения качественных результатов рекомендуется использовать структуру промта, представленную в Таблице 1.

Таблица 1 – Структура эффективного запроса

Элемент	Описание	Пример формулировки
Контекст	Указание дисциплины, уровня подготовки, цели	«В рамках курса "Методология научных исследований", уровень – бакалавриат, 2 курс»
Задача	Четкое описание требуемого действия	«Необходимо составить план аналитического обзора по теме...»
Ограничения	Требования к объему, стилю, источникам	«Объем – до 1500 знаков, стиль – академический, без использования непроверенных источников»
Формат	Предпочтительная форма представления ответа	«Ответ представить в виде нумерованного списка с краткими пояснениями»

Пример корректно сформулированного запроса в интерфейсе QWEN (Рисунок 18):

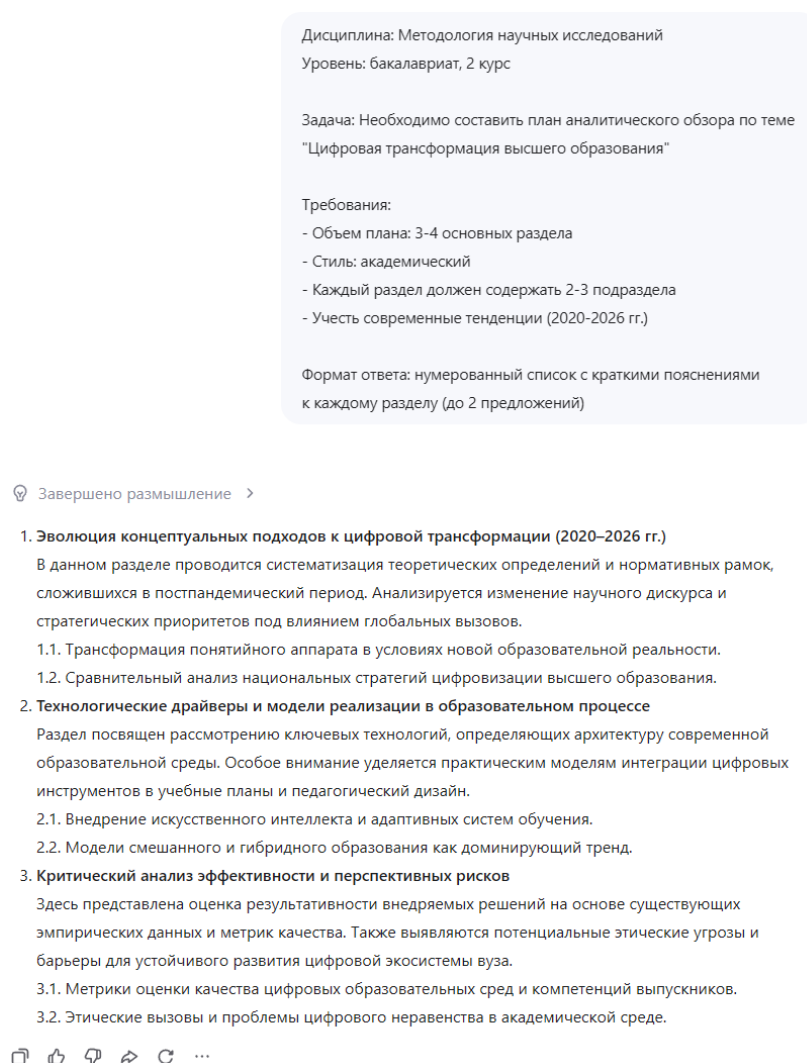


Рисунок 18 - Пример корректно сформулированного запроса в интерфейсе QWEN.

Работа с учебными материалами

Рекомендации по загрузке документов:

1. Материалы следует подготавливать тематическими группами (3–5 файлов на одну сессию).
2. Каждый файл должен иметь понятное наименование, отражающее содержание.
3. При загрузке источников необходимо кратко обозначить цель их анализа (Рисунок 19).
4. После обработки партии материалов целесообразно запросить у ассистента резюме или структурированный вывод.

Пример последовательности действий (Рисунок 20):

1. Загрузка: «Предоставляю 3 научных статьи по теме "Цифровая трансформация образования"».
2. Запрос: «Выделите ключевые подходы, описанные в данных источниках».
3. Уточнение: «Сравните представленные методики по критериям: применимость, ресурсоемкость».
4. Фиксация результата: «Сформируйте таблицу сравнения для включения в курсовую работу».

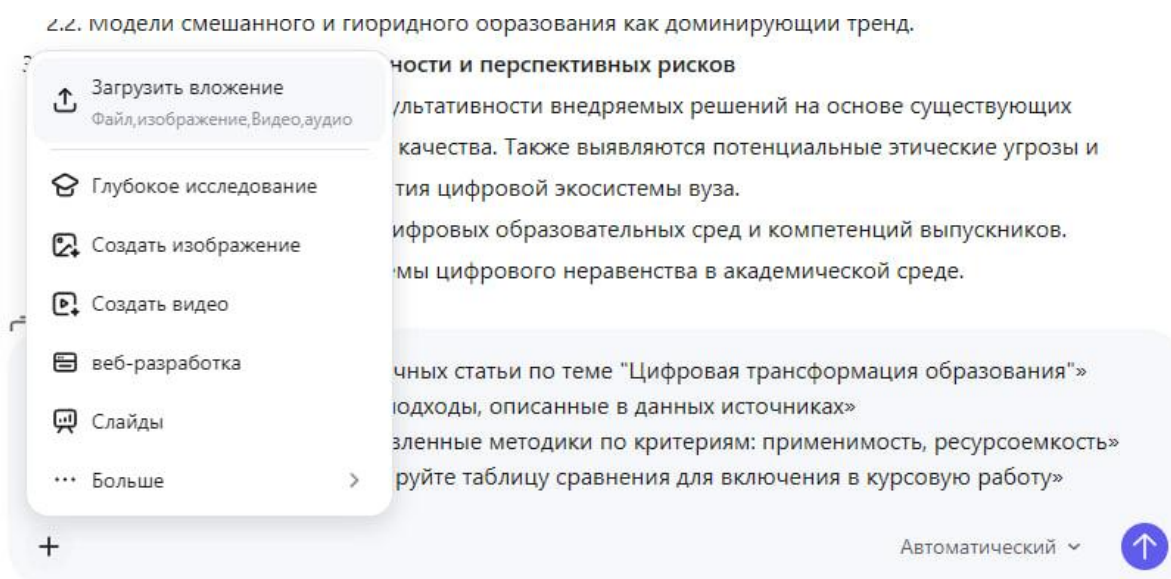


Рисунок 19 - Возможность загружать вложения в ассистента.

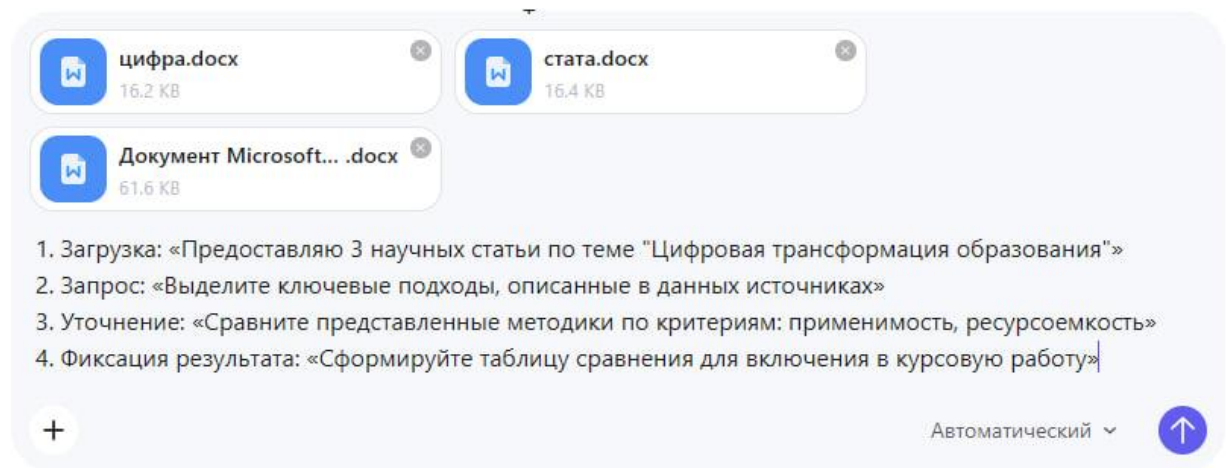


Рисунок 20 - Процесс загрузки файлов и соответствующего запроса.

Оптимизация запросов

При использовании нейросети Qwen качество получаемого результата напрямую зависит от того, насколько точно и подробно сформулирован запрос. Если запрос слишком общий, краткий или не содержит конкретной цели, нейросеть может дать поверхностный, неполный или слишком общий ответ. Примеры «плохих» и «хороших» запросов приведены в Таблице 2.

Таблица 2 - Рекомендации по повышению эффективности запросов.

Не рекомендуется	Рекомендуемая формулировка
«Расскажи про маркетинг»	«Предоставьте структурированный обзор основных концепций маркетинга для подготовки к семинару по дисциплине "Основы менеджмента"»
«Проверь текст»	«Выполните лингвистический анализ текста на соответствие академическому стилю: укажите стилистические ошибки, тавтологии, нарушения логики изложения»
«Найди источники»	«Сформулируйте поисковые запросы и ключевые слова для подбора научной литературы по теме X в базах данных eLibrary, Scopus»

При работе с Qwen рекомендуется:

- формулировать запросы максимально чётко и понятно;
- указывать конкретную задачу, которую необходимо выполнить;
- задавать желаемую структуру ответа;
- при необходимости разбивать большую задачу на несколько последовательных запросов;
- использовать уточняющие запросы для доработки результата;
- внимательно проверять полученную информацию перед использованием в учебной работе.

Модели организации сессий

1. Модель «Единый контекст». Применяется для последовательной работы над одним проектом. Преимуществом выступает сохранение истории диалога, целостность контекста.

Рекомендации: использовать служебные маркеры для навигации: [СТРУКТУРА], [АНАЛИЗ], [РЕДАКТУРА], [ПРОВЕРКА].

2. Модель «Разделение по задачам»

Предполагает использование отдельных чатов для различных типов деятельности (Таблица 3).

Таблица 1 - Типы учебных сессий с ИИ-ассистентом QWEN.

Тип сессии	Функция ассистента	Тип загружаемых материалов
Аналитическая	Обработка источников, выявление закономерностей	Научные статьи, статистические данные
Структурная	Разработка планов, схем, логики изложения	ТЗ, требования, примеры работ
Редакционная	Корректировка текста, проверка стиля и грамматики	Черновики, методические указания

На рисунке 21 представлен пример организации рабочего пространства посредством создания тематических чатов. Каждый чат соответствует определенному типу учебной деятельности:

1. Чат «Анализ источников» – предназначен для обработки научной литературы, выявления ключевых концепций и закономерностей.

2. Чат «Структура работы» – используется для разработки плана исследования, логической схемы изложения материала.

3. Чат «Редактура» – применяется для проверки черновиков на соответствие академическим стандартам, исправления стилистических и грамматических ошибок.

Данная организация позволяет сохранять контекст для каждого направления работы, избегать смешения различных учебных задач, оперативно возвращаться к предыдущим этапам работ и систематизировать историю взаимодействий с ассистентом.

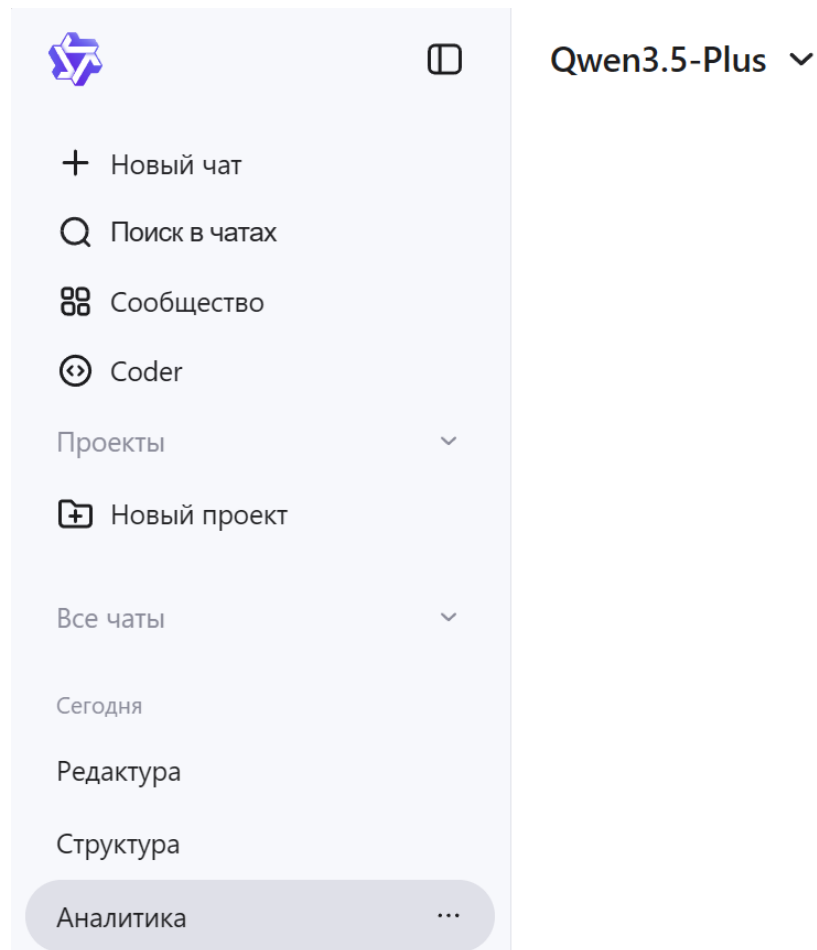


Рисунок 21 - Пример организации нескольких чатов для разных задач.

Рекомендация: присваивайте чатам понятные названия, отражающие их функциональное назначение, для удобства навигации по истории диалогов.

Совместный доступ

Платформа QWEN позволяет предоставлять доступ к чату другим пользователям через ссылку. Для этого нажмите кнопку «Поделиться» прямо под предоставленным агентом ответом (Рисунок 22-23).

💡 Завершено размышление >

Для того чтобы проанализировать статьи по цифровой трансформации образования и выделить ключевые тенденции, мне необходим доступ к самим материалам.

Пожалуйста, предоставьте:

- ◆ **Ссылки на статьи (URL)** — если они опубликованы онлайн, или
- ◆ **Текст статей** — вы можете скопировать и вставить их прямо в чат, или
- ◆ **Названия и источники** — если это известные публикации, я попробую найти их самостоятельно.

Как только я получу доступ к материалам, я смогу:

- ✅ Провести сравнительный анализ трёх источников
- ✅ Выделить общие и уникальные тренды цифровой трансформации
- ✅ Структурировать выводы по категориям (технологии, педагогика, управление, инклюзия и др.)
- ✅ Подготовить краткое резюме с практическими инсайтами

Жду ваши материалы! 📁 ✨

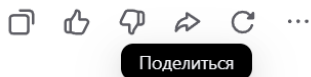


Рисунок 22 - Кнопка "Поделиться"

Данная функция рекомендуется для:

- организации групповой работы над проектами;
- демонстрации результатов преподавателю;
- обмена лучшими практиками между обучающимися.

Перед предоставлением доступа убедитесь в отсутствии конфиденциальной информации в истории диалога.

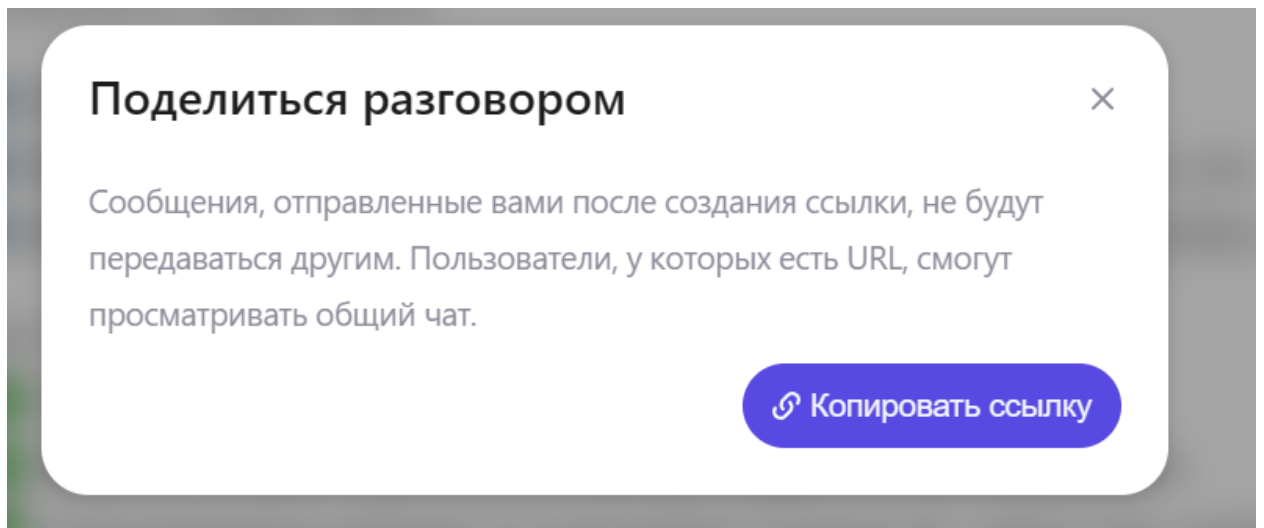


Рисунок 23 - Дальнейшие инструкции после нажатия кнопки "Поделиться"

Работа через Visual Studio Code

До сих пор мы работали в Word, в веб-сервисах. Но для серьёзной учебной деятельности (написание больших отчётов, анализ данных, создание презентаций, систематизация материалов) нужен единый инструмент, который объединяет текстовый редактор, файловый менеджер, среду для кода и нейросети. Таким инструментом является Visual Studio Code (VS Code).

Visual Studio Code является удобным и современным инструментом для учебной и профессиональной работы с файлами, текстами и проектами. Программа распространяется бесплатно, быстро запускается и стабильно работает даже на не самых мощных компьютерах. В отличие от обычных текстовых редакторов, VS Code позволяет удобно работать сразу с большим количеством файлов и папок, быстро находить нужную информацию по всему проекту и использовать встроенные инструменты для структурирования материалов. Дополнительным преимуществом является возможность установки расширений, которые добавляют поддержку работы с документами, таблицами, PDF-файлами, нейросетями и анализом данных. Благодаря этому Visual Studio Code может использоваться как единая рабочая среда для подготовки отчётов, анализа информации, написания текстов и организации учебных материалов.

Установка Visual Studio Code

1. Переход на официальный сайт

Откройте любой браузер (например, Chrome, Edge, Firefox) введите запрос «vs code» и перейдите на официальный сайт Visual Studio Code (Рисунок 24).

Либо воспользуйтесь ссылкой ниже для перехода на официальный сайт:
<https://code.visualstudio.com/>

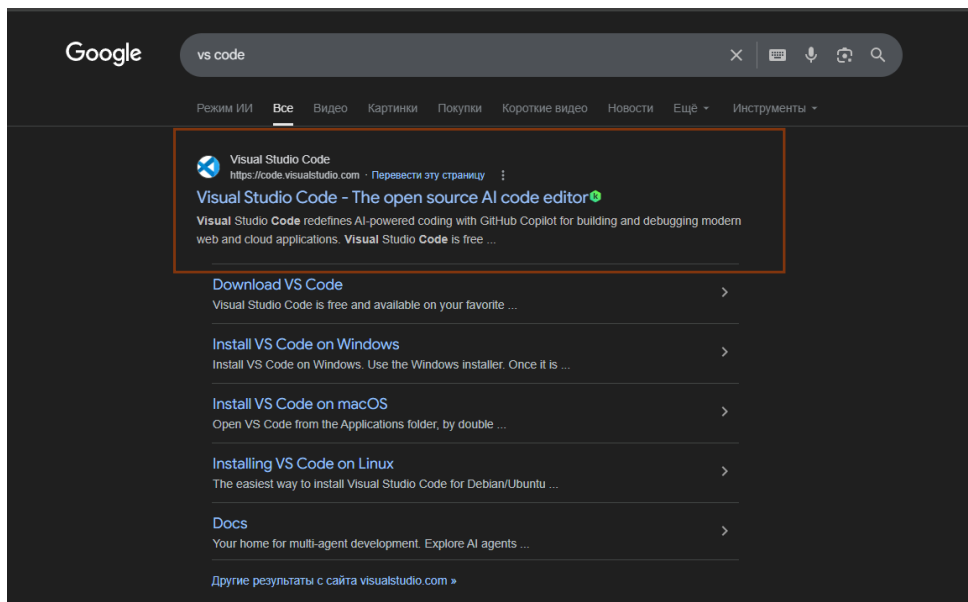


Рисунок 24 – Запрос в браузер

2. Скачивание установочного файла

На главной странице нажмите кнопку Download (Рисунок 25). Сайт автоматически предложит версию под вашу операционную систему (Windows, macOS или Linux).

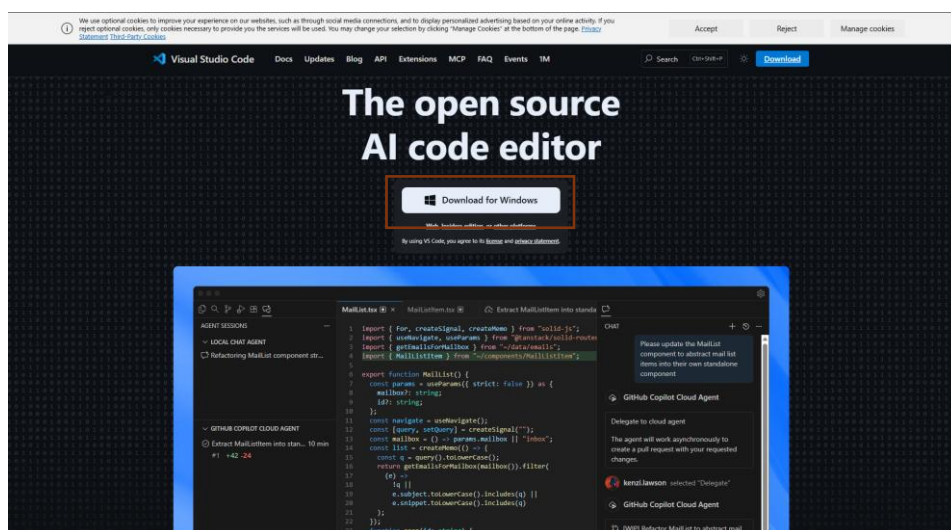


Рисунок 25 – Официальный сайт

После нажатия начнётся загрузка установочного файла (Рисунок 26).

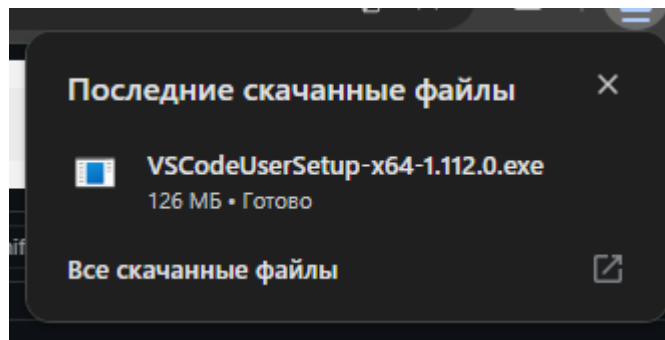


Рисунок 26 – Установочный файл

3. Запуск установщика

После завершения загрузки можно нажать на скаченный файл либо открыть папку Загрузки (Downloads), найти скаченный файл и запустить установку.

4. Принятие лицензионного соглашения

В открывшемся окне установщика:

- выберите пункт I accept the agreement (Я принимаю условия соглашения);
- нажмите кнопку Next (Далее) пока не появится кнопка установки

(Рисунок 27).

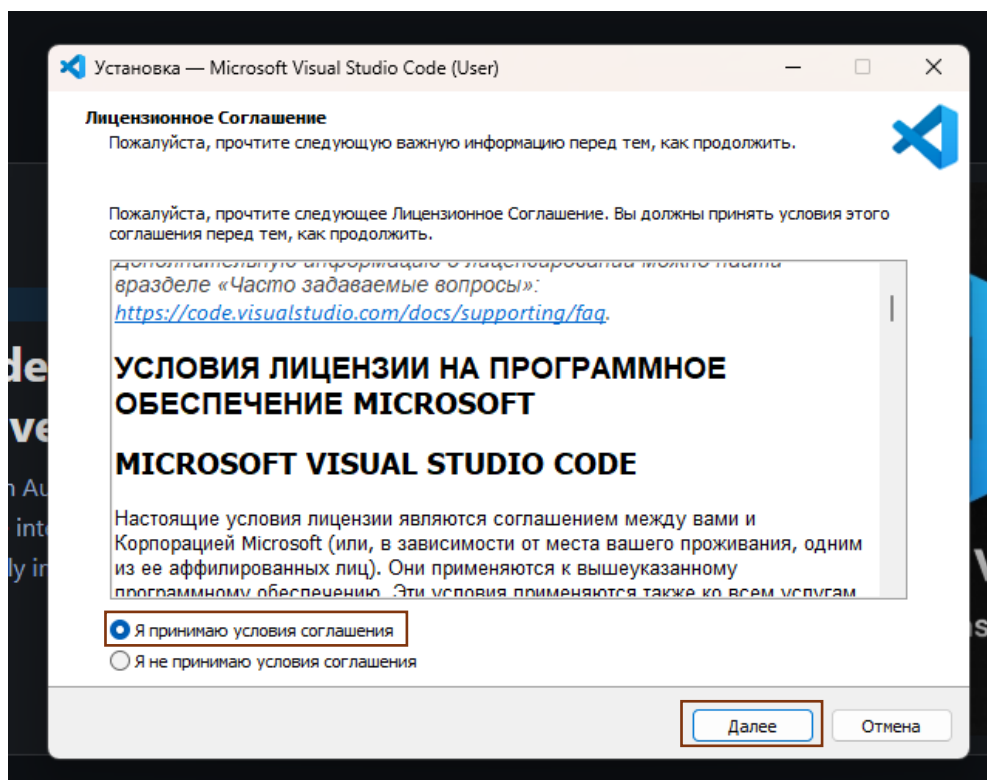


Рисунок 27 – Лицензионное соглашение

5. Выбор папки установки

Установщик предложит папку для установки программы. Рекомендуется оставить путь по умолчанию и нажать Next (Далее).

6. Выбор дополнительных параметров

На этом этапе можно включить полезные опции:

- add to PATH – позволяет запускать VS Code из командной строки;
- create a desktop icon – создать значок на рабочем столе;
- add "Open with Code" – добавить пункт в контекстное меню.

Рекомендуется отметить все эти пункты, затем нажать Next

7. Установка программы

Нажмите кнопку Install (Установить), чтобы начать установку. Дождитесь завершения процесса.

8. Завершение установки и запуск

После окончания установки нажмите Finish (Завершить) и программа автоматически запустится (если стоит галочка) (Рисунок 28).

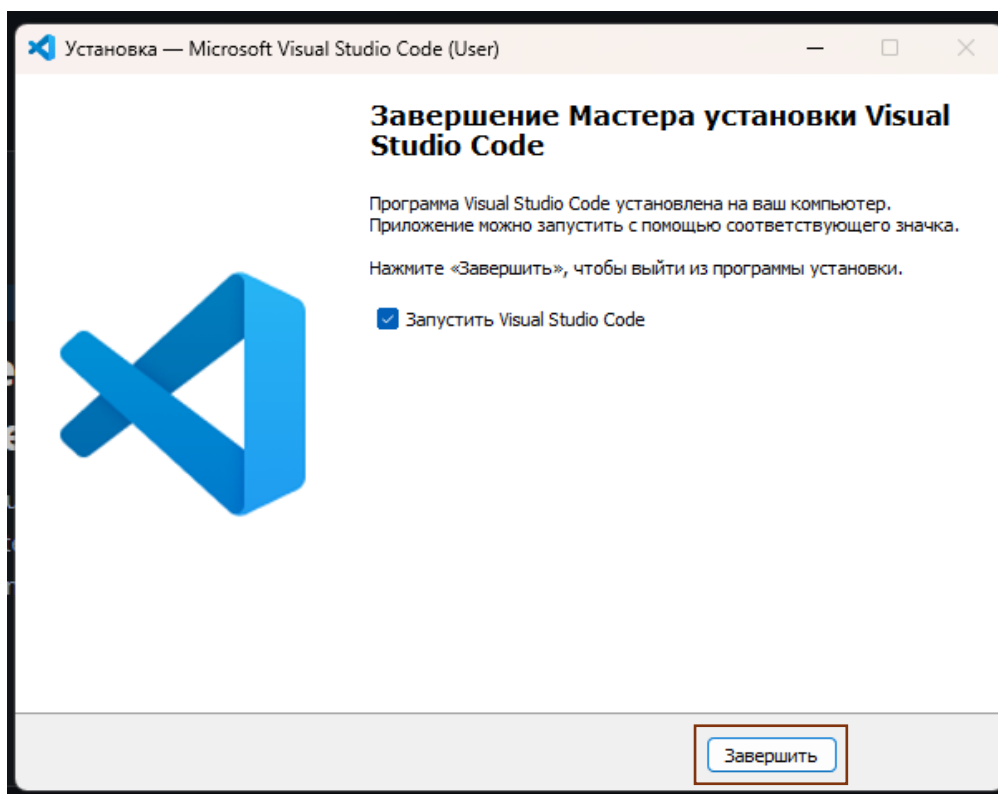


Рисунок 28 – Завершение установки

9. Первая проверка

После запуска убедитесь, что программа открылась корректно (Рисунок 29). При необходимости можно установить расширения (например, для Python, JavaScript и др.) через вкладку Extensions.

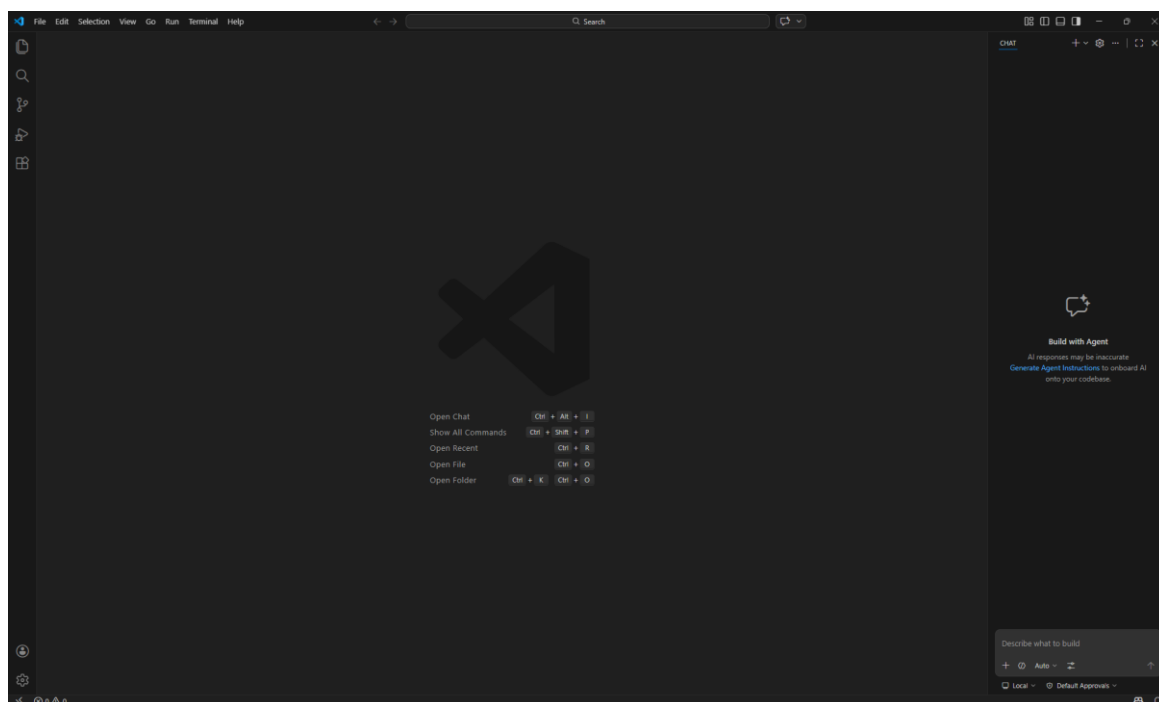


Рисунок 29 – Интерфейс

Привязка VS Code к GitHub

Git – это программа, которую называют системой контроля версий. Её основная задача – отслеживать изменения в файлах. Вы словно создаёте «снимки» вашего проекта на разных этапах. Git хранит всю историю правок, поэтому в любой момент можно увидеть, что изменилось, и при необходимости вернуться к более ранней версии. Это особенно важно для больших работ, где легко запутаться в черновиках.

GitHub – это онлайн-сервис для хранения Git-репозитория. Проще говоря, это облачное хранилище для ваших проектов с удобным веб-интерфейсом. Но GitHub – это не просто диск для файлов.

Важно понимать разницу: Git – это сама технология, а GitHub – это веб-сайт, который использует эту технологию. Вместе они помогают вам, во-первых, надёжно сохранять историю ваших учебных проектов, во-вторых, легко обмениваться файлами с одноклассниками или преподавателем, а в-третьих, получить полезный навык, который ценится в любой IT-сфере.

Итак, разберем как зарегистрироваться на GitHub и привязать к VS Code.

Регистрация на GitHub

1. Откройте браузер и перейдите на официальный сайт: <https://github.com/>.
2. Нажмите на кнопку «Sign up» (Зарегистрироваться) в правом верхнем углу.
3. Введите адрес вашей электронной почты, придумайте пароль и имя пользователя (username) (Рисунок 30).

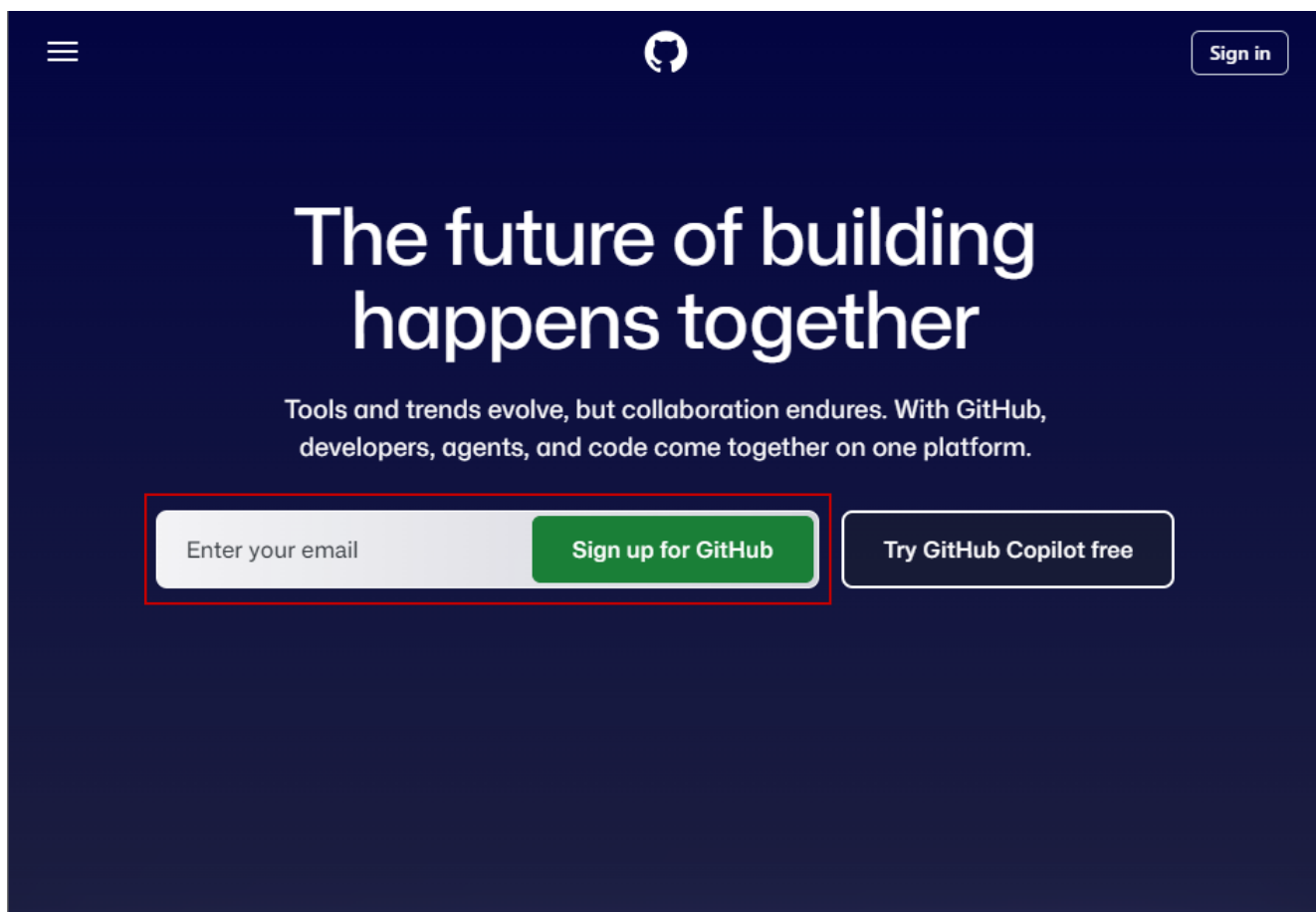


Рисунок 30 – Официальный сайт GitHub

4. GitHub отправит на вашу почту код подтверждения. Введите его в соответствующее поле на сайте. Подтверждение адреса электронной почты – обязательный шаг. Без него вы не сможете выполнять многие действия, например, создавать репозитории.
5. После подтверждения почты вы попадёте в личный кабинет GitHub. Регистрация завершена!

Установка Git на компьютер

Иногда может быть недостаточно просто зарегистрироваться в GitHub, поэтому понадобится Git.

1. Скачайте установщик Git для вашей операционной системы с официального сайта: <https://git-scm.com/downloads>.
2. Запустите скачанный файл и следуйте инструкциям установщика. Настройки по умолчанию подходят для большинства пользователей. После установки перезагрузите VS Code.

Привязка GitHub к VS Code

Теперь нужно «познакомить» ваш редактор с вашим аккаунтом на GitHub.

1. Откройте Visual Studio Code.
2. Посмотрите на левую боковую панель. В самом низу находится иконка с изображением шестерёнки или ваш аватар (если он уже загружен). Это кнопка Accounts (Управление аккаунтами). Нажмите на неё.
3. В появившемся меню выберите пункт «Sign in to GitHub...» (Войти в GitHub) (Рисунок 31).

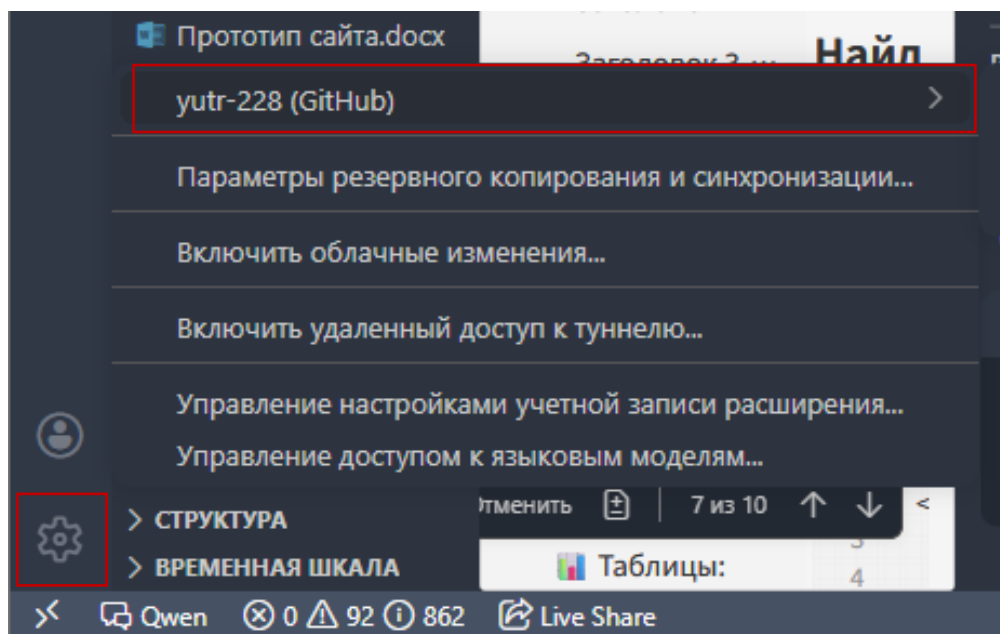


Рисунок 31 – Управление аккаунтом в VS Code

4. Браузер откроется автоматически с запросом на авторизацию. Нажмите зелёную кнопку «Authorize Visual-Studio-Code» (Авторизовать Visual Studio Code).

5. После подтверждения появится окно с предложением открыть ссылку в приложении. Нажмите «Open Link». VS Code автоматически активируется, и в нижнем левом углу появится имя вашего GitHub-аккаунта. Привязка выполнена.

Установка расширений в Visual Studio Code

Для расширения функциональных возможностей Visual Studio Code необходимо установить дополнительные расширения (Extensions).

1. Запустите программу Visual Studio Code.
2. В левой части окна найдите панель инструментов и нажмите на значок Extensions (Расширения) (иконка в виде четырёх квадратов на Рисунке 32).

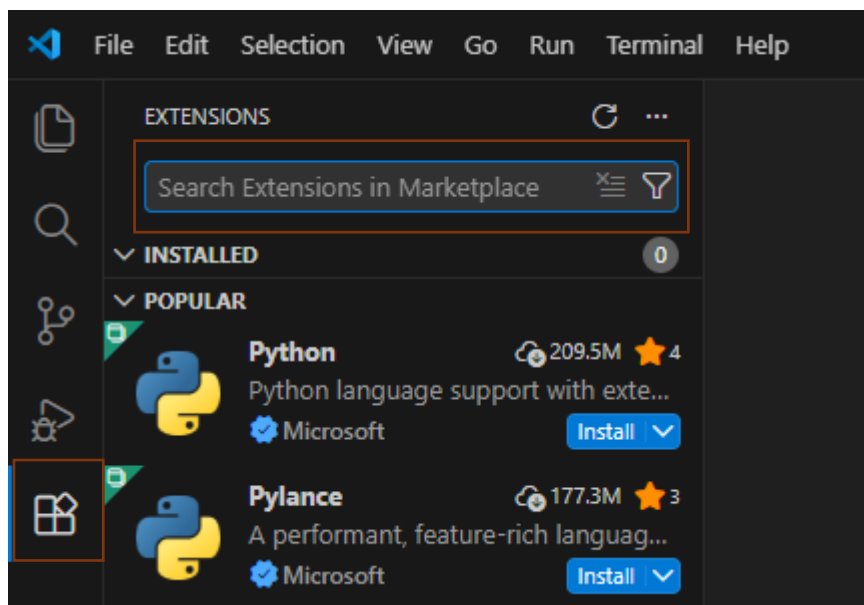


Рисунок 32 – Поисковик расширений

3. В открывшемся окне в строке поиска введите название нужного расширения.
4. В списке результатов выберите требуемое расширение.
5. Нажмите кнопку Install (Установить).
6. Дождитесь завершения установки. После установки расширение активируется автоматически.

Рекомендуемые расширения

Для работы с файлами, документами и анализом данных рекомендуется установить расширения, представленные в Таблице 4. При необходимости можно установить дополнительные расширения в зависимости от вашей сферы работы. Важно отметить, что при установке некоторых расширений требуется совершить ряд действий, указанных при скачивании расширения.

Таблица 4 – Расширения

№	Название расширения	Описание	Фото
1	Office Viewer	Используется для просмотра файлов форматов .docx, .xlsx, .pptx непосредственно в Visual Studio Code без открытия сторонних программ	
2	Jupyter	Предназначено для анализа данных, работы с таблицами и создания интерактивных отчётов с возможностью выполнения ячеек	
3	Markdown All in One	Используется для написания и форматирования текстовых документов, отчётов и заметок с поддержкой Markdown	
4	Excel Viewer	Обеспечивает удобный просмотр и навигацию по таблицам Excel внутри среды разработки	
5	Qwen	Используется для анализа информации, генерации текстов и получения пояснений с помощью технологий искусственного интеллекта	
6	vscode-icons	Добавляет визуальные иконки для различных типов файлов, упрощая навигацию по проекту	
7	File Utils	Предназначено для удобного управления файлами и папками (переименование, перемещение, создание структуры)	
8	Code Spell Checker	Выполняет проверку орфографии в текстовых файлах и документах	
9	Bookmarks	Позволяет ставить закладки в файлах для быстрого перехода между важными местами	
10	Better Comments	Делает комментарии более наглядными за счёт цветового выделения	
11	PDF Viewer	Позволяет открывать и просматривать PDF-файлы прямо в VS Code	
12	Live Share	Обеспечивает совместную работу над файлами и проектами в реальном времени	
13	Russian Language Pack for Visual Studio Code	Делает интерфейс Visual Studio Code на русском языке	

Окончание Таблицы 4

14	Book Reader	Поддерживает PDF, FB2, EPUB, MOBI и другие форматы электронных книг	
15	PDF Viewer	Специализированный просмотрщик для PDF файлов	
16	GitHub Copilot Chat	Нейросеть, адаптированная для работы непосредственно в файловой среде Visual Studio Code	

После установки расширений рекомендуется:

- убедиться, что они отображаются во вкладке Extensions (Расширения);
- перезапустить Visual Studio Code.

Создание папки для работы с файлами проекта

Для удобной организации файлов рекомендуется создать отдельную папку проекта перед началом работы в Visual Studio Code.

1. Откройте проводник операционной системы (Проводник в Windows или Finder в macOS).
2. Перейдите в директорию, где планируется хранение проекта (например, «Документы» или «Рабочий стол»).
3. Нажмите правой кнопкой мыши в пустой области. Выберите пункт «Создать» → «Папку» (Рисунок 33).

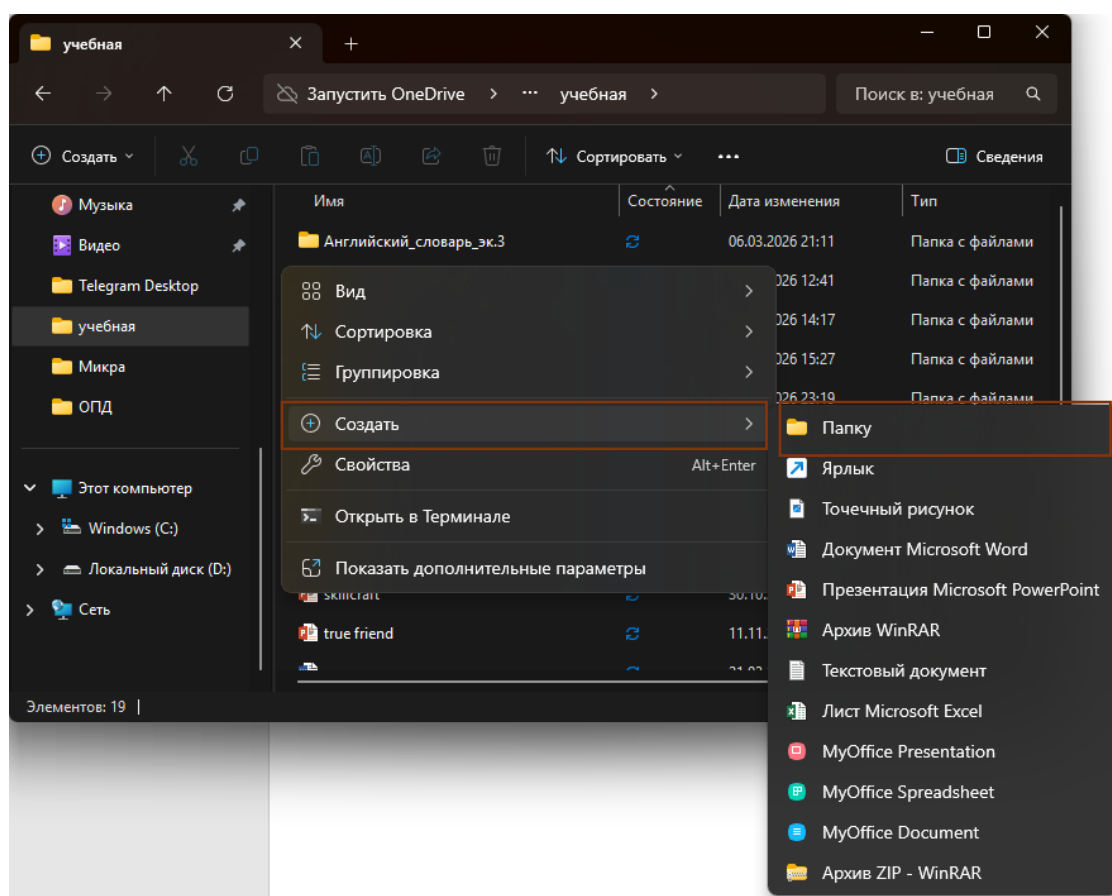


Рисунок 33 – Создание папки

4. Введите название папки. Рекомендуется использовать понятное имя, отражающее содержание проекта (например, Отчёт_по_практике, Анализ_данных, Проект_1).
5. Запустите Visual Studio Code. В главном меню выберите: File - Open Folder (Файл → Открыть папку). В открывшемся окне найдите созданную папку (Рисунок 34).

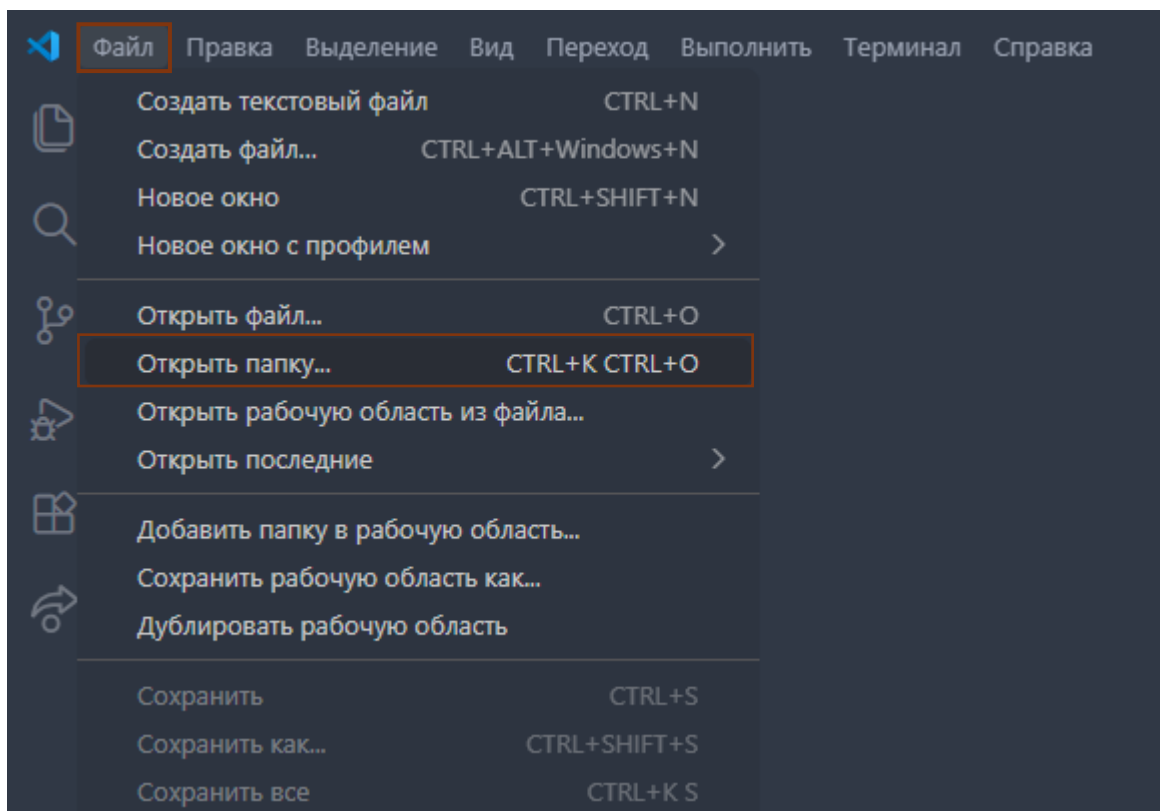


Рисунок 34 – Открытие папки

6. Выделите папку и нажмите кнопку «Выбрать папку». После выполнения действий папка отобразится в левой части окна (Explorer). В неё можно добавлять файлы и создавать структуру проекта (Рисунок 35).

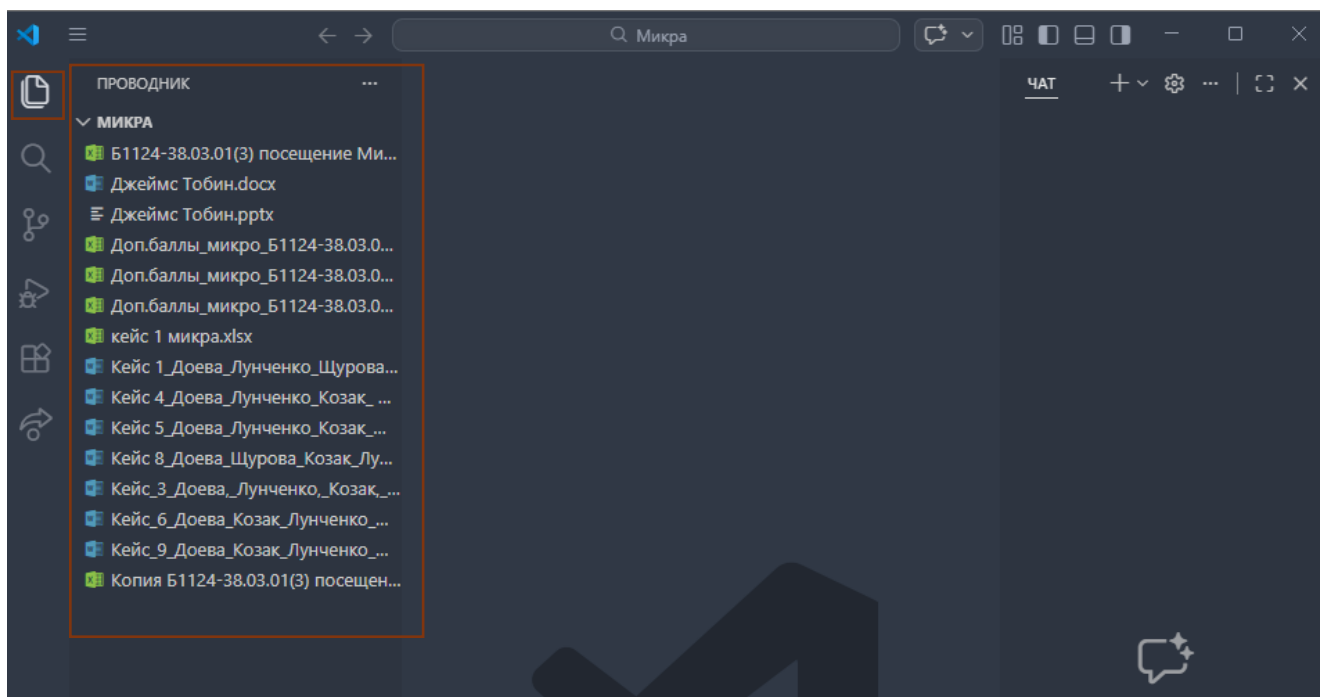


Рисунок 35 – Папка проекта

Рекомендации:

- храните все файлы проекта в одной папке;
- создавайте подпапки (например: *Документы, Данные, Презентации*);
- используйте понятные и единообразные названия файлов.

Работа с файлами в Visual Studio Code

После создания папки проекта пользователь может приступить к работе с различными учебными файлами. Visual Studio Code позволяет удобно организовать материалы, открывать документы и быстро переходить между ними.

Открытие файлов

Для открытия файла необходимо:

- открыть ранее созданную папку проекта через меню: File - Open Folder;
- в левой части окна (панель Explorer) выбрать нужный файл;
- щёлкнуть по нему левой кнопкой мыши (Рисунок 36).

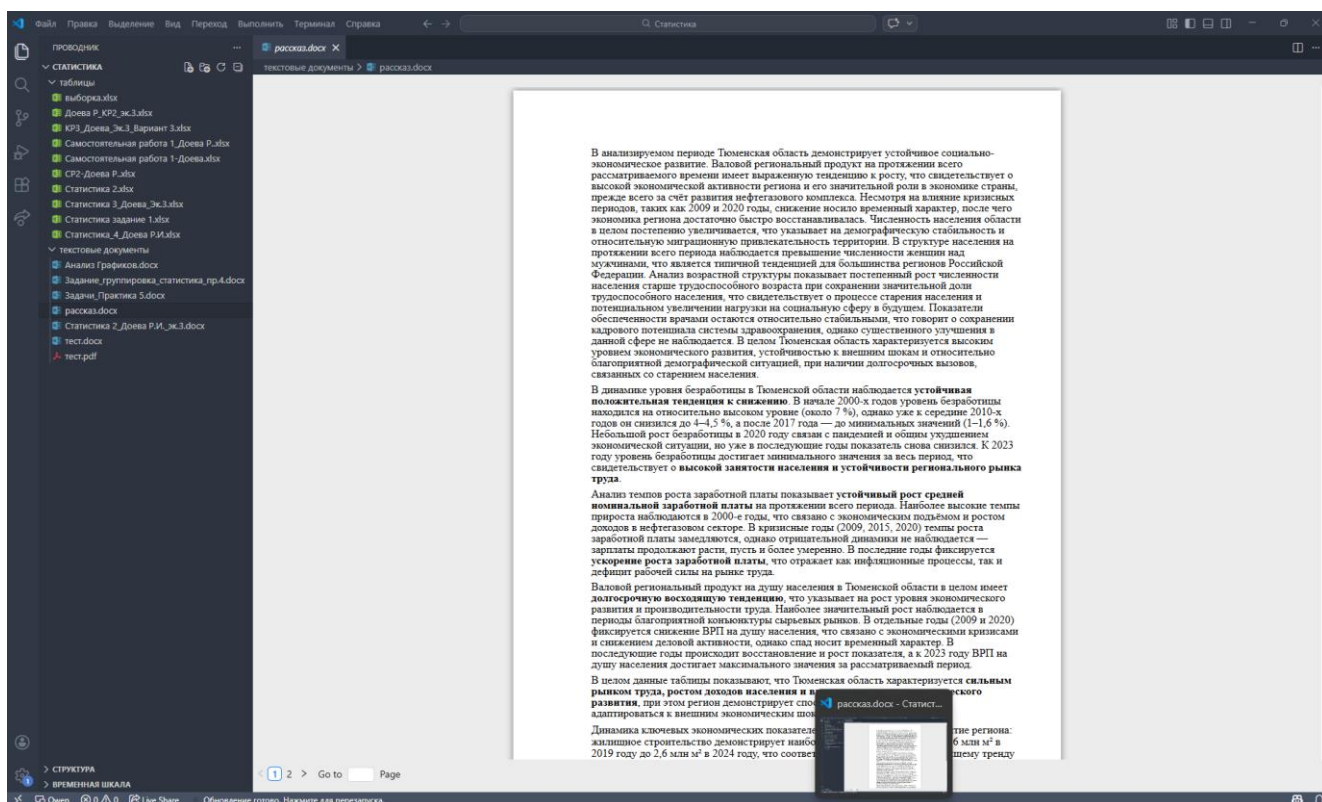


Рисунок 36 – Открытие файла

В Visual Studio Code можно работать с различными типами файлов, например:

- текстовые документы;
- таблицы;
- презентации;
- PDF-файлы;

- заметки и черновики.

Организация учебных материалов

Для удобства рекомендуется распределять файлы по тематическим папкам, например:

- документы – отчёты, рефераты, текстовые материалы;
- презентации – материалы для выступлений;
- данные – таблицы, результаты опросов, числовые данные;
- изображения – схемы, рисунки, иллюстрации;
- черновики – рабочие записи и промежуточные версии.

Такая структура позволяет быстрее находить нужные материалы и хранить их в одном месте.

Работа с несколькими файлами

При открытии нескольких файлов они отображаются во вкладках в верхней части окна программы. Пользователь может быстро переключаться между ними, не закрывая текущую работу. По умолчанию в Visual Studio Code файл может открываться в режиме предварительного просмотра. В этом режиме при открытии следующего файла предыдущий автоматически заменяется, и отдельная вкладка не создаётся.

Чтобы открыть несколько файлов одновременно, необходимо:

1. В левой панели Explorer выбрать нужный файл.
2. Чтобы файл открылся как отдельная вкладка, необходимо: дважды щёлкнуть по файлу левой кнопкой мыши или открыть файл и затем нажать на него повторно.
3. После этого вкладка файла закрепится в верхней части окна.
4. Аналогичным образом можно открыть и остальные файлы (Рисунок 37).

	A	B	C
1	Домохозяйство	Обследовано домохозя...	Доля домохозя находящихся в крайне...
2	Без детей	200	6
3	С детьми в возрасте до 16 лет	500	18
4	пенсионеров	100	25
5	Итого	800	
6	R=0,954	t=2	
7	Средняя доля по выборке	0.15875	
8	Мужгрупповая дисперсия	0.003760937	
9	Ошибка выборки средняя	0.002168219	
10	Предельная ошибка	0.004336437	
11	Доверительный интервал	15,44% <= p <= 16,31%	
12			
13			
14	w-Dw	w+Dw	
15	0.154413563	0.163086437	

Рисунок 37 – Работа с несколькими файлами

Поиск по файлам

Для быстрого поиска информации можно использовать встроенную функцию поиска. Для этого необходимо:

1. Нажать сочетание клавиш Ctrl + F.
2. Ввести нужное слово или фразу.
3. Просмотреть найденные совпадения в документе (Рисунок 38).

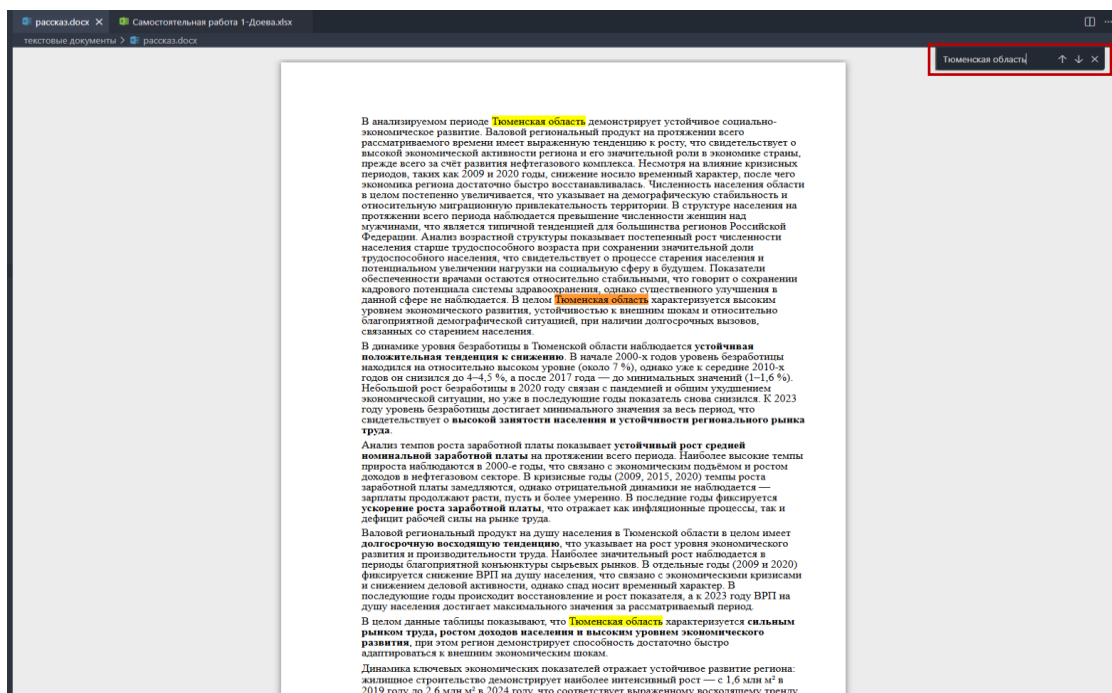


Рисунок 38 – Поиск по ключевым словам

Использование нейросети в Visual Studio Code

Работа с расширением Qwen

Современные расширения Visual Studio Code позволяют использовать нейросетевые инструменты для решения учебных задач.

После установки соответствующего расширения (см. п. установка расширений в Visual Studio Code) пользователь может открыть встроенное окно чата или панели помощи.

Для начала работы необходимо:

1. Открыть установленное AI-расширение в нижней части боковой панели Visual Studio Code (Рисунок 39).

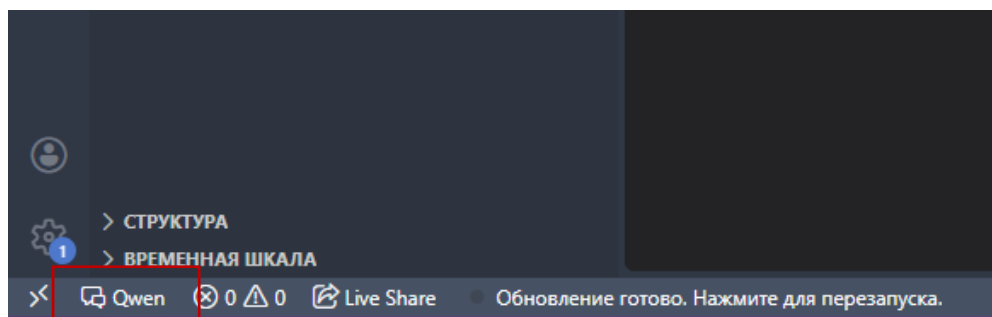


Рисунок 39 – Расширение нейросети

2. Ввести запрос в текстовое поле.
3. Дождаться ответа нейросети (Рисунок 40).

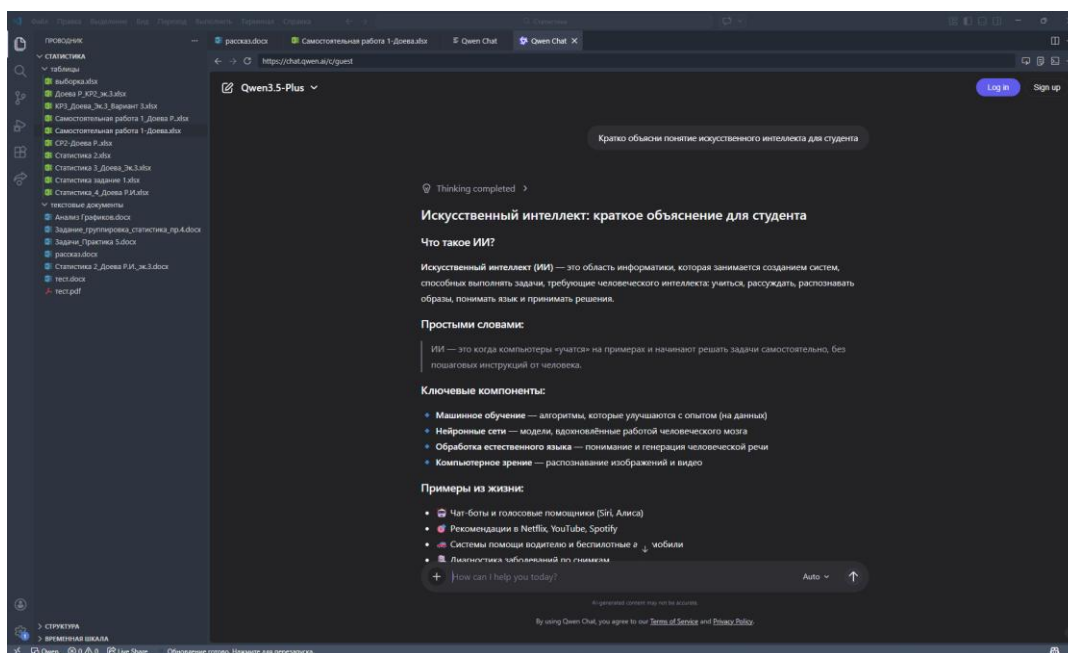


Рисунок 40 – Вкладка нейросети

Работа с GitHub Copilot

В отличие от рекомендуемой нейросети Qwen, которая в большей степени используется как инструмент для получения текстовых ответов, формулирования идей, составления материалов и генерации содержимого по запросу, GitHub Copilot более тесно интегрирован в среду Visual Studio Code и лучше подходит для практической работы непосредственно с файлами проекта.

Важно: если задача студента связана с тем, чтобы прочитать материалы из рабочей папки, создать новый файл, собрать итоговый документ, обновить существующий файл или структурировать содержимое проекта, предпочтительнее использовать GitHub Copilot, поскольку он лучше адаптирован для работы непосредственно в файловой среде Visual Studio Code.

У GitHub Copilot есть ограничения по запросам: 2000 автодополнений кода в месяц и 2000 автодополнений кода в месяц (каждый вопрос в чате = 1 premium запрос).

После установки соответствующего расширения (см. п. установка расширений в Visual Studio Code) пользователь может открыть встроенное окно чата или панели помощи. Для начала работы необходимо:

1. Открыть установленное AI-расширение в верхней части правой боковой панели Visual Studio Code (Рисунок 41). Или можно открыть через: Ctrl + Shift + P. Далее нужно ввести: GitHub Copilot: Open Chat или Copilot Chat. После этого откроется окно встроенного ИИ-помощника.

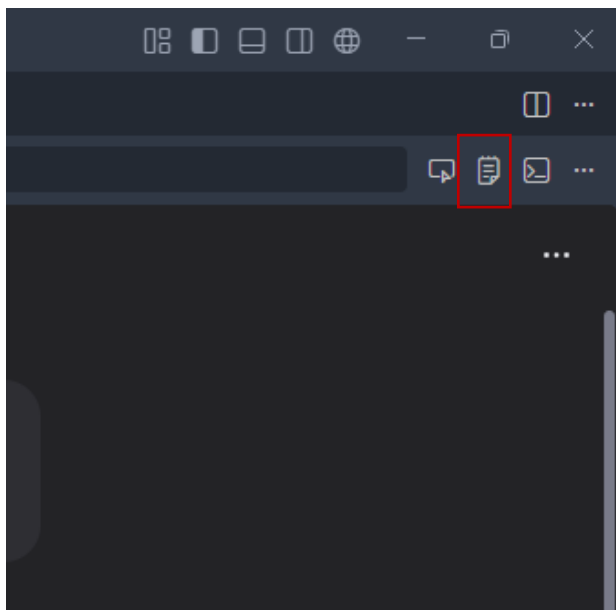


Рисунок 41 – Открытие GitHub Copilot

2. Ввести запрос в текстовое поле.
3. Дождаться ответа нейросети (Рисунок 42).

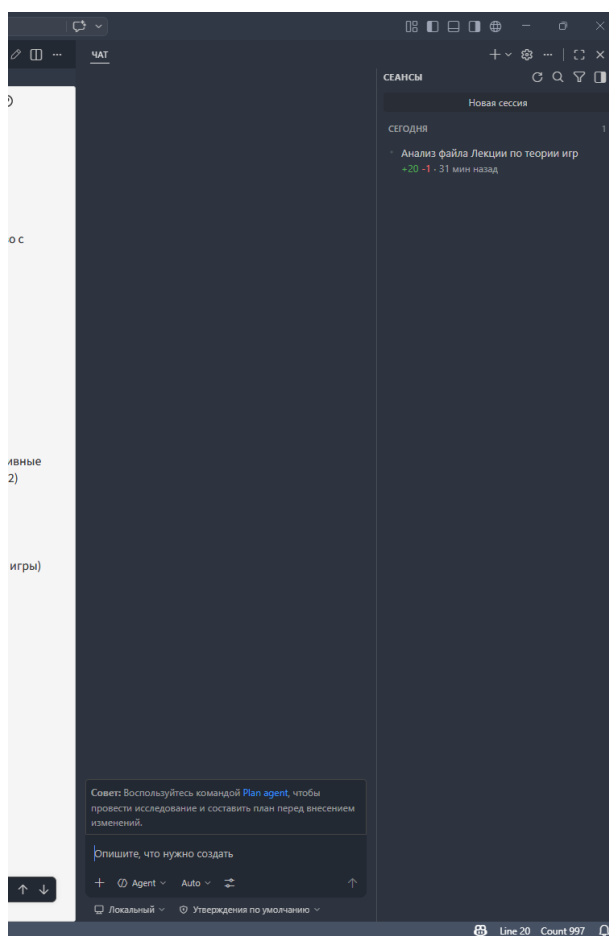


Рисунок 42 - Интерфейс GitHub Copilot

Настройка AI Tunnel и интеграция с VS Code

В стандартной конфигурации Visual Studio Code возможности работы с нейросетями ограничены встроенными или официально поддерживаемыми решениями (например, GitHub Copilot). Однако в ряде случаев пользователю требуется более гибкий доступ к различным моделям искусственного интеллекта, таким как ChatGPT, Claude, Qwen, Kimi и другим.

Сервис AI Tunnel (<https://aitunnel.ru>) предоставляет такую возможность, выступая в роли единого API-шлюза для подключения к различным языковым моделям без необходимости использования VPN. Это особенно актуально в условиях ограниченного доступа к зарубежным сервисам.

Важно учитывать следующие особенности:

- сам сервис AI Tunnel является бесплатным;
- однако использование моделей платное – списание средств происходит за обработку запросов;
- стоимость зависит от выбранной модели и объёма передаваемых данных (входных и выходных токенов).

Таким образом, AI Tunnel целесообразно использовать в случаях:

- при необходимости профессиональной работы с ИИ;
- при интеграции ИИ в собственные проекты;
- при выполнении сложных задач (анализ, генерация, обработка данных);
- при разработке программных продуктов с использованием ИИ.

Интеграция с Visual Studio Code осуществляется через расширение **Cline**, которое позволяет напрямую взаимодействовать с моделями внутри редактора.

Регистрация в сервисе AI Tunnel

Регистрация в AI Tunnel является обязательным этапом, поскольку доступ к нейросетевым моделям осуществляется через API, требующий авторизации. В рамках данного сервиса создаётся уникальная учётная запись пользователя, к которой привязываются: API-ключи, баланс средств и история запросов.

1. Откройте любой современный браузер (например, Google Chrome, Microsoft Edge или Mozilla Firefox). В адресной строке введите: <https://aitunnel.ru>. Либо можно прямо перейти по ссылке с электронной версии методички.

2. На главной странице найдите кнопку «Войти в панель» или «Sign up». В открывшейся форме введите адрес электронной почты и пароль. Также можно воспользоваться готовыми сервисами для регистрации (Рисунок 43-44).

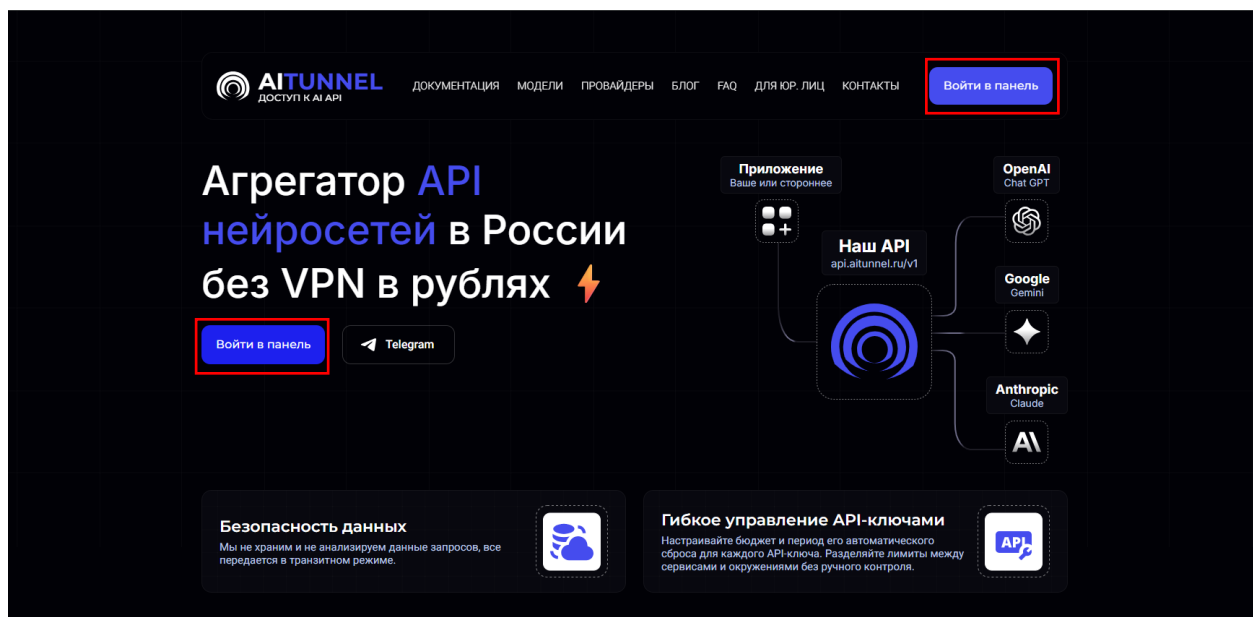


Рисунок 43 – Интерфейс AI Tunnel

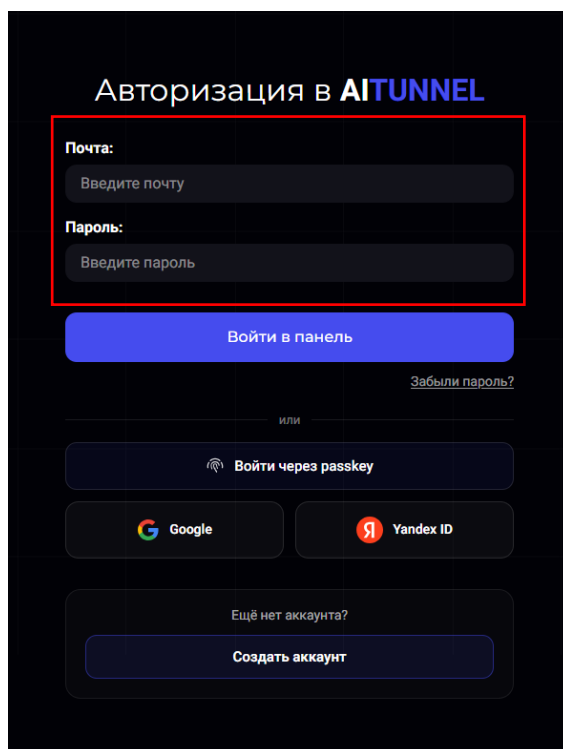


Рисунок 44 – Авторизация в AI Tunnel

3. Перейдите в почтовый ящик, указанный при регистрации. Найдите письмо от AI Tunnel. Откройте письмо и перейдите по ссылке подтверждения.

Важно: без подтверждения почты доступ к функционалу сервиса может быть ограничен.

4. Вернитесь на сайт AI Tunnel. Нажмите кнопку «Войти» и введите логин и пароль.

Просмотр стоимости моделей и тарифов

После регистрации и входа в сервис AITunnel рекомендуется ознакомиться со стоимостью использования различных нейросетевых моделей. Это особенно важно, поскольку разные модели существенно отличаются по цене, скорости работы и качеству генерации. Для просмотра тарифов и параметров моделей выполните следующие действия:

1. Откройте главную страницу сайта и пролистайте в самый низ. Перейдите в раздел «Модели» (Рисунок 45).

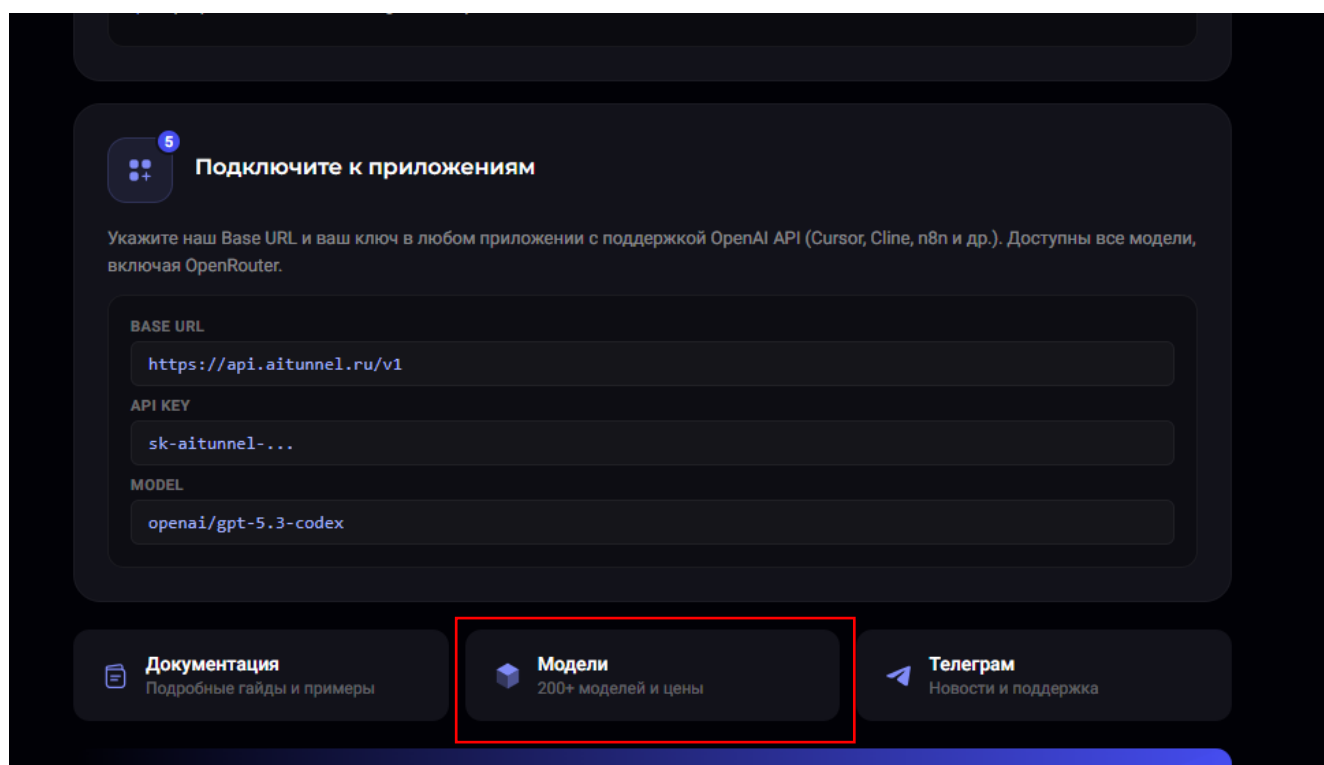


Рисунок 45 – Главная страница AI Tunnel

2. В открывшемся списке выберите интересующую модель (Рисунок 46). После открытия страницы модели обычно отображается следующая информация:

- Model ID – техническое название модели, которое используется при подключении в VS Code и Cline;
- стоимость входящих токенов (Input) – цена за обработку текста, который отправляет пользователь;
- стоимость исходящих токенов (Output) – цена за генерацию ответа нейросетью;
- контекстное окно – максимальный объём текста, который модель может обработать за один запрос;
- поддержка функций – работа с кодом, файлами, изображениями, reasoning-режимами и др.;
- провайдер модели – OpenAI, Anthropic, Qwen, DeepSeek, Google и т.д.

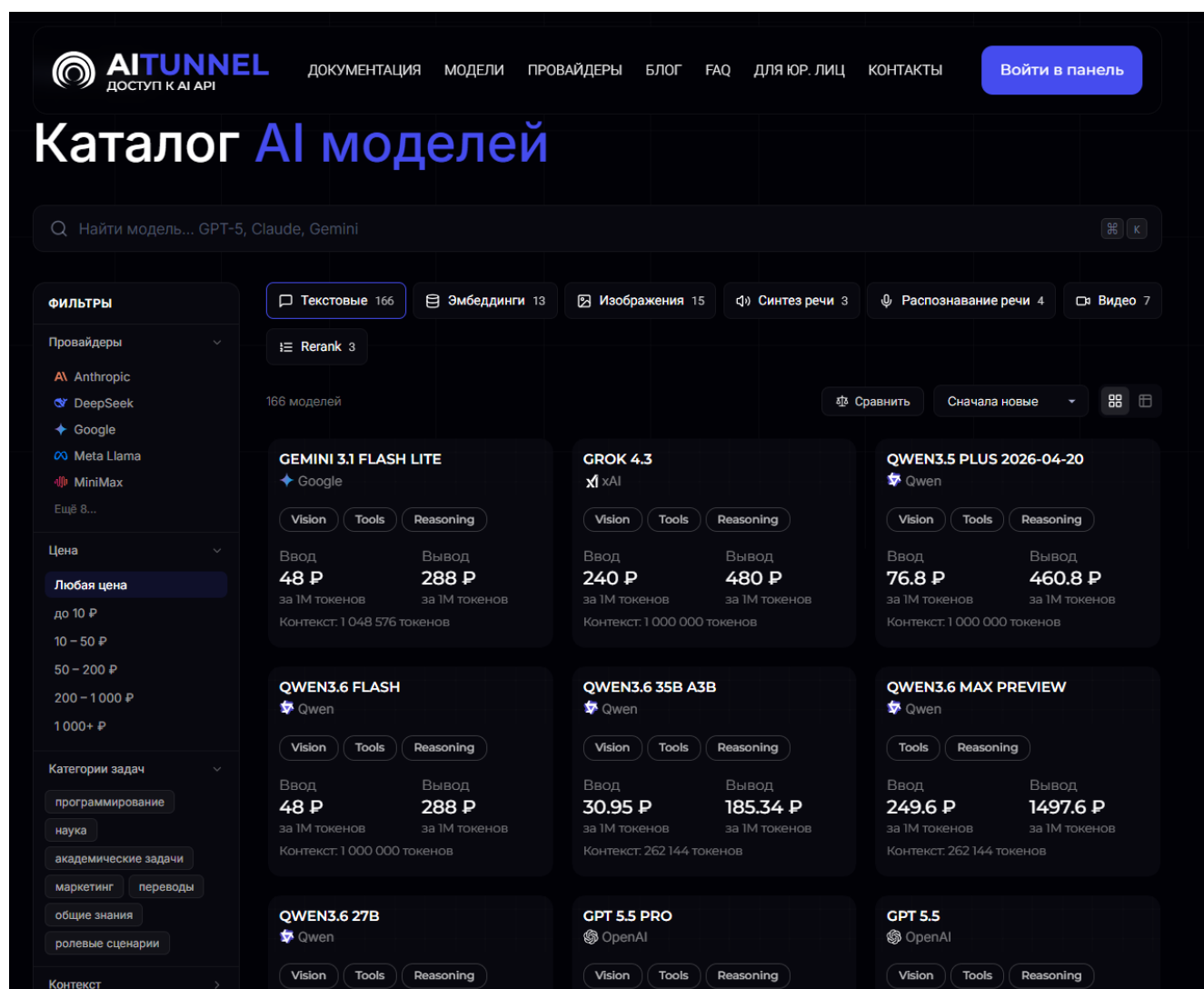


Рисунок 46 – Каталог моделей AI Tunnel

Как правило, цены указываются за 1 миллион токенов либо за 1000 токенов. Следует учитывать, что дешёвые модели подходят для повседневной учебной работы, а более дорогие модели используются для сложного анализа, генерации кода и больших проектов.

Например:

- модели семейства Qwen и DeepSeek обычно стоят значительно дешевле;
- модели уровня OpenAI GPT или Anthropic Claude обеспечивают более высокое качество, но требуют больших затрат.

При большом количестве доступных моделей в AITunnel особенно полезной становится система фильтрации каталога. Она позволяет быстро находить модели под конкретные задачи и сравнивать их характеристики.

Во вкладке каталога моделей обычно доступны следующие инструменты фильтрации (Рисунок 47):

- по провайдеру – позволяет отображать модели только определённой компании (например, OpenAI, Anthropic, Qwen, DeepSeek, Google и др.);
- по стоимости – помогает выбрать наиболее дешёвые или, наоборот, наиболее производительные модели;
- по категории задач – можно отфильтровать модели для генерации текста, программирования, анализа документов и т.д.
- по размеру контекстного окна – полезно при работе с большими документами и длинными диалогами;

Кроме фильтрации, каталог обычно поддерживает:

- строку поиска по названию модели;
- сортировку по цене;
- сортировку по популярности;
- сортировку по дате обновления.

Это особенно удобно при сравнении моделей перед подключением к Visual Studio Code через Cline.

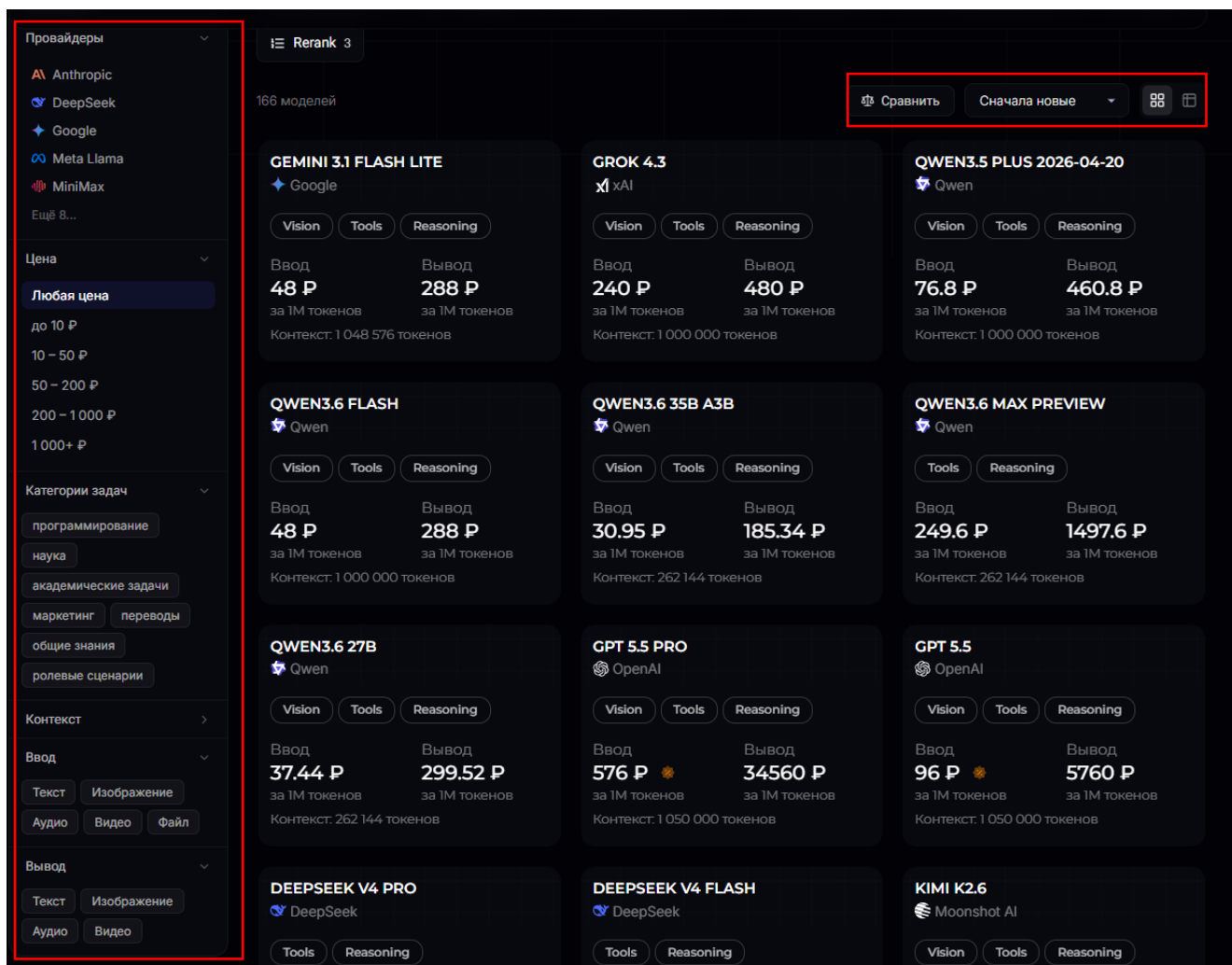


Рисунок 47 – Фильтрация в каталоге моделей AI Tunnel

Перед началом работы рекомендуется: сравнить цены нескольких моделей; проверить размер контекстного окна; оценить предполагаемый объём запросов. Это позволит подобрать оптимальный баланс между стоимостью и качеством генерации.

Пополнение баланса

Несмотря на то, что сам сервис AI Tunnel является бесплатным, выполнение запросов к моделям осуществляется на платной основе. Оплата производится за фактическое использование вычислительных ресурсов.

1. В боковой панели, на главной или в верхнем правом углу найдите раздел «Пополнение баланса» (Рисунок 48).

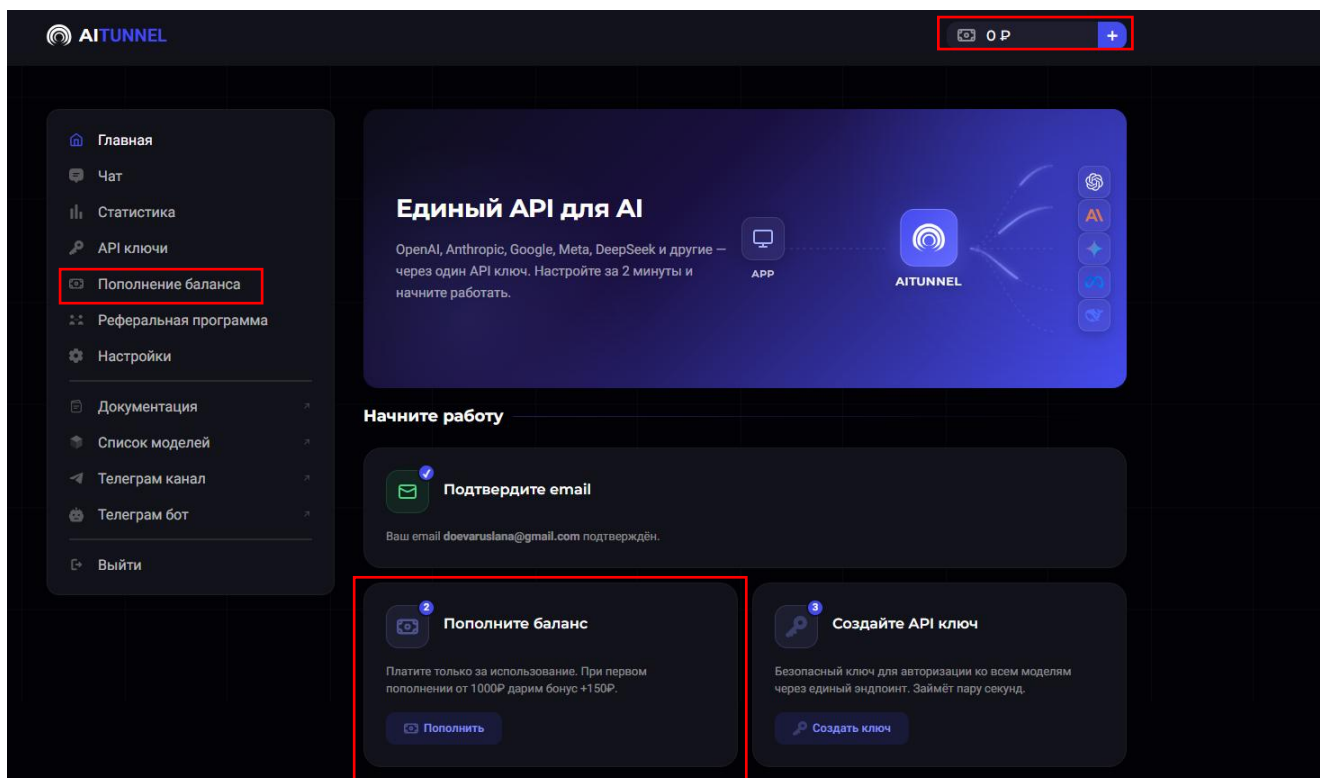


Рисунок 48 – Главная страница AI Tunnel

2. Укажите сумму (например, 400–500 рублей). Минимальная сумма на пополнение – 399 рублей. Далее выберите способ оплаты. Размер суммы зависит от предполагаемого объёма работы. Для учебных задач обычно достаточно небольшого баланса.

Во вкладке «Баланс» пользователь может управлять средствами, необходимыми для работы с нейросетевыми моделями. Здесь отображается текущий остаток на счёте, история пополнений, а также информация о расходах при выполнении запросов. Через данную вкладку можно быстро пополнить баланс с помощью банковской карты или других доступных способов оплаты (Рисунок 49).

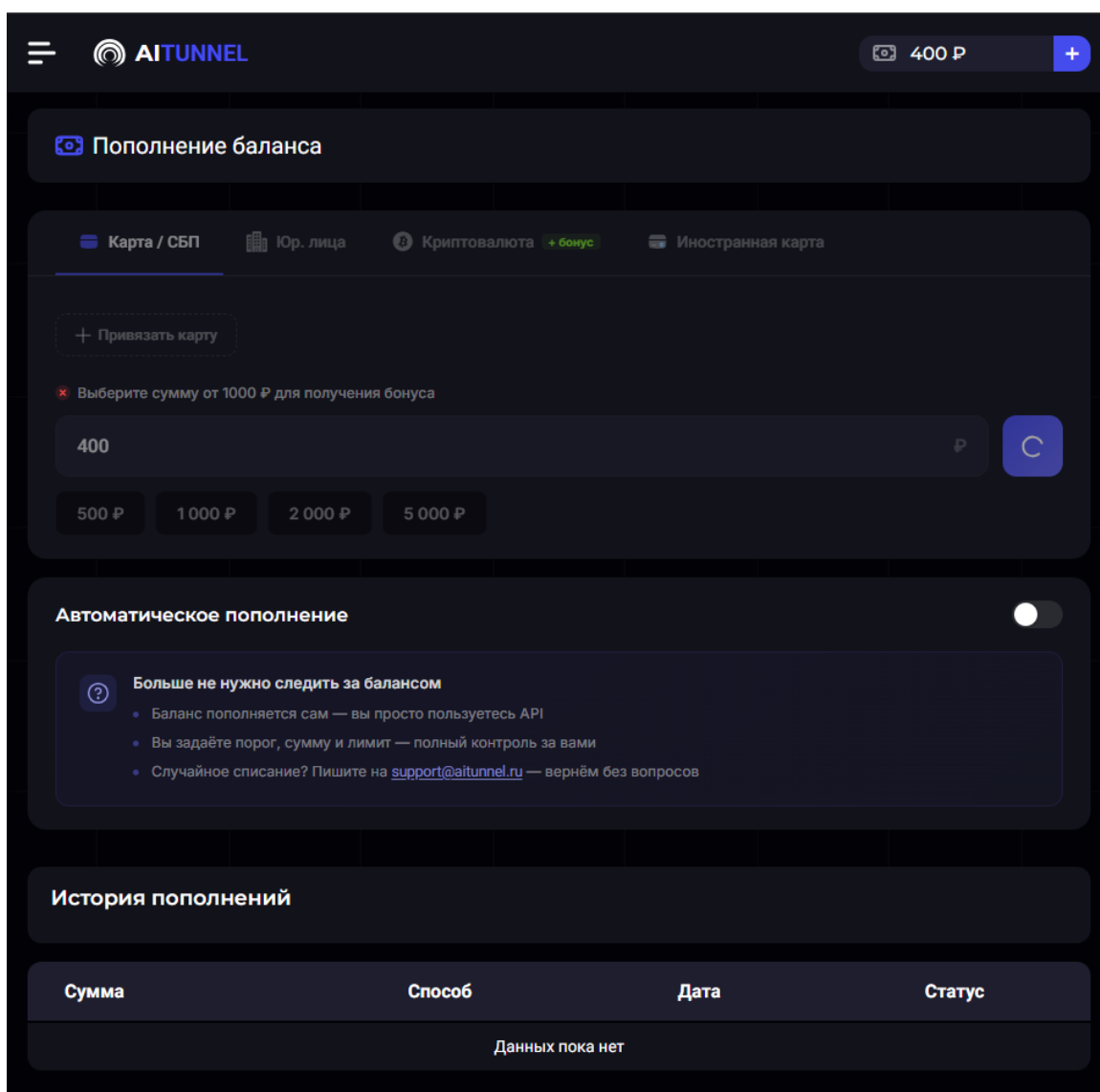


Рисунок 49 – Вкладка пополнения баланса в AI Tunnel

3. Подтвердите платёж. Дождитесь зачисления средств. Ваш баланс отобразится в правом верхнем углу.

Получение и настройка API-ключа

API-ключ является основным механизмом аутентификации при работе с нейросетевыми моделями. Он выполняет роль “цифрового идентификатора”, который позволяет системе определить, от какого пользователя поступает запрос. Обратите внимание, что без пополнения баланса, ключ создать нельзя!

1. В боковой панели или на главной странице найдите раздел «API» («API Keys» или «Настройки API») (Рисунок 50).

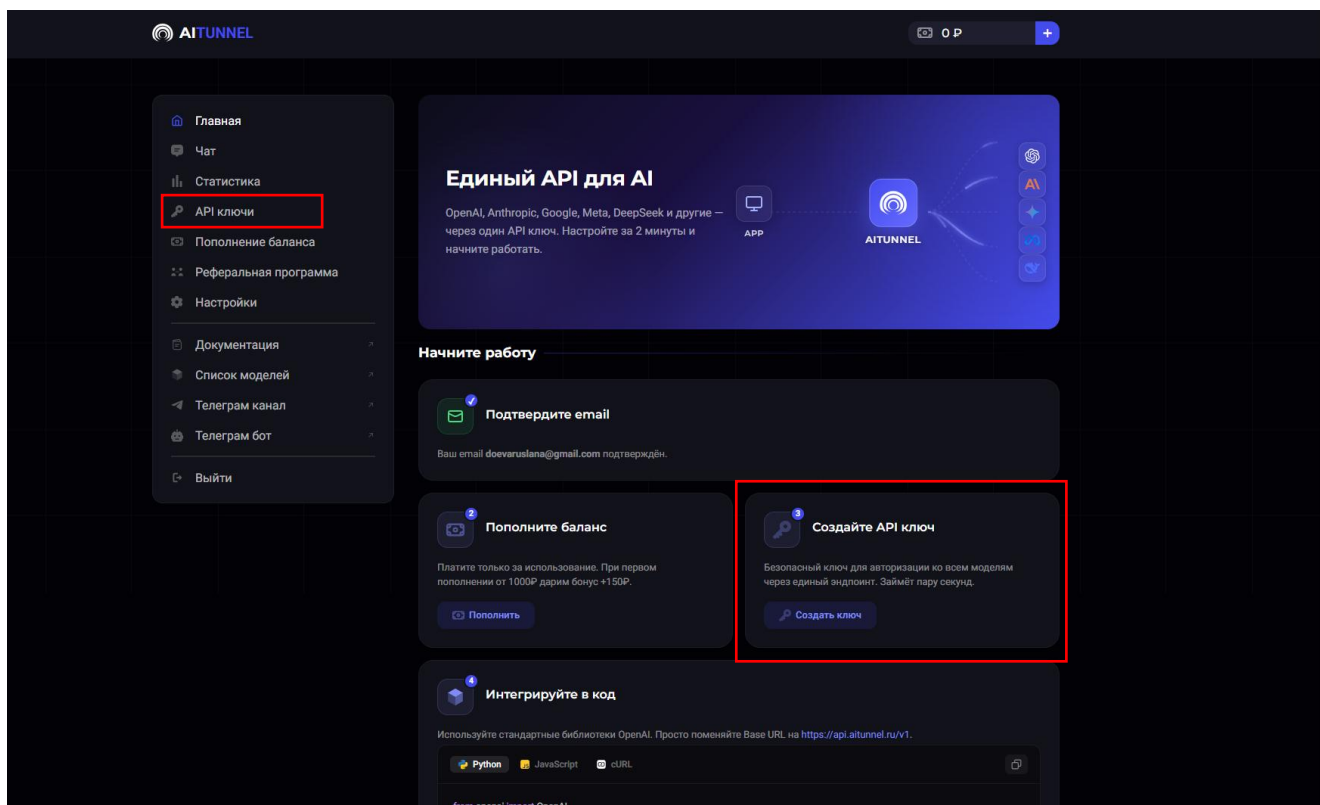


Рисунок 50 – Главная страница AI Tunnel

2. Нажмите кнопку «Создать ключ» или «Generate API Key». Система создаёт уникальную строку, которая привязана к вашему аккаунту и используется при каждом запросе к модели. Сервис сгенерирует строку вида: sk-xxxxxxxxxxxxxxxxxxxx (Рисунок 51).

Что вводить при создании API-ключа?

- название ключа (Name / Key Name) – это произвольное имя, которое нужно только для вашего удобства. Например, VS Code, Cline, Учебный проект и т.д. Рекомендуется указывать понятное название, чтобы потом не путаться между ключами;
- предлагается также ограничить максимальные траты по этому ключу (Бюджет) и выбрать срок действия данного ключа.

Новый ключ API

Название
Придумайте понятное название, чтобы различать ключи между собой.

Например: Рабочий проект

Бюджет (необязательно)
Ограничьте максимальные траты по этому ключу. Оставьте пустым для безлимитного использования.

Сумма в рублях

Срок действия (необязательно)
Ключ будет автоматически удалён по истечении выбранного срока.

Без срока

Рисунок 51 – Создание ключа в AI Tunnel

3. Скопируйте полученный ключ. Сохраните его в текстовом файле или в защищённом месте.

Важно: API-ключ необходимо хранить в безопасности, так как он даёт доступ к вашему балансу и через него могут выполняться платные запросы.

Установка расширения Cline и настройка подключения (Cline + AI Tunnel)

Расширение Cline обеспечивает интеграцию нейросетевых моделей непосредственно в среду разработки, позволяя выполнять запросы без перехода в браузер.

1. Запустите Visual Studio Code. В левой панели нажмите на иконку Extensions. Введите в строке поиска «Cline»

2. Выберите нужное расширение. Нажмите кнопку Install. Дождитесь завершения установки (Рисунок 52).

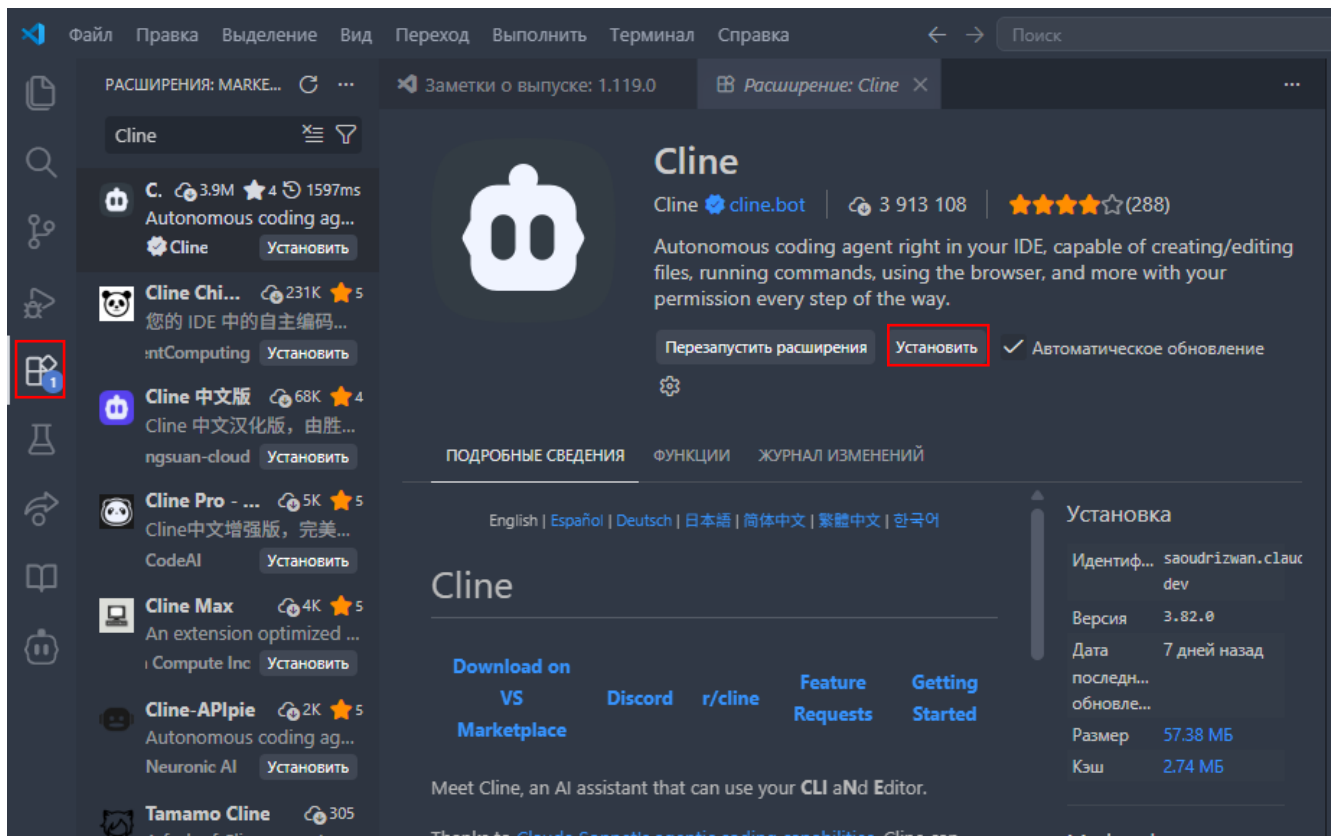


Рисунок 52 – Установка расширения Cline

3. Следующим этапом нужно открыть настройки Cline. Нажмите **Ctrl + Shift + P** и введите **Cline Settings**. Либо откройте расширение в левой боковой панели. Далее в правой верхней части нажмите на значок шестеренки (Рисунок 53).

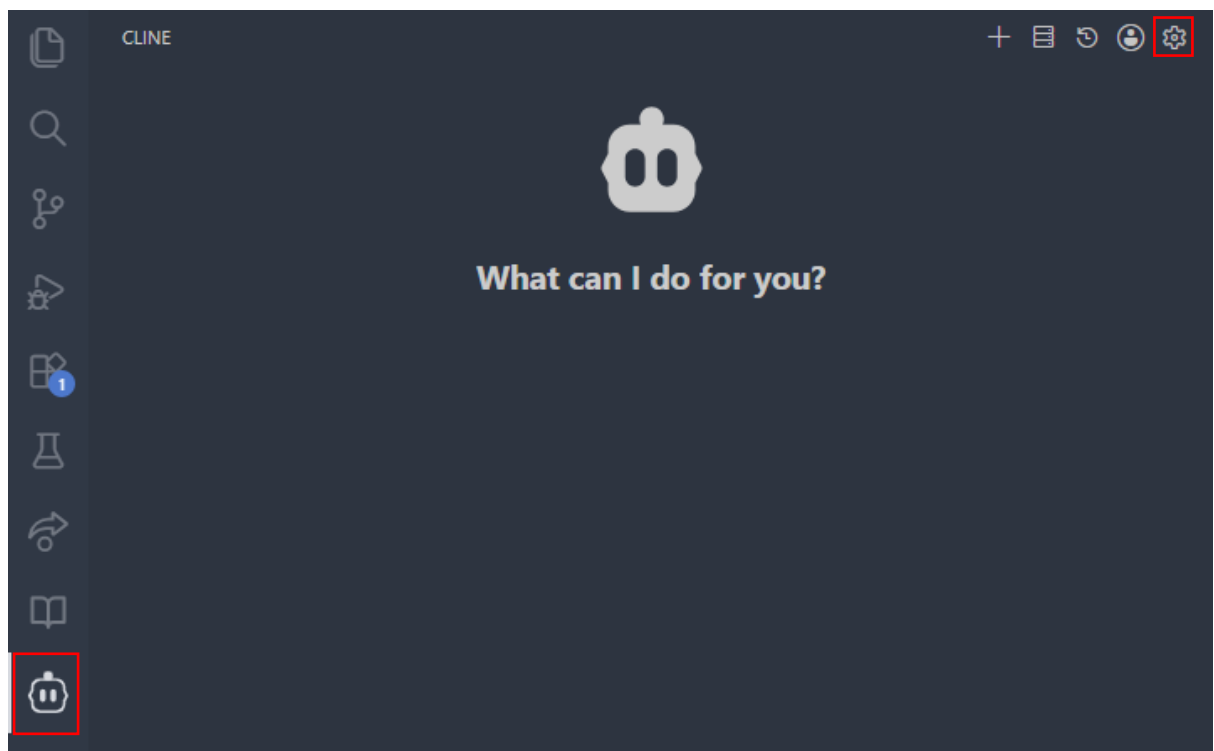


Рисунок 53 – Интерфейс Cline

В настройках найдите вкладку API Configuration. При настройке расширения Cline в Visual Studio Code необходимо заполнить параметры подключения к нейросетевой модели. Основные поля представлены в окне API Configuration:

1) API Provider – в данном поле выбирается поставщик API. Даже если вы используете модели DeepSeek, Qwen, Kimi, Claude или другие, в поле API Provider всё равно обычно выбирается OpenAI или OpenAI Compatible. AI Tunnel работает как универсальный «посредник» и предоставляет единый API-интерфейс, совместимый со стандартом OpenAI. То есть, запросы отправляются в формате OpenAI API, а уже внутри AI Tunnel перенаправляет их в нужную модель.

2) OpenAI API Key – в это поле необходимо вставить API-ключ, созданный в личном кабинете AI Tunnel.

3) Model – в данном поле указывается идентификатор модели (Model ID), который берётся со страницы выбранной модели в каталоге AI Tunnel (Рисунок 54).

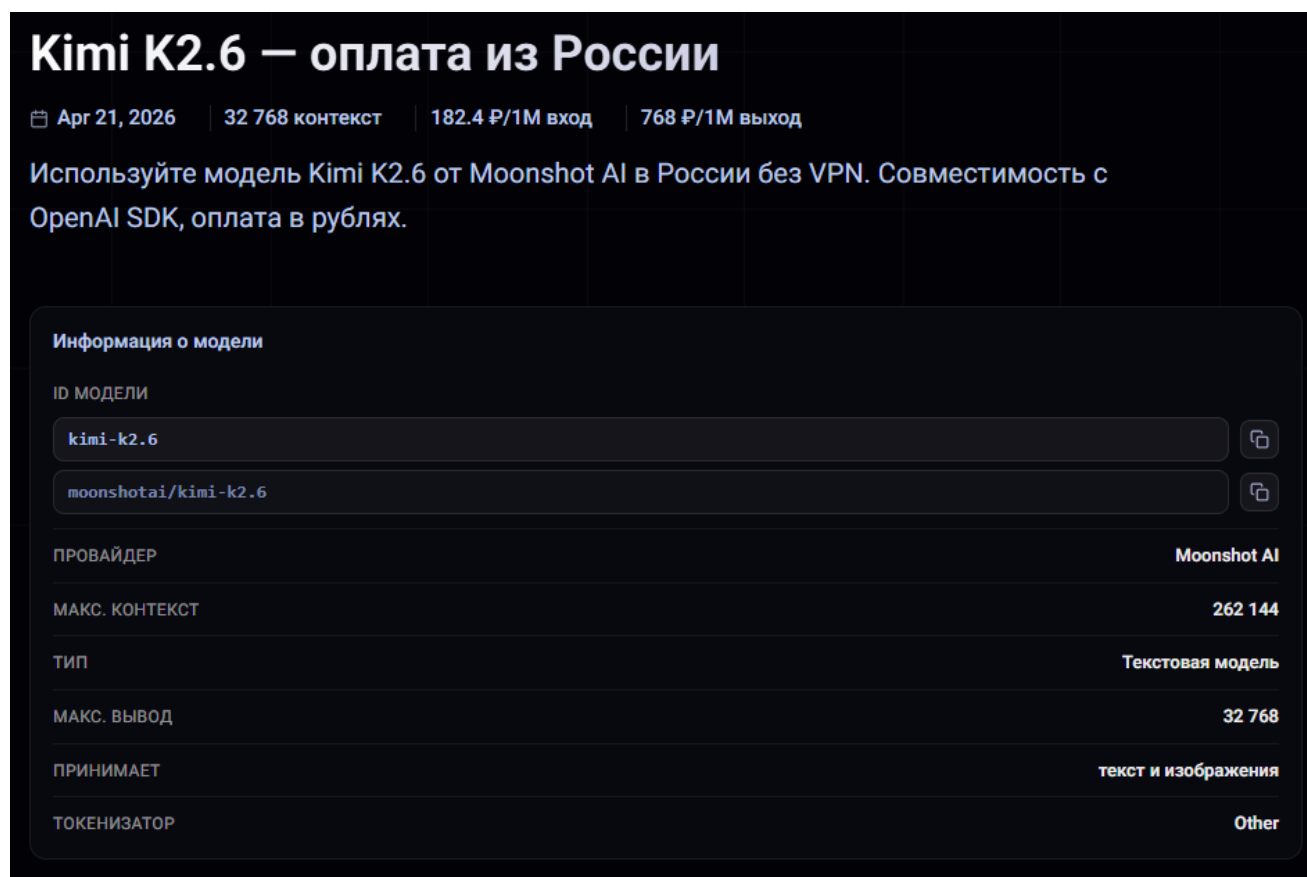


Рисунок 54 – Информация о выбранной модели

4) Reasoning Effort – данный параметр отвечает за «глубину размышления» модели. Обычно доступны режимы:

- Medium – для обычной учебной работы;
- High – для сложного анализа и программирования;
- Low – для быстрых и дешёвых ответов.

Следует учитывать, что высокий уровень размышления увеличивает расход токенов и стоимость запросов.

5) Advanced Settings (Дополнительно) – в некоторых нейросетевых моделях в расширенных настройках необходимо указать Base URL (<https://api.aitunnel.ru/v1>). Данный адрес не является обычным веб-сайтом. Это API-адрес для программного подключения, который используется расширением Cline и другими приложениями.

4. Сохраните изменения нажав на кнопку «Done» в правом верхнем углу (Рисунок 55). Теперь каждый запрос из VS Code будет отправляться через AI Tunnel к выбранной модели.

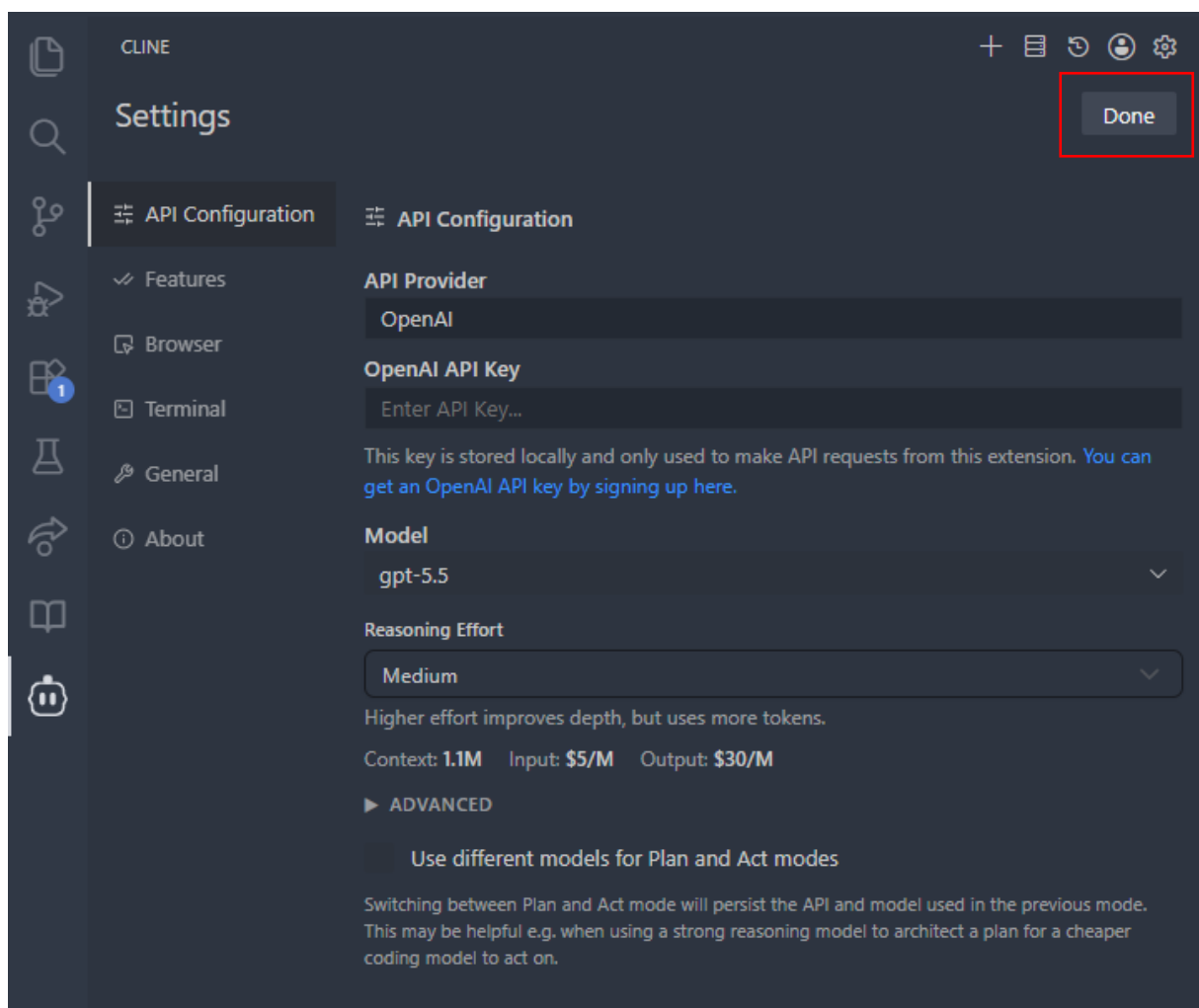


Рисунок 55 – Установка модели

5. Введите простой запрос, например, «напиши короткое описание технологии искусственного интеллекта». Если настройка выполнена корректно, то появляется ответ и происходит списание средств.

Основные элементы интерфейса Cline

1. В верхней части окна расположены элементы управления сессией (Рисунок 56). Обычно здесь находятся:

- создание нового чата;
- история диалогов;
- настройки подключения моделей;
- управление аккаунтом;
- MCP Servers¹.

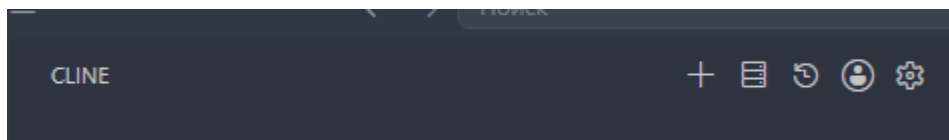


Рисунок 56 – Верхняя панель Cline

Через данную панель выполняется настройка подключения к API и переключение между различными моделями.

2. Центральная часть окна используется для взаимодействия с ИИ-ассистентом (Рисунок 57). Здесь отображаются сообщения пользователя и ответы модели. Именно в этой области происходит основная работа с проектом. В верхней части ответа иногда отображается стоимость обработки запроса. Например, \$0.0000 или другое значение. Это позволяет контролировать расходы и отслеживать стоимость генерации.

¹ MCP (Model Context Protocol) Servers – это специальные серверы или п модули, которые позволяют ИИ-ассистенту получать дополнительный доступ к внешним инструментам, данным и функциям.

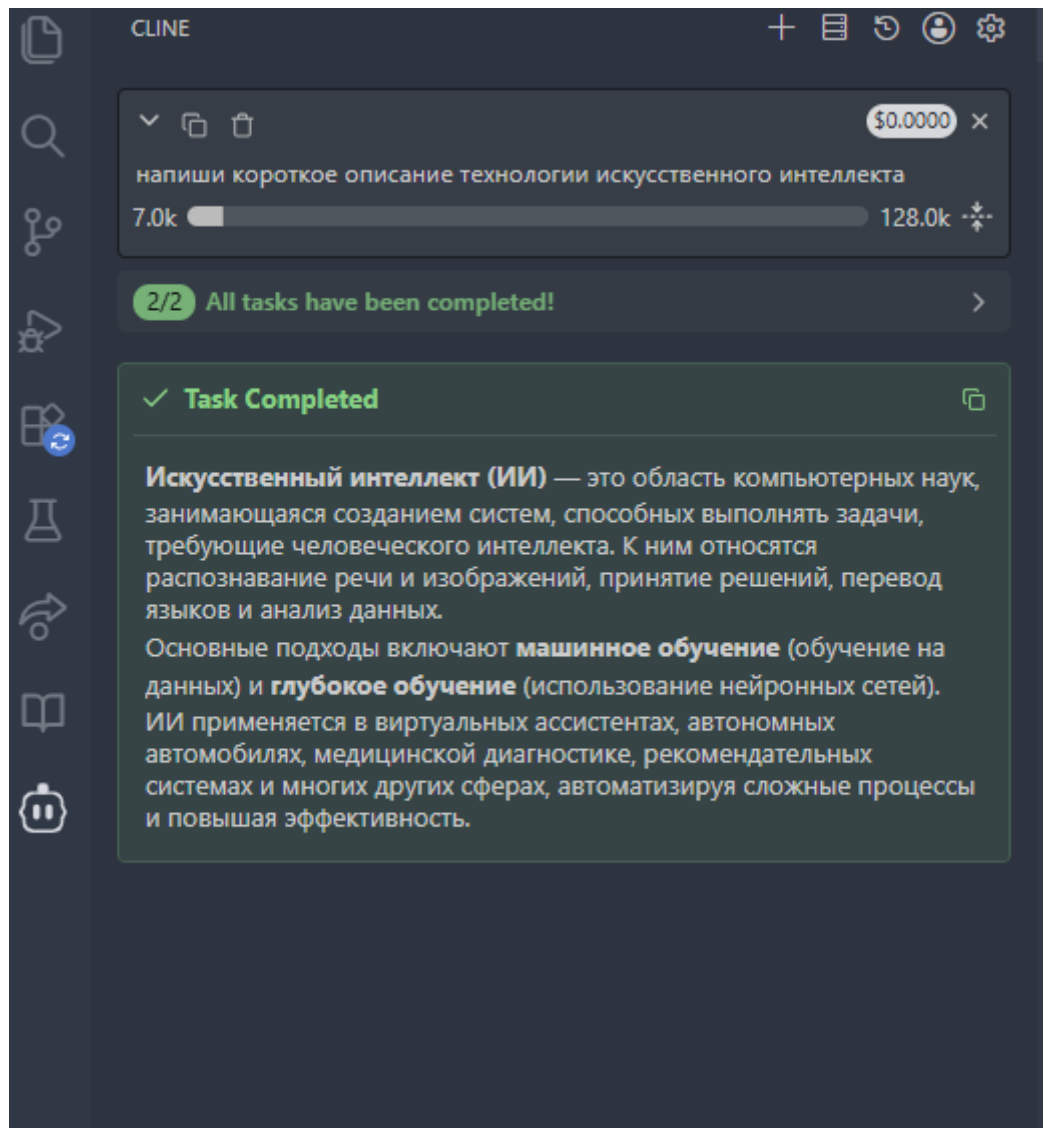


Рисунок 57 – Центральная панель Cline

3. В нижней части интерфейса расположено текстовое поле (Рисунок 58). В данное поле пользователь вводит запросы для ИИ-ассистента.

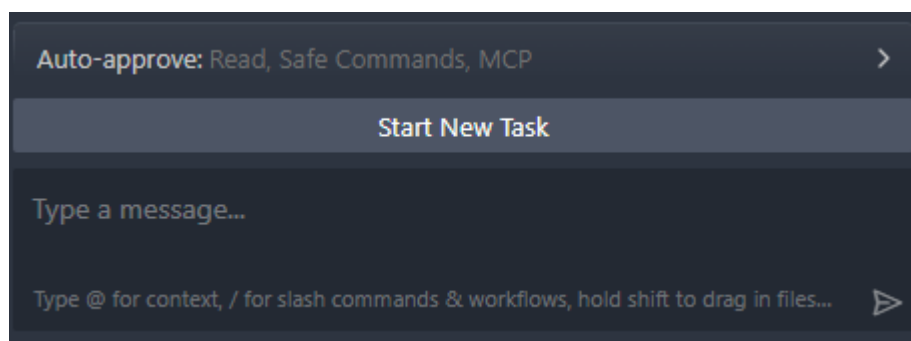


Рисунок 58 – Нижняя Cline

4. В нижней панели отображается текущая используемая модель. Например: moonshot: kimi-k2.5. Это означает, что Cline отправляет запросы через выбранную нейросетевую модель. Пользователь может переключать модели нажав на текущую версию (Рисунок 59).

5. В правом нижнем углу находятся два режима работы «Plan» и «Act». Plan – режим анализа и планирования. В этом режиме Cline: объясняет действия, предлагает структуру решения, не изменяет файлы автоматически. Act – режим активного выполнения действий. В данном режиме Cline может: создавать файлы, редактировать код, изменять структуру проекта, автоматически вносить исправления и т.д. Перед выполнением действий Cline обычно запрашивает подтверждение пользователя (Рисунок 59).

6. Одним из главных преимуществ Cline является доступ к файловой системе проекта. Знак «@» используется для добавления контекста (Рисунок 59). С его помощью можно:

- подключать файлы;
- указывать папки проекта;
- выбирать открытые документы;
- передавать ИИ содержимое определённых частей проекта.

Примеры: @report.docx исправь оформление по ГОСТ, @main.py найди ошибки в коде, @notes.md сделай краткий конспект и т.д.

7. Символ «/» открывает систему встроенных команд Cline (Рисунок 59). Через slash-команды можно: запускать автоматические сценарии, выполнять действия с проектом, активировать инструменты. Примеры: /help – показывает список команд, /newtask – создаёт новую задачу.

8. Нижняя панель поддерживает drag-and-drop (Рисунок 59). Пользователь может, нажав на значок «+»:

- перетаскивать файлы прямо в чат;
- добавлять изображения;
- прикреплять PDF;
- загружать документы;

- передавать таблицы и исходный код.

После загрузки ИИ получает возможность анализировать содержимое файлов.

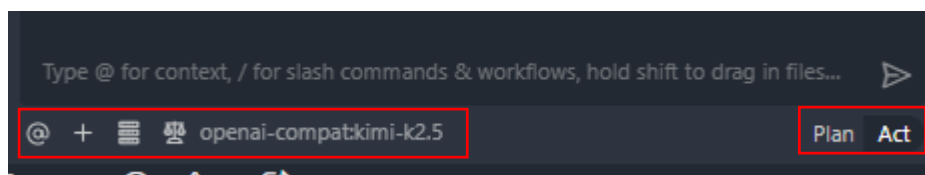


Рисунок 59 – Нижняя панель Cline

Настройка агентов: системные файлы AGENTS.md, QWEN.md, Project.md и другие

Агент – это ИИ-помощник, встроенный в вашу среду разработки (VS Code). Он может читать и создавать файлы, отвечать на вопросы, писать код, анализировать документы. Примеры агентов: Claude Code (от Anthropic), Qwen Code, GitHub Copilot, Cursor. В отличие от простого чата в браузере, агент имеет доступ к файловой системе вашего проекта и может выполнять действия (создать файл, изменить код, запустить команду).

По умолчанию агент не знает ваших предпочтений: на каком языке отвечать, какой стиль кода использовать, как организован проект, какие библиотеки запрещены. Каждый раз объяснять это вручную – утомительно.

Файлы конфигурации позволяют задать правила один раз, и агент будет им следовать в каждой сессии, экономя ваше время и обеспечивая единообразие. Обычно используют следующие расширения:

- **.txt** – обычный текстовый файл без форматирования. Только буквы, цифры и знаки. Подходит для заметок, черновиков, простых записей;
- **.md** – файл в формате Markdown. Позволяет делать заголовки, списки, жирный/курсивный текст, таблицы, ссылки и вставлять код. При этом остаётся читаемым и в обычном блокноте. Удобен для конспектов, отчётов, инструкций и документации.

В экосистеме VS Code сложилось несколько стандартов. Самые важные для студента в таблице 5.

Таблица 5 – Типичные файлы агентов

Файл	Для кого?	Где размещать?
AGENTS.md	Универсальный – понимают Qwen Code, Claude Code, Copilot (частично), Cursor	В корне ² проекта или в папке .qwen/
CLAUDE.md	Claude Code (агент от Anthropic)	Корень проекта или глобально ~/.claude/
copilot-instructions.md	GitHub Copilot	Папка .github/ в корне проекта
.cursorsrules	Cursor (редактор на базе VS Code)	Корень проекта
Project.md	Произвольный – часто используется как описание проекта для человека и для ИИ	Корень проекта

² Корень проекта — это главная (верхняя) папка, которую вы открыли в Visual Studio Code.

Важно: наиболее универсальным форматом сейчас является AGENTS.md. Многие современные агенты (включая Claude Code и Qwen Code) читают этот файл, если он находится в корне или в специальной папке.

Далее рассмотрим, как создать и настроить файл для агента.

1. Определитесь с агентом, которым вы пользуетесь. Для большинства учебных проектов достаточно создать AGENTS.md в корневой папке – он будет работать и с Qwen, и с Copilot (при поддержке), и с Cursor.

2. Создайте файл в нужном месте. Откройте VS Code, в вашей папке проекта (например, Курсовая_работа). Создайте новый файл:

- для AGENTS.md: просто создайте файл AGENTS.md в корне;
- для copilot-instructions.md: сначала создайте папку .github, а внутри – файл copilot-instructions.md;
- для CLAUDE.md: создайте файл CLAUDE.md в корне.

3. Напишите правила (содержимое файла). Правила пишутся на естественном языке (русском или английском) в формате Markdown. Чем чётче – тем лучше.

Базовый шаблон для AGENTS.md (учебный проект):

Инструкции для ИИ-агента

1. Стил ь общения

- Отвечай на русском языке.
- Используй академический (научно-учебный) стиль. Избегай разговорных выражений.
- Если информации недостаточно – скажи об этом прямо, не придумывай.

2. Формат ответов

- По возможности структурируй ответ: списки, заголовки, таблицы.
- Если генерируешь код (Python, HTML, JavaScript) – добавляй комментарии на русском.
- Код должен быть полностью самодостаточным (все стили и скрипты внутри одного файла, если не указано иное).

3. Работа с файлами

- При создании нового файла всегда указывай расширение (.md, .html, .py и т.д.).
- Не изменяй файлы без явного разрешения, если только задача не требует правки.
- Сохраняй созданные файлы в текущую папку, если не указан другой путь.

4. Структура проекта

- `/data` – исходные данные (CSV, Excel).
- `/docs` – методички, статьи, PDF.
- `/src` – скрипты и код.
- `/output` – итоговые отчёты, презентации, результаты.

5. Ограничения

- Не предлагай решения, требующие платных подписок (если это не оговорено).
- Не удаляй файлы из папок `/data` и `/docs`.
- При сомнениях запрашивай уточнение.

Пример для copilot-instructions.md (более короткий, так как Copilot специфичен для кода):

Copilot Instructions

- Ответы давай на русском языке.
- Для Python используй стиль PEP 8.
- Для HTML/CSS/JS создавай отдельные файлы или встроенный код по запросу.
- При генерации презентаций обязательно добавляй кнопки навигации и поддержку стрелок.

Пример для CLAUDE.md (для Claude Code):

Claude Code Project Instructions

Project context

This is a student coursework project on "Neural Networks in Education".

Rules

- Always respond in Russian.
- Use academic style.
- Never hallucinate facts. If you don't know, say so.
- Prefer creating new files over displaying code in chat.
- For HTML presentations, use light background, large headings, and arrow key navigation.

Project structure

See `README.md` for details.

4. Проверьте, что агент видит файл

- для Qwen Code: просто напишите в чате «Какие правила ты видишь в этом проекте?» – он должен прочитать AGENTS.md;
- для Copilot: инструкции из copilot-instructions.md применяются автоматически (проверить можно, задав вопрос, требующий стилового решения);

- для Claude Code: запустите `claude` в терминале в папке проекта – он автоматически загрузит `CLAUDE.md`.

5. Используйте в работе. После настройки вы можете давать агенту короткие запросы, и он будет следовать прописанным правилам. Например:

- «Создай HTML-презентацию на тему „История ДВФУ“» – агент автоматически сделает файл `.html` с кнопками, русским языком и академическим стилем.

- «Проанализируй файлы из папки `/docs` и напиши конспект в `output/conspekt.md`» – агент будет знать, куда сохранять результат.

Как писать хорошие инструкции?

Эффективность `AGENTS.md` сильно зависит от качества написанных инструкций. Вот ключевые правила:

1. Пишите кратко и по делу: инструкции должны быть короткими, четкими и релевантными для всех задач в проекте. Если файл будет перегружен, агент может просто его проигнорировать.

2. Отдавайте предпочтение "что делать", а не "как не надо": вместо списка запретов, лучше писать четкие позитивные инструкции. Агенты лучше следуют прямым указаниям, а не пытаются обойти ограничения.

3. Что должно быть в файле: в идеале файл должен дать агенту полную картину мира:

- общая информация: краткое описание проекта и его архитектуры;
- технологический стек: основные языки, фреймворки, библиотеки;
- структура проекта: карта директорий с пояснениями, что где лежит;
- код-стайл и соглашения: требования к форматированию, именованию переменных, написанию комментариев;
- критичные запреты, например, «никогда не удаляй файлы из папки `/data`»;
- команды для сборки и тестов: как запустить проект, проверить код и убедиться, что изменения ничего не сломали.

Практическое применение Visual Studio Code

Visual Studio Code может использоваться студентами не только для хранения и просмотра файлов, но и как удобный инструмент для организации учебной деятельности, подготовки материалов и работы с нейросетевыми технологиями. Visual Studio Code превосходит другие сервисы при подготовке отчётов и рефератов тем, что объединяет в одной среде работу с текстами, файлами и источниками, позволяет удобно структурировать большие проекты, быстро искать информацию по всем документам, использовать расширения (для PDF, таблиц, Markdown и ИИ), сохранять историю изменений через Git и при этом работает быстро даже на слабых устройствах без необходимости постоянного подключения к интернету.

Подготовка отчётов и рефератов

Visual Studio Code можно использовать для написания и структурирования учебных текстов, таких как отчёты, рефераты, эссе, курсовые и аналитические материалы.

1. Откройте папку проекта в Visual Studio Code.
2. Создайте новый файл для чернового текста (Рисунок 60). После названия обязательно нужно указать расширение файла.

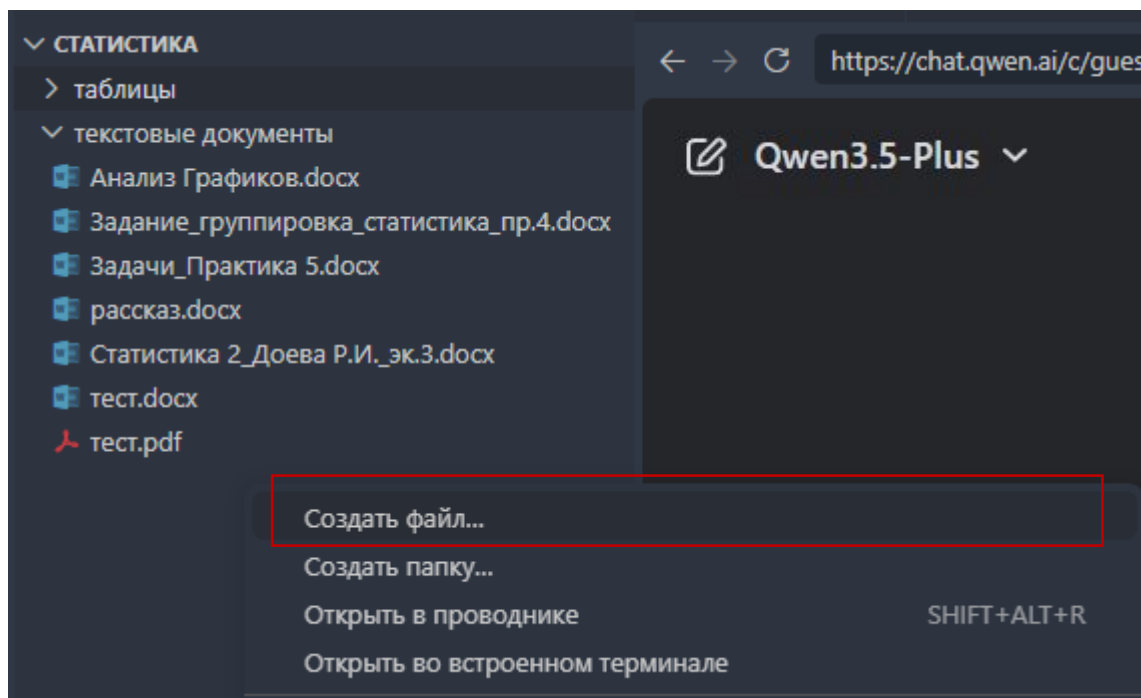


Рисунок 60 – Создание файла

3. Открытие файла для ввода текста. После создания файла он автоматически откроется в центральной части окна Visual Studio Code. Именно в этом окне и выполняется набор текста. Следует учитывать, что Visual Studio Code отображает текст в виде строк, как в редакторе. Несмотря на внешний вид, файл можно использовать как обычный черновик текста (Рисунок 61).

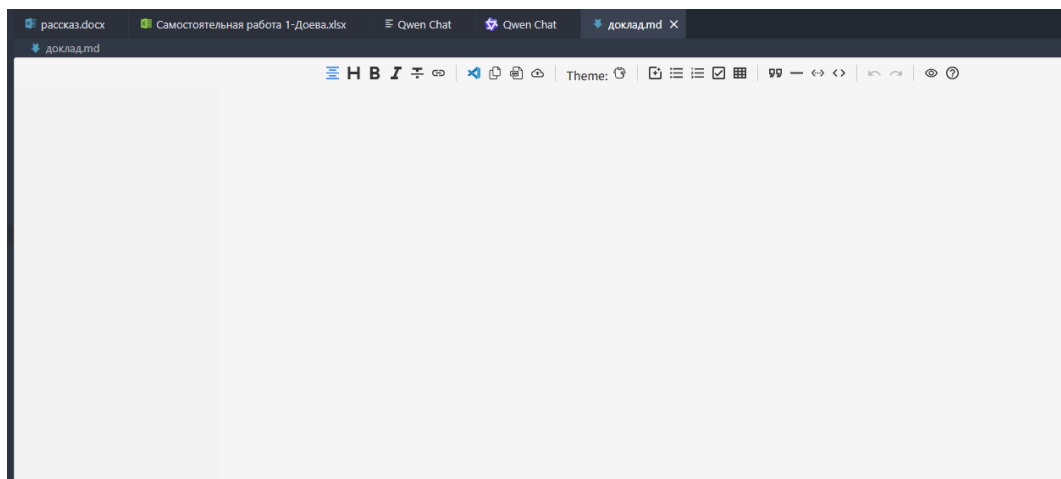


Рисунок 61 – Созданный файл .md

4. Подготовьте структуру будущей работы. Используйте нейросеть для составления плана, подготовки черновика и редактирования текста (Рисунок 62).

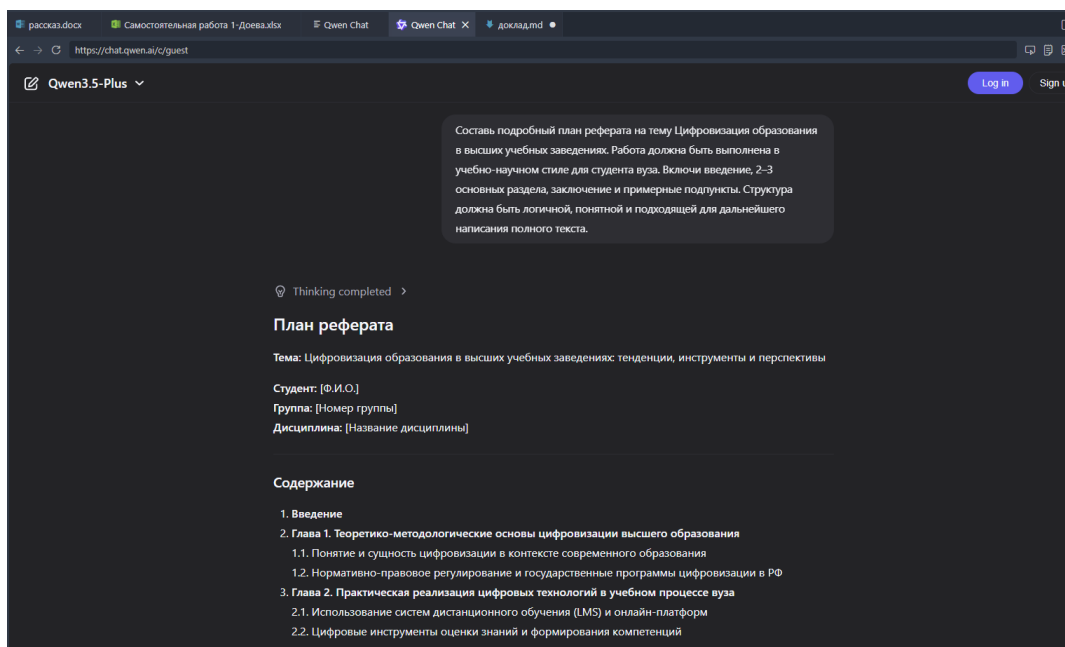


Рисунок 62 – Создание структуры через нейросеть в VS code

5. Написание основной части по разделам. Нейросеть может сгенерировать реферат одним запросом, однако такой способ не рекомендуется как основной, поскольку итоговый текст часто требует существенной проверки, редактирования,

уточнения структуры и адаптации под требования учебного задания. Наиболее эффективным является поэтапный способ подготовки работы.

6. После ввода текста файл необходимо сохранить (Рисунок 63). Для этого можно: нажать сочетание клавиш Ctrl + S или выбрать в верхнем меню File – Save (Файл – Сохранить). Рекомендуется сохранять изменения после каждого важного фрагмента текста, чтобы избежать потери информации.

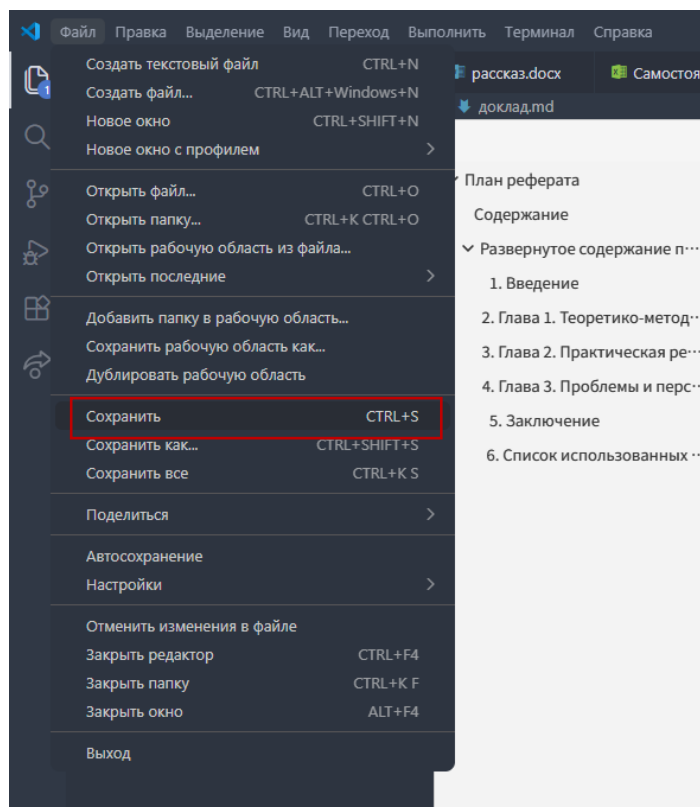


Рисунок 63 – Сохранение файла

7. Открытие файла через Word. Иногда можно не копировать текст вручную, а открыть файл напрямую в Word:

- открыть Microsoft Word;
- выбрать: Файл → Открыть;
- найти ранее сохранённый файл;
- открыть его. После этого Word может загрузить текст из файла.

Но важно:

- структура иногда отображается неидеально;
- переносы строк могут сбиваться;
- форматирование всё равно придётся настраивать вручную.

Поэтому основным рекомендуемым способом остаётся копирование текста.

Пример запроса к нейросети:

– «Составь подробный план реферата на тему [*название темы*]. Работа должна быть выполнена в учебно-научном стиле для студента вуза. Включи введение, 2–3 основных раздела, заключение и примерные подпункты. Структура должна быть логичной, понятной и подходящей для дальнейшего написания полного текста»;

– «Проверь структуру данного реферата и скажи, всё ли изложено логично и последовательно»;

– «Сделай текст более связным и плавным между разделами»;

– «Проверь, соответствует ли текст учебно-научному стилю»;

– «Напиши введение к реферату на тему [*название темы*]. Объём – 1–2 абзаца. Текст должен быть написан в официально-научном стиле, без сложных терминов, но с логичным обоснованием актуальности темы»;

– «Переформулируй следующий текст в более официальный и академический стиль, сохранив смысл и структуру. Текст должен подойти для студенческой учебной работы»;

– «Сформулируй заключение по теме [*название темы*]. В выводе кратко подведи итоги, выдели значимость темы и оформи текст в научно-учебном стиле».

Важно: если пользователь создал, например, PDF, Word-документ, то такие файлы часто открываются только для просмотра, а не для редактирования. То есть, файл открывается, но печатать в него нельзя.

Создание презентации в Visual Studio Code с помощью HTML

В Visual Studio Code студент может подготовить презентацию не только в виде текстового плана, но и в виде HTML-презентации, то есть набора слайдов, открываемых в браузере. Создание презентации в Visual Studio Code с помощью HTML аналогично разработке веб-страницы: каждый слайд оформляется как отдельный блок, структура и содержание задаются с помощью HTML, внешний вид настраивается через CSS (цвета, шрифты, анимации). Такой подход позволяет создавать гибкие, интерактивные презентации с полной настройкой дизайна и логики отображения, в отличие от классических инструментов вроде Microsoft PowerPoint.

1. Создание файла для презентации. После открытия рабочей папки в Visual Studio Code необходимо создать новый файл, в котором будет размещён код будущей презентации. Для этого необходимо:

- в левой панели Explorer нажать кнопку «Новый файл»;
- ввести название файла, например: Презентация.html
- нажать Enter.

Именно в этот файл в дальнейшем будет вставляться HTML-код презентации, сгенерированный с помощью нейросети или доработанный вручную (Рисунок 64).

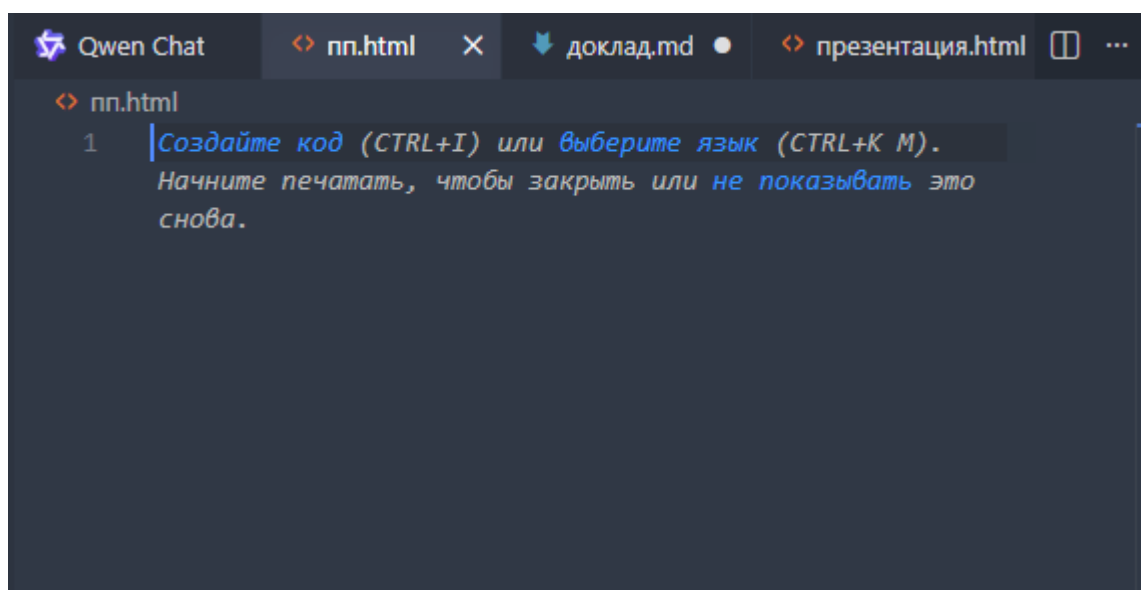


Рисунок 64 – Созданный файл

2. Подготовка содержания презентации. Перед генерацией HTML-кода рекомендуется сначала продумать содержание будущей презентации. На этом этапе желательно определить:

- тему презентации;
- предполагаемое количество слайдов;
- структуру выступления;
- ключевые тезисы;
- основные выводы.

Это необходимо для того, чтобы нейросеть могла сгенерировать более логичную и качественную презентацию.

3. Использование нейросети для создания структуры презентации. Если студенту сложно самостоятельно составить структуру, можно сначала обратиться к нейросети с запросом на создание плана слайдов.

Пример промта:

«Составь структуру учебной презентации на [*количество слайдов*] слайдов по теме [*название темы*] для студента вуза. Для каждого слайда укажи его название и краткое содержание. Структура должна быть логичной, академичной, понятной и подходить для устного выступления.»

После получения ответа студент может использовать предложенную структуру как основу будущей презентации.

4. После того как структура определена, нейросеть можно использовать для подготовки краткого содержания каждого слайда.

Пример промта:

«Подготовь краткий текст для учебной презентации на тему [*название темы*]. Для каждого слайда сформулируй заголовок и [*количество тезисов*] кратких тезисов. Текст должен быть академичным, понятным, логичным и подходить для студенческого выступления.»

5. После подготовки структуры и содержания можно перейти к созданию самой HTML-презентации. Для этого необходимо попросить нейросеть не просто составить текст, а сгенерировать полный HTML-код презентации (Рисунок 65).

Пример промта:

«Создай HTML-презентацию на тему [название темы] для студенческого выступления в вузе. Презентация должна содержать [количество слайдов] слайдов и быть выполнена в современном, аккуратном и визуально привлекательном стиле. Используй светлый фон, крупные заголовки, карточки, списки, удобочитаемый текст и логичное расположение элементов. Добавь кнопки переключения между слайдами «Назад» и «Вперёд», а также поддержку переключения стрелками клавиатуры. Подготовь полный HTML-код со встроенными CSS-стилями и JavaScript.»

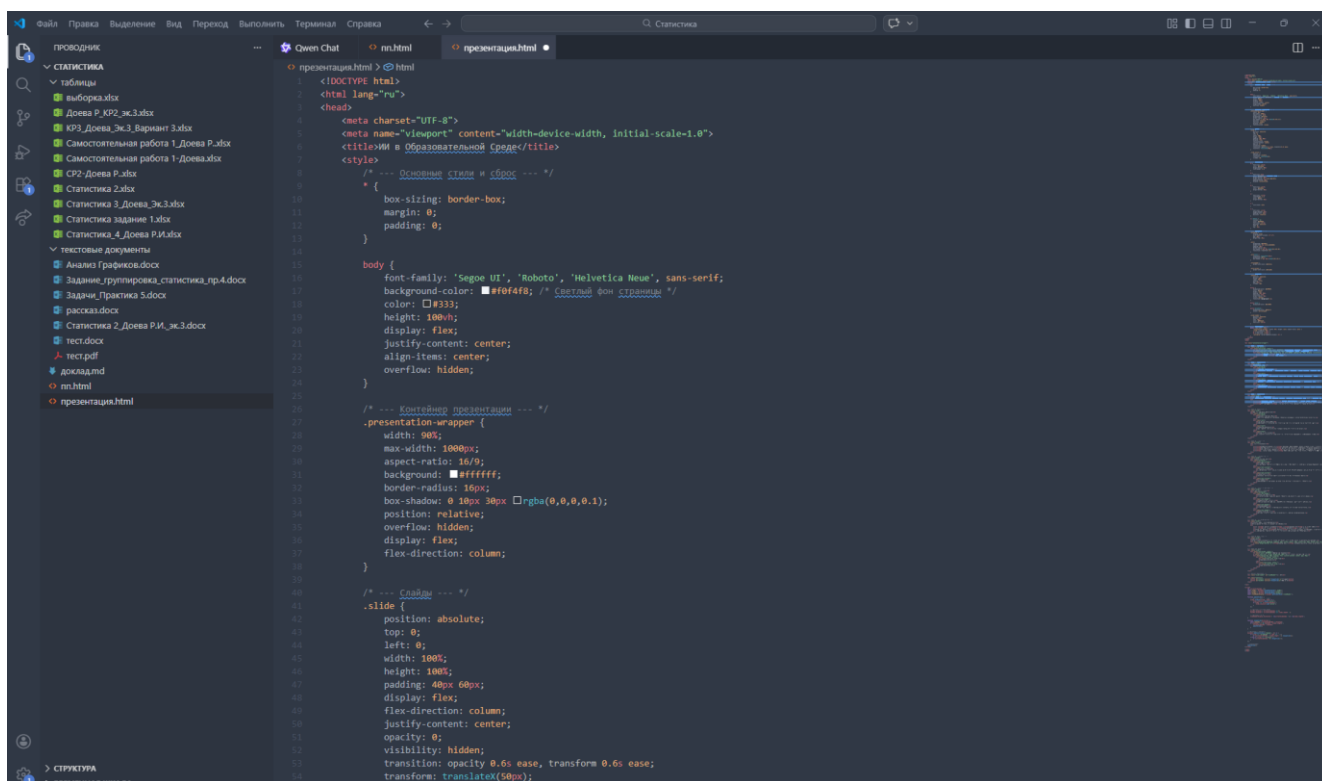


Рисунок 65 – Пример кода в файле

6. Попросите нейросеть самостоятельно создать файл в вашей папке и записать туда сгенерированный код. После этого HTML-презентация уже будет сохранена как рабочий файл проекта.

7. После сохранения HTML-файла презентацию необходимо открыть в браузере. Для этого можно:

- найти файл Презентация.html в папке проекта;
- дважды щёлкнуть по нему мышью;
- либо открыть его через браузер вручную.

После этого презентация отобразится как полноценная веб-страница (Рисунок 66).

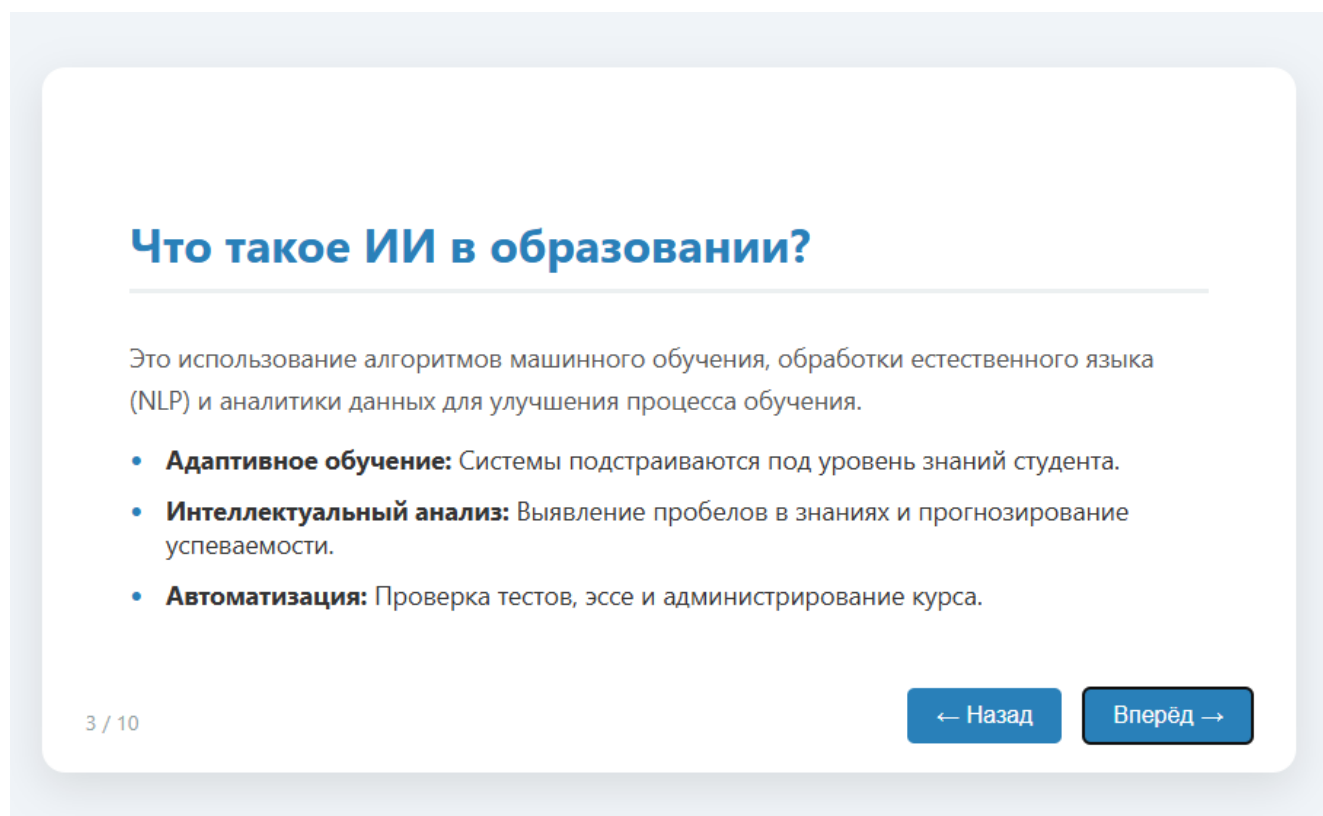


Рисунок 66 – Пример готовой презентации

Как понять, что код вставлен правильно?

Если код вставлен корректно, внутри файла Презентация.html должны присутствовать следующие основные элементы:

- структура HTML-документа (`<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`);
- блоки оформления (`<style>`);
- блоки с содержанием слайдов;
- элементы навигации;
- скрипт переключения слайдов (`<script>`).

Как сделать презентацию визуально красивой?

После получения первого варианта HTML-презентации рекомендуется улучшить её оформление. Часто нейросеть создаёт рабочую, но достаточно простую версию. Поэтому полезно дополнительно запросить улучшение дизайна (Рисунок 67).

Пример подробного промта:

«Улучшай оформление данной HTML-презентации. Сделай её более современной, красивой и удобной для учебного выступления. Используй крупные заголовки, мягкие тени, карточки, аккуратные отступы, современную типографику, приятную цветовую гамму и плавные переходы между слайдами. Сохрани структуру презентации, но сделай её визуально более профессиональной и привлекательной.»

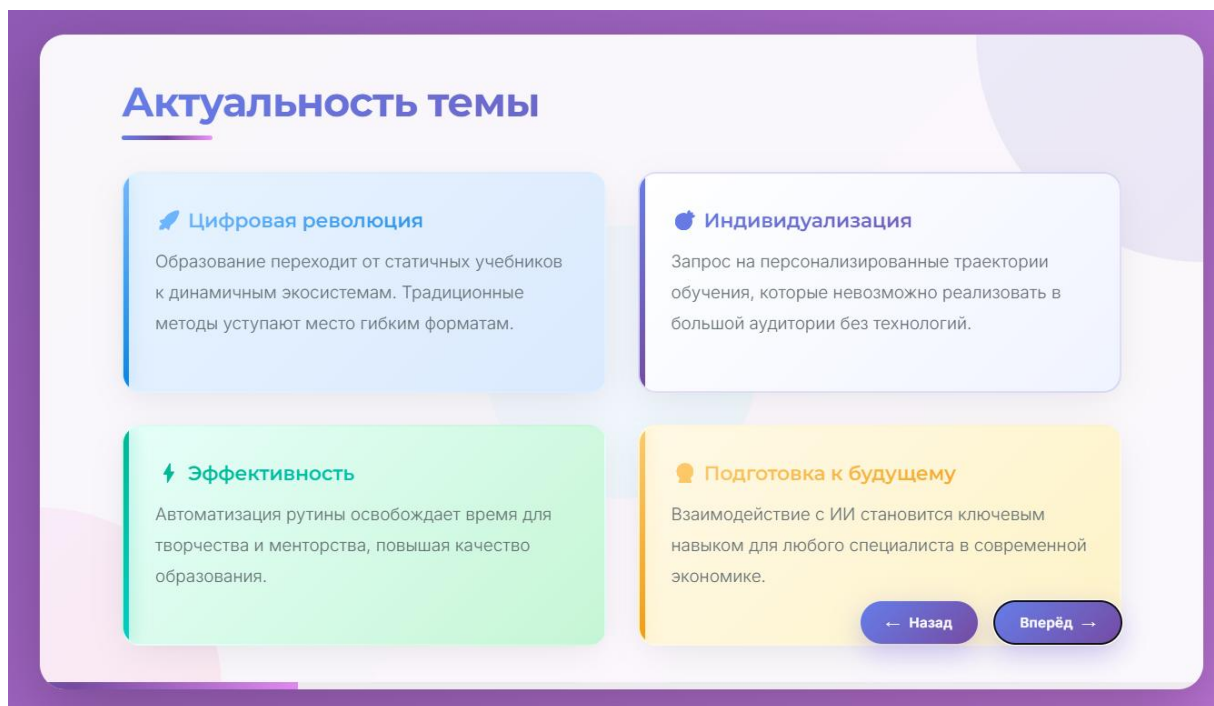


Рисунок 67 – Улучшенная презентация

Как сделать презентацию более академичной?

Если презентация предназначена для защиты реферата, проекта или учебного доклада, её оформление должно быть не только красивым, но и уместным для академической среды (Рисунок 68). Для этого можно использовать отдельный запрос.

Пример подробного промта:

«Переделай HTML-презентацию в более академичном стиле. Сделай дизайн аккуратным, строгим, современным и подходящим для выступления студента в вузе. Используй спокойные цвета, читабельные шрифты, логичное расположение текста, чистые блоки и минималистичное оформление. Не перегружай слайды лишними декоративными элементами.»

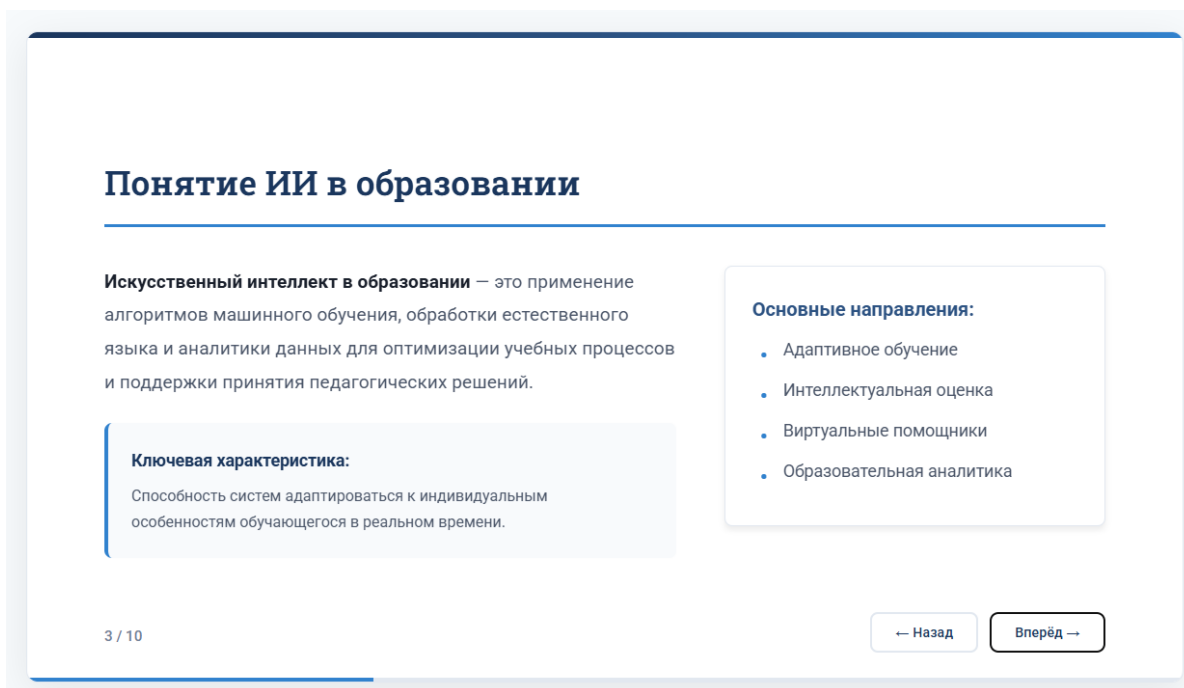


Рисунок 68 – Пример более строгой презентации

Как сократить или улучшить текст на слайдах?

После генерации презентации студент может заметить, что некоторые слайды перегружены текстом. В этом случае нейросеть можно использовать для сокращения и упрощения содержания.

Примеры полезных промтов:

- «Сократи текст данной презентации так, чтобы каждый слайд содержал только ключевые мысли.»
- «Сделай текст на слайдах более кратким и удобным для восприятия.»
- «Преобразуй длинные абзацы в короткие тезисы для слайдов.»
- «Сделай текст более понятным и логичным для устного выступления.»

Как вставить изображение в HTML-презентацию?

При создании HTML-презентации в Visual Studio Code нейросеть может не только сгенерировать структуру и оформление слайдов, но и вставить изображения в код презентации. Однако для этого необходимо правильно указать путь к файлу изображения, чтобы нейросеть смогла встроить его в HTML-код.

1. Перед тем как просить нейросеть вставить изображение в презентацию, необходимо:

- создать папку images (или другое название) внутри папки проекта;

- поместить в неё нужные изображения;
- убедиться, что файл имеет понятное имя;
- желательно использовать латинские буквы и цифры в названии файла (например, ai.jpg, education_slide.png).

Чем проще название файла, тем легче его использовать в коде и в запросах к нейросети.

2. В запросе необходимо прямо указать: название файла, путь, куда вставить и как оформить. То есть нейросети нужно дать почти техническое задание. Путь можно найти в созданной папке (Рисунок 69) или напрямую с Visual Studio Code (Рисунок 70).

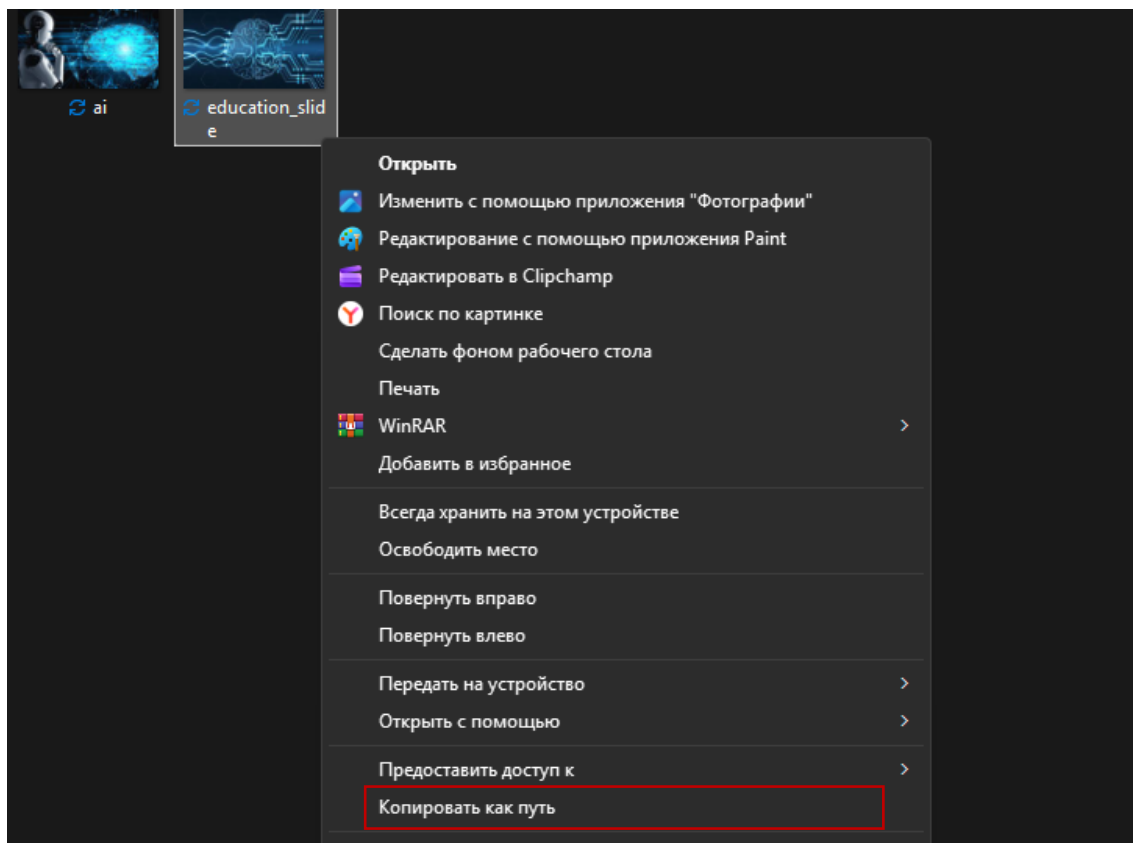


Рисунок 69 – Путь файла

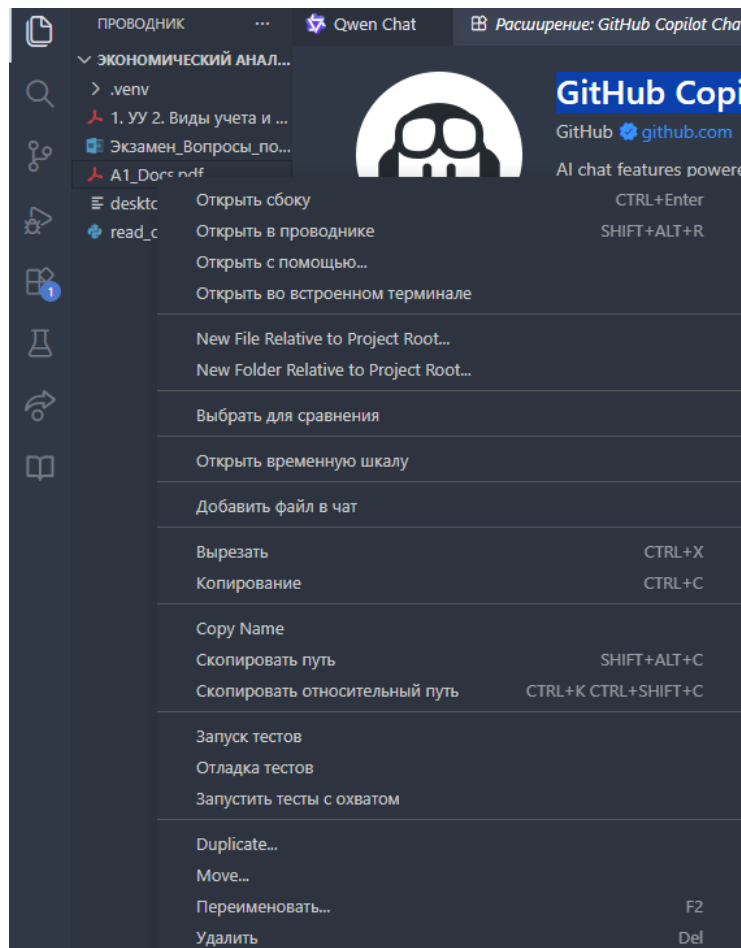


Рисунок 70 – Путь файла через Visual Studio Code

Пример промта для вставки изображения (Рисунок 71):

– «Добавь в HTML-презентацию изображение с путём [вставьте путь файла] на слайд [название слайда]. Размести изображение под текстом слайда, сделай его аккуратным, адаптивным, со скруглёнными углами и современным оформлением.»

– «Вставь в HTML-код моей презентации изображение [вставьте путь файла] на слайд [название слайда]. Размести изображение под списком тезисов, сделай его шириной около 500 пикселей, добавь скруглённые углы, отступ сверху и мягкую тень. Сохрани общий современный стиль презентации.»

– «Добавь в HTML-презентацию изображение [вставьте путь файла] на слайд [название слайда]. Размести текст слева, а изображение справа. Сделай слайд современным и аккуратным: используй карточки, отступы, скругления, удобочитаемый текст и адаптивное расположение элементов.»

– «Добавь в HTML-презентацию два изображения: [вставьте путь файла] и [вставьте путь файла]. Первое изображение вставь на слайд [название слайда], второе – на слайд [название слайда]. Сделай оба изображения аккуратными, адаптивными, одинаково оформленными и визуально согласованными с общим стилем презентации.»

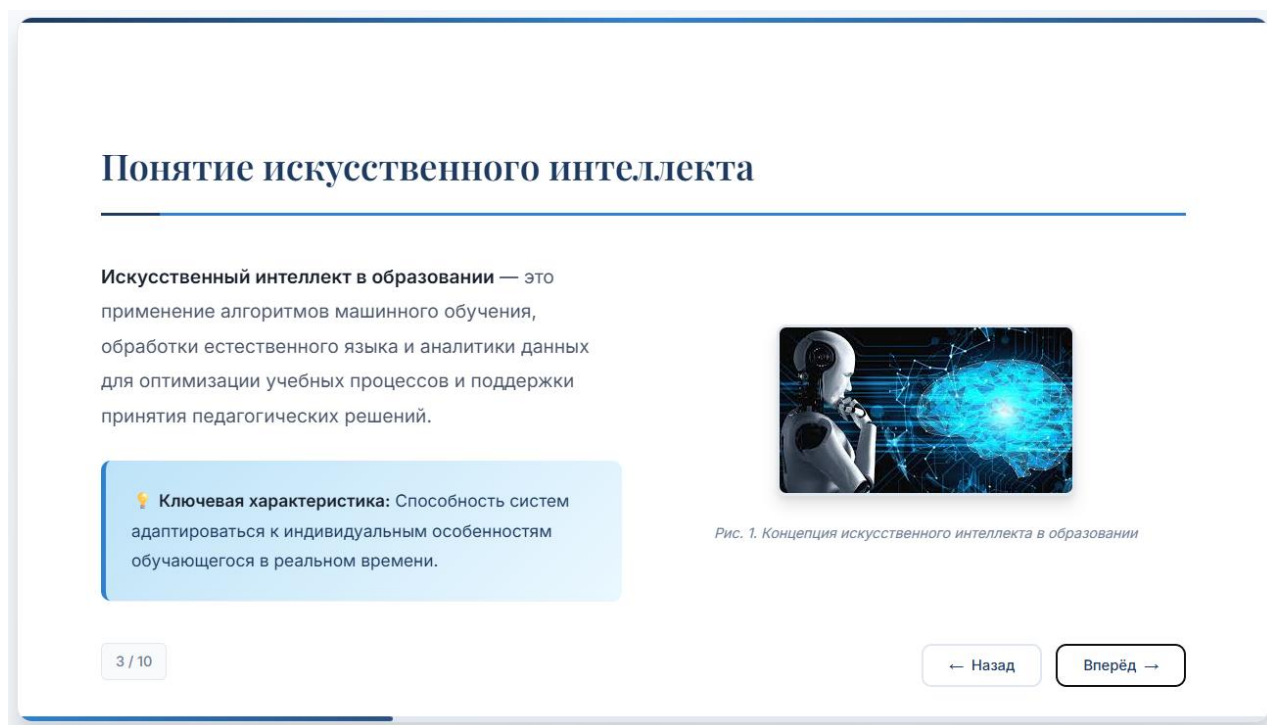


Рисунок 71 – Пример презентации с изображениями

Анализ информации

Под анализом информации в данном случае понимается работа с текстами, статьями, PDF-документами, таблицами, заметками, учебными материалами, интернет-источниками, результатами наблюдений, исследованиями и иными файлами, необходимыми для выполнения учебной работы.

Анализ информации в Visual Studio Code осуществляется за счёт работы с различными типами данных и инструментов в одной среде: пользователь может открывать текстовые документы, таблицы, PDF-файлы, выполнять поиск по всем материалам проекта, использовать расширения (например, Jupyter для анализа данных или ИИ-ассистентов для обработки текста), а также структурировать и сопоставлять информацию из разных источников. Такой подход позволяет не только просматривать данные, но и выявлять закономерности, сравнивать материалы и формировать обоснованные выводы в рамках учебной работы.

1. Создание рабочей папки для анализа информации

Перед началом анализа рекомендуется создать отдельную папку проекта, в которой будут храниться все материалы. Для этого необходимо в левой панели Explorer (Проводник) нажать кнопку «Новая папка» и ввести название (например, Анализ_информации)

2. Создание структуры файлов для анализа

Чтобы работа была организованной, рекомендуется заранее создать несколько файлов. В папке Анализ_информации можно создать, например, такие файлы (подробнее про используемые форматы смотреть в п. Настройка агентов):

- материалы_для_анализа.md;
- ключевые_идеи.md;
- сравнительный_анализ.md;
- таблица_сравнения.md;
- выводы.md;
- итоговый_анализ.md.

3. При работе с нейросетью в Visual Studio Code важно уметь точно указывать, с какими материалами нужно работать. Если в папке проекта есть несколько файлов, то в запросе рекомендуется прямо указывать их названия.

Пример правильной формулировки:

«Проанализируй материалы [*название файла. расширение*] и [*название файла. расширение*], выдели ключевые идеи, основные различия и подготовь краткие выводы.»

Или:

«Сравни данные из файла [*название файла. расширение*] с основными тезисами из файла [*название файла. расширение*] и сформулируй аналитические наблюдения.»

4. Если нейросеть не может автоматически “понять” содержимое нужного файла, студент может:

- открыть документ;
- скопировать нужный фрагмент;
- вставить его в окно чата;
- дать конкретное задание.

Пример подробного промта:

«Ниже приведён фрагмент текста из файла [*название файла. расширение*] Проанализируй его, выдели основные идеи, ключевые понятия, важные аргументы и подготовь краткое содержательное обобщение для учебной работы.»

5. Как попросить нейросеть сделать именно готовый текст для файла Чтобы результат был удобен для вставки в файл, рекомендуется сразу уточнять формат ответа.

Пример подробного промта:

«Сформируй результат не как ответ в чате, а как готовый текст для файла Итоговый_анализ.md. Текст должен быть написан в официально-учебном стиле, без лишних комментариев, без пояснений для пользователя и без обращений. Нужен готовый материал, который можно сразу сохранить в файле проекта.»

6. После создания файлов студент также может обратиться к GitHub Copilot и попросить её подготовить готовый итоговый файл с текстом (Рисунок 72).

Пример:

«На основе файлов [*название файла. расширение*] подготовь содержимое итогового файла Итоговый_анализ.md. Объедини материалы в единый логичный текст, оформи его в учебно-академическом стиле и подготовь результат для сохранения в рабочей папке проекта.»

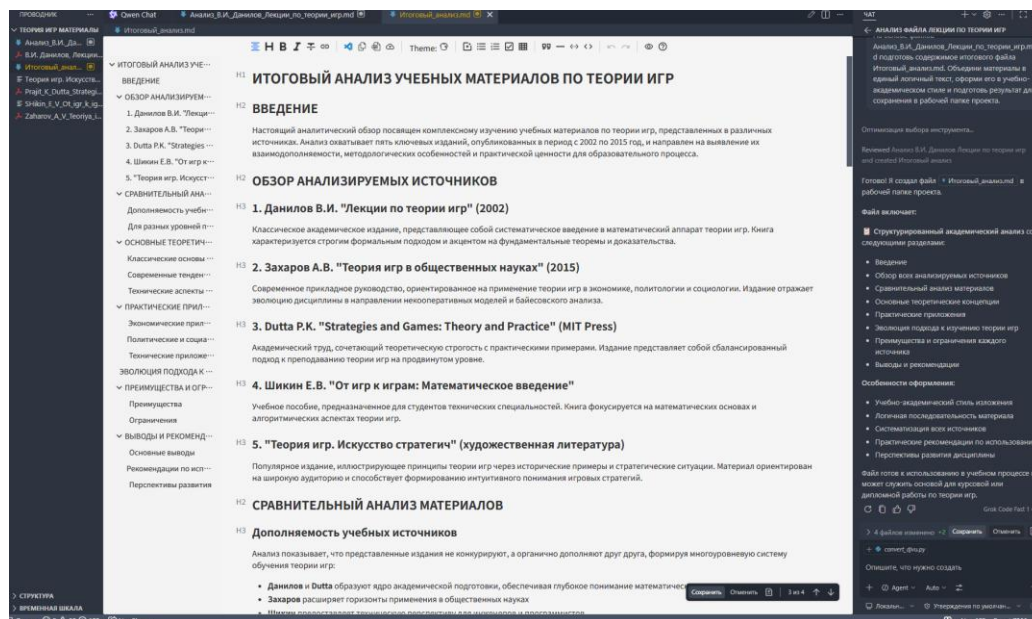


Рисунок 72 – Итоговый файл

Как просить нейросеть находить преимущества, недостатки и ограничения?

Во многих учебных работах важно не только пересказать материал, но и провести оценочный анализ.

Пример промта:

«На основе материалов [*название файла. расширение*] и [*название файла. расширение*] выдели преимущества, недостатки, ограничения, возможные риски и перспективы [*ваша тема*]. Представь результат в академической и логичной форме.»

Как сделать аналитическую таблицу по нескольким параметрам?

Одним из наиболее удобных способов анализа является представление информации в виде таблицы. Для этого можно создать файл: Таблица_сравнения.md

Пример промта:

«Составь сравнительную таблицу по материалам [*название файла. расширение*] и [*название файла. расширение*]. Сравни их по следующим параметрам: основная тема, ключевые идеи, преимущества, недостатки, практическое значение, применимость в образовании, итоговые выводы. Представь результат в удобной табличной форме.»

Как попросить нейросеть оформить итоговый анализ более структурированно?

Если студент хочет, чтобы итоговый аналитический файл выглядел более аккуратно и академично, можно попросить нейросеть заранее оформить его по структуре.

Пример промта:

«Подготовь итоговый аналитический текст по теме [*название темы*] в структурированном виде. Раздели материал на логические части: введение, основные идеи, сравнительный анализ, результаты анализа, преимущества и ограничения, выводы. Текст должен быть пригоден для сохранения в итоговом файле учебной работы.»

Работа с таблицами и данными

Visual Studio Code может использоваться для хранения таблиц, просмотра данных и подготовки аналитических комментариев по результатам исследования.

Работа с таблицами и данными в Visual Studio Code осуществляется с помощью специализированных расширений для просмотра и редактирования Excel-, CSV- и JSON-файлов, а также инструментов анализа данных, таких как Jupyter Notebook. Это позволяет открывать табличные данные прямо внутри среды, выполнять их структурирование, фильтрацию, визуализацию и последующую обработку без перехода в сторонние приложения.

Порядок работы:

1. Создайте отдельную подпапку «Данные».
2. Сохраните в неё таблицы и связанные материалы.
3. Откройте таблицу в Visual Studio Code.
4. Создайте отдельный файл для фиксации наблюдений и выводов.
5. Если в рабочей папке проекта находится несколько файлов, при работе с нейросетью рекомендуется точно указывать названия нужных файлов (например, Результаты_опроса.xlsx, Сравнение_платформ.xlsx)

Пример запроса:

«Проанализируй данные из файла *[название файла. расширение]* и выдели основные наблюдения, тенденции и результаты, которые можно использовать в учебной работе.»

Или:

«Сравни данные из файлов Сравнение_платформ.xlsx и Статистика_использования.xlsx, выдели различия, сходства и аналитические выводы.»

6. Одной из основных задач при работе с данными является смысловой анализ таблицы.

Пример промта:

«Проанализируй следующие табличные данные. Выдели основные наблюдения, тенденции, различия, закономерности и результаты, которые можно

использовать в учебной аналитической работе. Представь результат в понятной, логичной и академичной форме.»

Как попросить нейросеть выделить ключевые наблюдения по данным?

После просмотра таблицы студенту часто нужно понять, что именно важно в этих данных.

Пример промта:

«На основе представленных данных выдели ключевые наблюдения, наиболее заметные показатели, важные различия и существенные тенденции. Сформулируй результат в виде кратких аналитических пунктов.»

Как попросить нейросеть сделать краткое текстовое описание таблицы?

Иногда студенту нужно не просто проанализировать данные, а подготовить краткое описание таблицы для реферата, отчёта или доклада.

Пример промта:

«Подготовь краткое текстовое описание представленных данных в учебно-академическом стиле. Опиши, что показывают данные, какие показатели являются наиболее заметными и какие общие выводы можно сделать на их основе.»

Как попросить нейросеть создать сравнительную таблицу?

Нейросеть можно использовать для создания удобной аналитической таблицы по заранее заданным параметрам.

Пример промта:

«Составь сравнительную таблицу по следующим данным. Сравни объекты по параметрам: название, назначение, основные функции, преимущества, недостатки, удобство использования, практическая ценность и итоговая оценка. Представь результат в табличной форме, пригодной для учебной работы.»

Подготовка к экзаменам и зачётам

Visual Studio Code может использоваться как удобное пространство для систематизации учебных материалов и подготовки к экзаменам, зачётам и тестированию.

Подготовка к экзаменам и зачётам в Visual Studio Code может быть организована за счёт систематизации конспектов, лекций и учебных материалов в едином проекте, быстрого поиска по всем документам, использования ИИ-ассистентов для генерации вопросов, кратких выжимок и пояснений, а также применения Markdown и других инструментов для создания структурированных шпаргалок, чек-листов и планов повторения.

Порядок работы:

1. Создайте отдельную папку по учебной дисциплине.
2. Разделите материалы по темам или билетам.
3. Создайте отдельные файлы для: кратких ответов, терминов, определений, вопросов для самопроверки и т.д.
4. На первом этапе рекомендуется определить, какие темы необходимо повторить. Если у студента есть список экзаменационных вопросов, его можно использовать как основу для структуры подготовки.

Пример запроса (Рисунок 73):

«На основе файла Вопросы_к_экзамену.docx выдели основные темы, которые необходимо изучить для подготовки к экзамену. Представь результат в виде структурированного списка.»

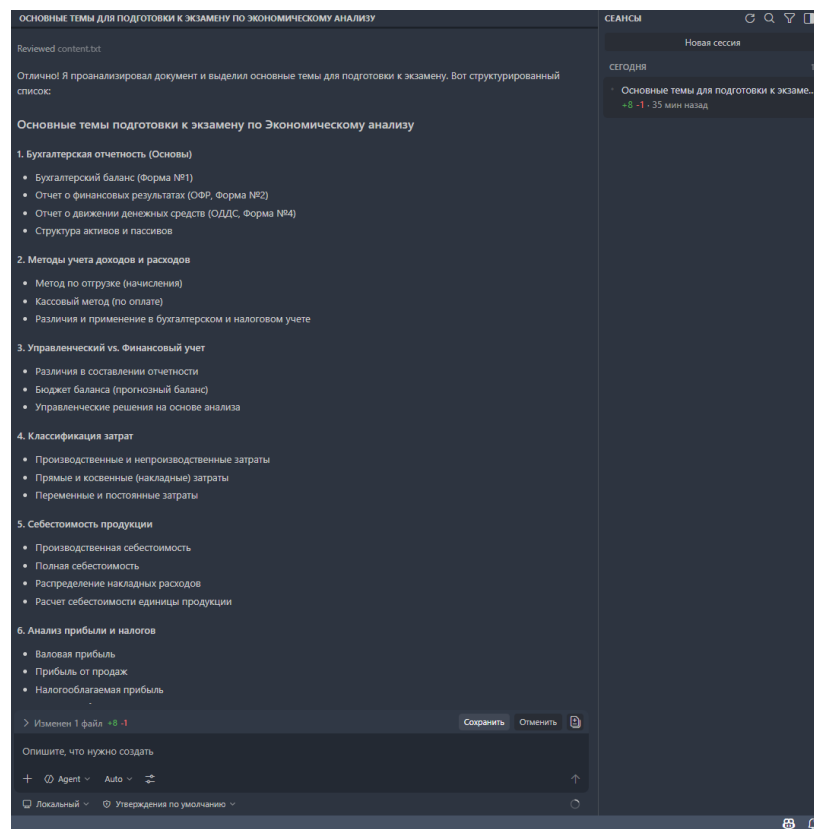


Рисунок 73 – Пример подготовки основных тем

5. После определения тем рекомендуется подготовить по каждой теме краткий и понятный конспект (Рисунок 74).

Пример запроса:

«Подготовь краткий конспект по теме на основе материалов [название файла. расширение] и [название файла. расширение]. Выдели основные понятия, определения, важные положения и ключевые выводы. Текст должен быть удобен для повторения перед экзаменом.»

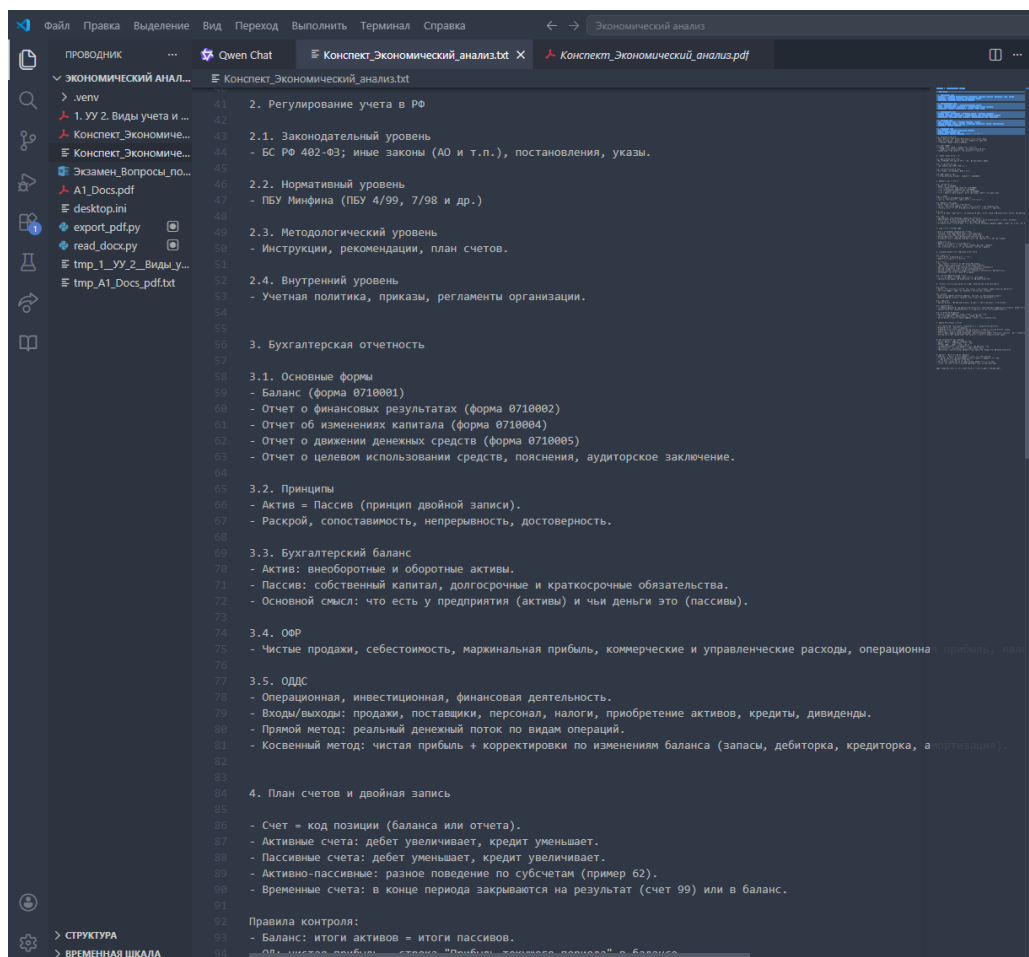


Рисунок 74 – Пример конспекта темы

6. Один из самых эффективных способов подготовки – это составление готовых ответов на вопросы.

Пример запроса:

«Подготовь развернутый, но краткий и понятный ответ на экзаменационный вопрос: [формулировка вопроса]. Используй учебный стиль, выдели основную суть, ключевые понятия и логичный вывод.»

Если у студента уже есть файл с вопросами, можно использовать более точный запрос:

Пример:

«На основе файла Вопросы_к_экзамену.docx подготовь ответы на вопросы в удобной форме для повторения перед экзаменом.»

7. Для эффективного повторения полезно собрать ключевые определения и термины в отдельный файл.

Пример запроса:

«На основе материалов [*название файла. расширение*] и [*название файла. расширение*] выдели основные термины, определения и важные понятия по теме. Представь результат в краткой и понятной форме для повторения.»

8. Если в процессе подготовки студент понимает, что какие-то темы усваиваются хуже, их рекомендуется вынести отдельно.

Пример запроса:

«На основе материалов по теме объясни данный вопрос простым и понятным языком. Сделай разбор по шагам, выдели главное и укажи, что нужно запомнить в первую очередь.»

9. Нейросеть можно использовать для подготовки вопросов для самопроверки.

Пример запроса:

«Составь вопросы для самопроверки по теме [*название темы*]. Вопросы должны проверять понимание основных понятий, связей между темами и умение кратко объяснить материал.»

Помощь нейросети в написании аннотаций

Аннотация – это краткая характеристика статьи, курсовой или исследовательской работы, которая раскрывает её тему, цель, основные результаты и выводы. Написание качественной аннотации часто вызывает трудности, но нейросеть может стать отличным помощником.

В Visual Studio Code с расширениями Qwen или GitHub Copilot Chat можно автоматически сгенерировать аннотацию на основе текста работы. Достаточно скопировать ключевые разделы (введение, цели, выводы) и дать чёткий запрос.

Как составить эффективный запрос для генерации аннотации?

Шаг 1. Соберите исходный материал – вставьте в чат нейросети следующие части вашей работы:

- тему работы;
- цель и задачи исследования;
- краткое описание методов или хода работы;
- основные выводы или полученные результаты.

Шаг 2. Используйте готовый шаблон промта – скопируйте и адаптируйте под свою тему:

«Напиши аннотацию для учебной работы по теме: [вставьте тему].

Цель работы: [опишите цель].

Методы / ход работы: [кратко опишите].

Полученные результаты: [основные выводы].

Требования к аннотации:

- объём: 100–200 слов;
- академический стиль;
- кратко отразить цель, ход и результаты;
- не использовать местоимения «я», «мы»;
- язык: русский.

Оформи аннотацию как готовый текст для курсовой работы.»

Шаг 3. Уточните и доработайте результат – нейросеть может выдать черновик, который лучше отредактировать. Попросите её сократить или расширить текст:

«Сократи аннотацию до 80 слов, сохрани главную мысль.» – или – «Расширь аннотацию, добавь информацию о практической значимости работы.»

Пример для научной статьи по педагогике

Тема: «Использование ИИ-ассистентов в обучении программированию»

Аннотация:

Статья посвящена применению ИИ-ассистентов (GitHub Copilot, Qwen) в процессе обучения программированию. Цель работы – оценить влияние ИИ-инструментов на успеваемость и мотивацию студентов. В ходе эксперимента студенты выполняли лабораторные работы с использованием ИИ-ассистентов. Результаты показали повышение скорости выполнения заданий на 35%, однако выявлены риски снижения самостоятельного мышления при чрезмерном использовании автодополнений. Предложены методические рекомендации по интеграции ИИ в учебный процесс. Материалы статьи могут быть полезны преподавателям информатики и разработчикам образовательных программ.

Совет: всегда проверяйте и редактируйте сгенерированную аннотацию. Нейросеть может допустить фактические ошибки или использовать не совсем точные формулировки. Итоговый текст должен соответствовать стилю вашего учебного заведения.

Универсальный промт для аннотации курсовой/статьи:

«Напиши аннотацию для [курсовой работы / научной статьи / диплома] на тему: [тема работы].

Описание работы: [краткое описание, 2–3 предложения о содержании].

Цель исследования: [цель].

Методы / подход: [какие методы использовались].

Основные результаты / выводы: [перечислите ключевые итоги].

Требования:

– объём: 100–200 слов;

- академический стиль (без «я», «мы»);
- отразить цель, ход, результаты и практическую значимость;
- язык: русский.

Выдай только текст аннотации, без лишних комментариев.»

Возможные проблемы

Проблема 1. Установил расширение, но оно не появилось на панели, и пользователь не знает, как его открыть

Пошаговое решение:

1. Сначала перезапустите VS Code.
2. Проверьте левую панель. После перезапуска на левой вертикальной панели должна появиться новая иконка (например, логотип Qwen или ChatGPT). Нажмите на неё – откроется чат с нейросетью (Рисунок 75).

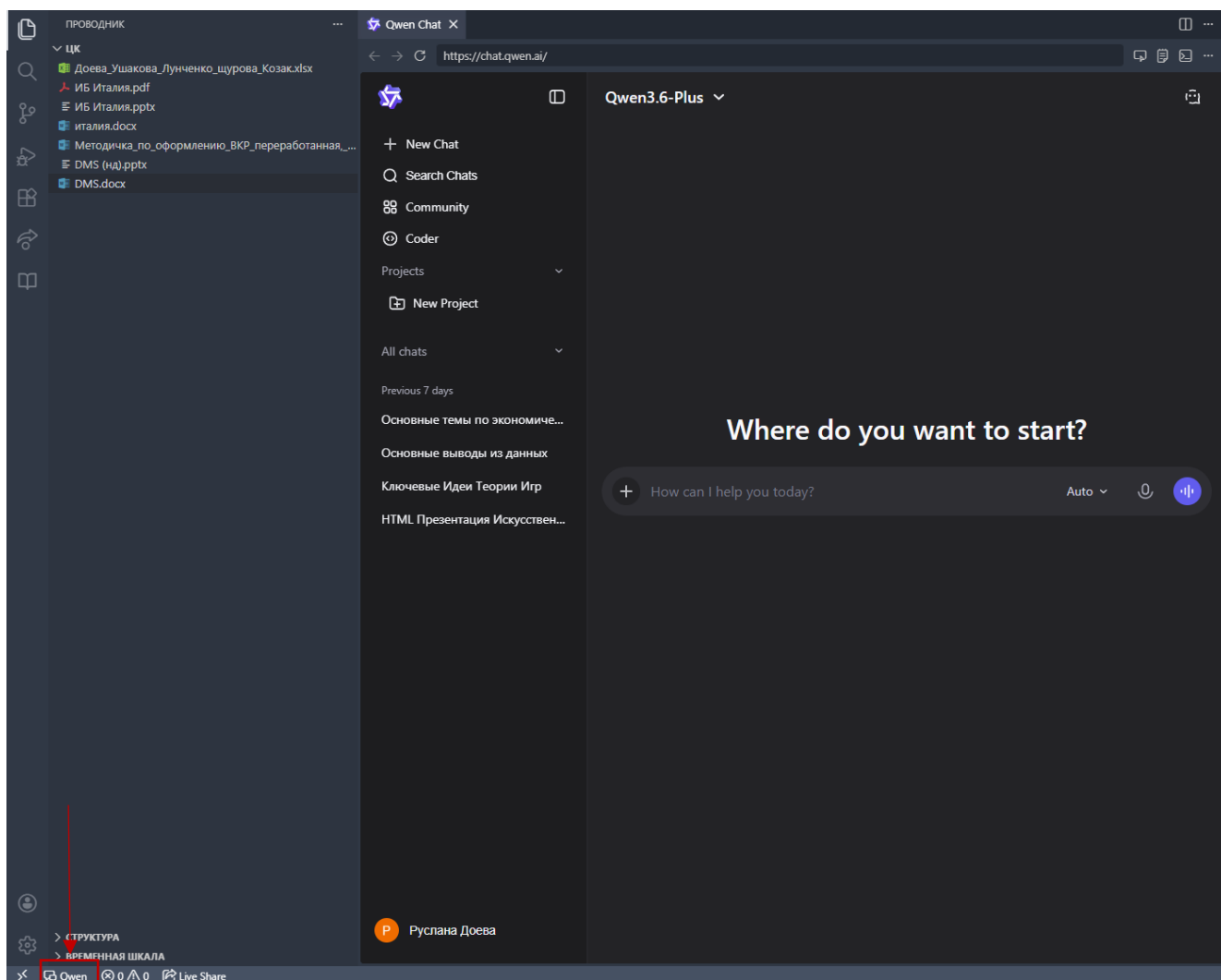


Рисунок 75 – Открытие установленной нейросети

3. Если иконки нет – откройте чат через командную строку:
 - нажмите Ctrl+Shift+P (откроется командная панель) (Рисунок 76);
 - начните вводить название расширения, например: Qwen: Open Chat;
 - выберите нужную команду из списка и нажмите Enter.

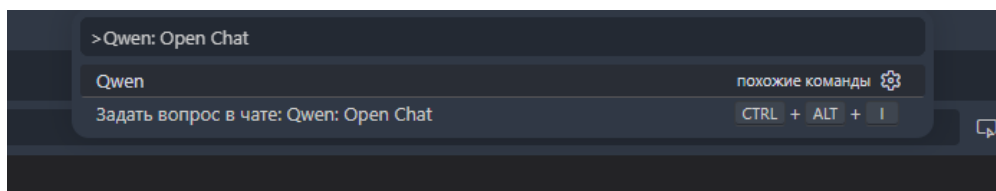


Рисунок 76 – Командная строка

4. Если команды тоже нет:
 - расширение не установилось правильно;
 - откройте расширения (Ctrl+Shift+X), найдите установленное расширение;
 - нажмите на значок шестерёнки (Manage) → выберите Disable, затем снова Enable (Рисунок 77);
 - повторите шаг 1 (перезапуск VS Code).

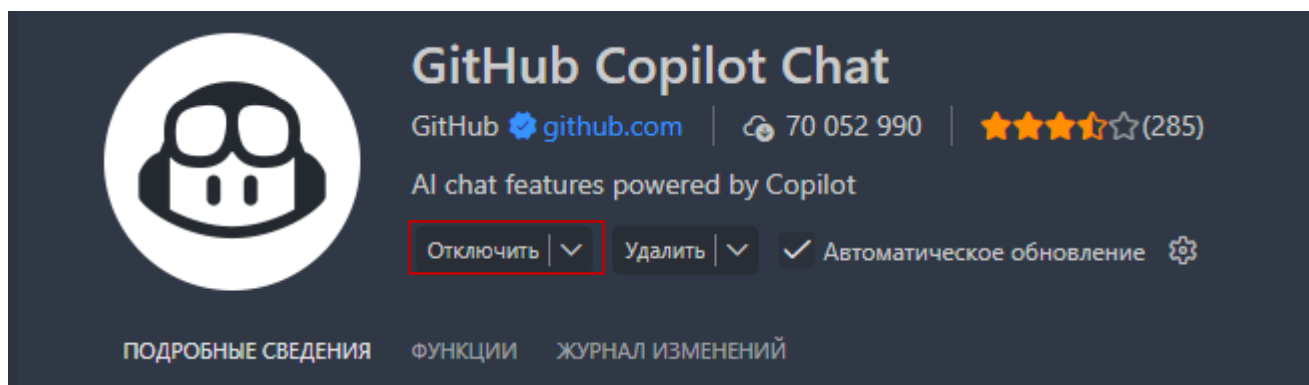


Рисунок 77 – Отключение установленного расширения

Проблема 2. После установки нейросеть всё равно не отвечает, и студент не понимает, как проверить, работает ли она вообще.

1. Проверьте интернет:
 - откройте любой сайт в браузере (например, google.com);
 - если сайты не открываются – проблема с интернетом.
2. Перезапустите VS Code:
 - закройте программу полностью;
 - откройте снова;
 - попробуйте написать простой запрос, например: «Привет, как дела?».
3. Проверьте, выбрано ли правильное расширение:
 - на левой панели нажмите на иконку нейросети;

- убедитесь, что вы в нужном чате;
- если у вас установлено несколько расширений нейросетей – они могут конфликтовать. Отключите ненужные (шестерёнка → Disable).

4. Если ничего не помогло, нажмите Ctrl+Shift+P, напишите «Reload Window» и нажмите Enter. Это перезагрузит VS Code без потери данных. Попробуйте снова.

5. Если ошибка осталась:

- сделайте скриншот ошибки (обычно она написана в чате или в терминале внизу).
- покажите скриншот преподавателю, создателю методички или другой нейросети.

Проблема 3. Не может переключиться между разными чатами (начал новую тему, а старая пропала)

1. В панели расширения (там, где открыт чат) обычно есть список всех ваших диалогов. Ищите слева или сверху раздел «Chats», «History» или просто список с названиями (Рисунок 78).

2. Чтобы открыть старый чат – нажмите на его название.

3. Чтобы создать новый – нажмите кнопку «New Chat» (плюсик или «+»).

4. Возможно, расширение не сохраняет историю. Некоторые расширения требуют входа в аккаунт для сохранения истории.

5. Проверьте настройки расширения (иконка шестерёнки рядом с расширением в списке расширений) – там может быть опция «Save chat history».

Важно: старые чаты не удаляются сами. Если вы не видите список, возможно, панель истории свёрнута – нажмите на иконку «History» или «Список чатов».

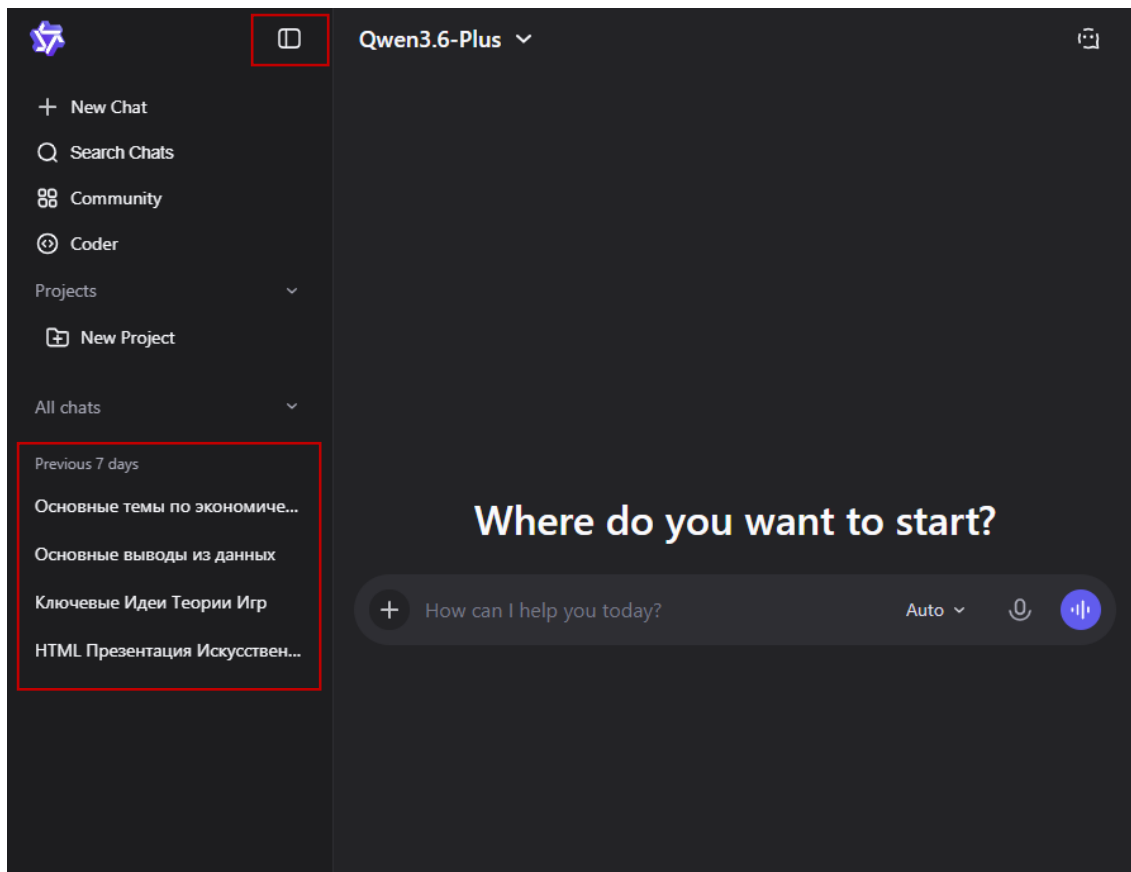


Рисунок 78 – История запросов Qwen

Проблема 4. Случайно закрыл панель с чатом и не знает, как её вернуть

1. Самый быстрый способ: нажмите Ctrl+B (это показывает/скрывает боковую панель).
2. Если боковая панель открыта, но чат не виден – посмотрите на иконки сверху. Нажмите на иконку вашего расширения (Рисунок 79).
3. Если иконка пропала – значит, расширение отключилось. Откройте расширения (Ctrl+Shift+X), найдите ваше расширение и нажмите «Enable».
4. Крайний случай: нажмите Ctrl+Shift+P, введите «Reload Window» и нажмите Enter. VS Code перезагрузится, и панели вернутся.

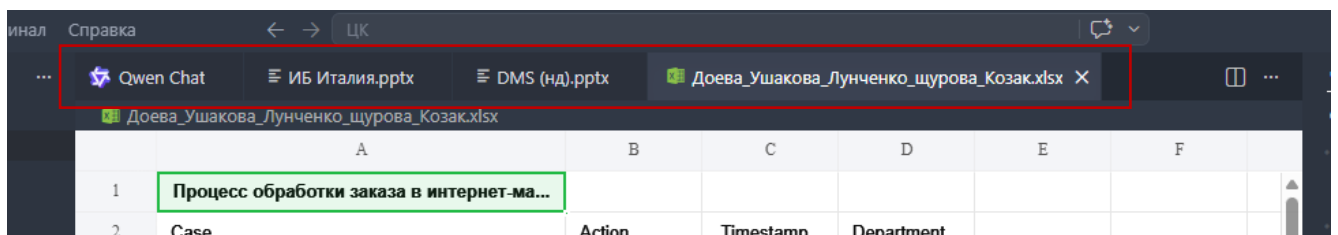


Рисунок 79 – Панель вкладок

Проблема 5. В чате пропадает часть текста или ответ обрезается.

1. У некоторых расширений есть ограничение на длину ответа. Если ответ слишком длинный, он может обрезаться.

2. Что делать:

– попросите нейросеть: «Выдавай ответ по частям. Сначала первую половину, потом я скажу "продолжи"»;

– или: «Разбей ответ на несколько сообщений».

3. Если проблема повторяется постоянно – проверьте настройки расширения. Возможно, там можно увеличить лимит символов.

Проблема 6. Не может открыть существующий файл, чтобы скопировать из него текст для нейросети

1. В VS Code файлы открываются так: Ctrl+O – открыть файл (выбрать в проводнике). Или перетащить файл мышкой из папки в окно VS Code

2. Если файл не открывается:

– проверьте, не занят ли файл другой программой (например, Word или Adobe Reader);

– закройте файл в других программах, попробуйте снова.

3. VS Code не умеет показывать PDF как текст. Нужно скопировать текст из другой программы (браузера, Adobe Reader) и вставить в чат. Также можно попросить GitHub Copilot Chat переконвертировать нужный файл в подходящий формат. Нейросеть попросить разрешение на установку расширения Python и совершение некоторых действий (Рисунок 80).

```
Searched for Экзамен_Вопросы_по_Экономическому_анализу.docx

Помогу вам! Сначала мне нужно найти и изучить файл. Давайте начнем с поиска.

Searched for files matching **/*Экзамен*Вопросы*

Файл найден! Теперь нужно его прочитать. Так как это файл .docx, я использую Python для извлечения текста.

Запущено from docx import Document; doc = Document('c:\User...

Попробую установить необходимую библиотеку:

Запуск в терминале

Дайте мне установить необходимые пакеты:

Installed python-docx and read .docx file

Отлично! Теперь прочитаю содержимое документа:

Created temporary Python script and ran it

Позвольте мне создать временный Python скрипт:

Reviewed content.txt

Отлично! Я проанализировал документ и выделил основные темы для подготовки к экзамену. Вот структурированный список:
```

Рисунок 80 – Конвертация файла через GitHub Copilot Chat

Проблема 7. Сохранил файл с ответом нейросети, но не может найти его потом.

1. Где VS Code сохраняет файлы по умолчанию:
 - если вы не выбирали папку, VS Code мог сохранить файл в последнюю открытую папку или в домашнюю директорию пользователя;
 - нажмите Ctrl+O (открыть файл) и посмотрите, в какой папке вы находитесь.
2. Как найти потерянный файл:
 - в VS Code: нажмите Ctrl+P (быстрый поиск файлов) и начните вводить название файла (Рисунок 81);
 - в Windows: откройте проводник, в строке поиска введите имя файла (или часть имени).
3. Чтобы не терять в следующий раз:
 - перед сохранением нажмите Ctrl+S – появится окно «Сохранить как», где вы сами выберете папку и имя файла;

– называйте файлы понятно, например:
«презентация_биология_1вариант.txt»

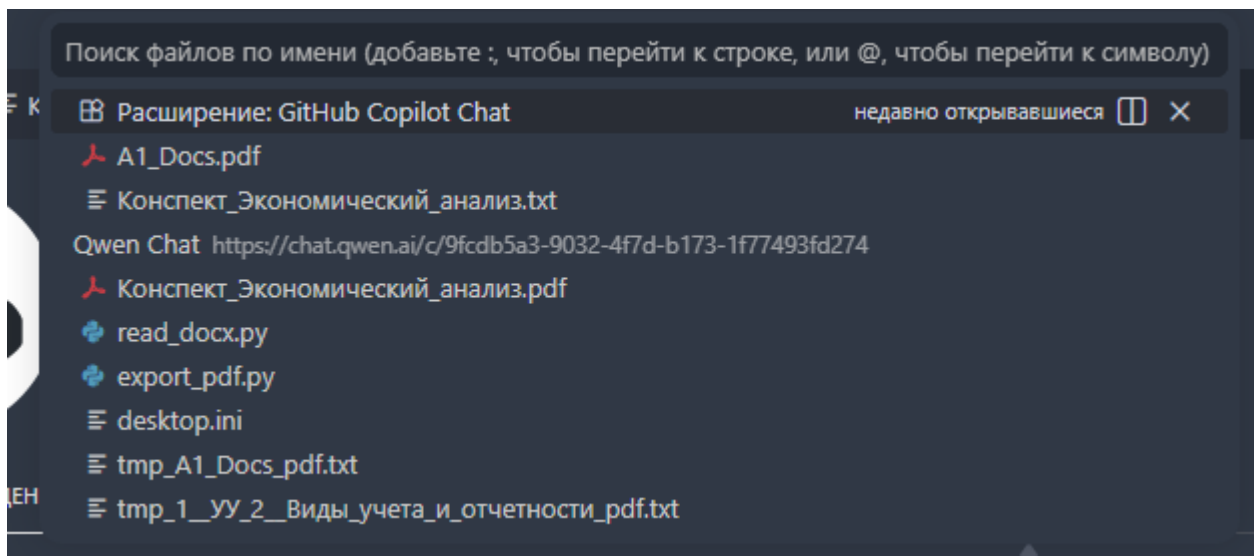


Рисунок 81 – Поиск файлов

Проблема 8. Нейросеть перестала отвечать после того, как VS Code долго был открыт.

1. VS Code иногда «устаёт» – особенно если открыто много вкладок или расширений.
2. Решение: перезагрузите VS Code без закрытия вкладок.
 - нажмите Ctrl+Shift+P;
 - введите «Reload Window» и нажмите Enter.
 - VS Code перезагрузится, чат откроется заново, история сохранится.
3. Если не помогло – закройте VS Code полностью и откройте снова.

Проблема 9. При попытке вставить длинный текст в чат VS Code зависает.

1. Некоторые расширения не справляются с очень длинным текстом (более 10–20 тысяч символов).
2. Что делать:
 - разбейте текст на части (по 5–10 абзацев);
 - отправляйте частями, как описано в проблеме 5.
3. Если VS Code завис:
 - подождите 1–2 минуты – возможно, он обрабатывает текст;

- если не отвечает – закройте VS Code через диспетчера задач (Ctrl+Shift+Esc → найти VS Code → «Снять задачу»);
- откройте заново и попробуйте отправить текст меньшими частями.

Проблема 10. Установил VS Code, но при попытке открыть чат с нейросетью вылезает ошибка про Python.

Пошаговое решение:

1. Многие расширения для нейросетей требуют, чтобы был установлен Python.

2. Как проверить:

- откройте терминал в VS Code (нижняя панель, иконка >< или нажмите Ctrl+`) (Рисунок 82);
- напишите `python --version` и нажмите Enter;
- если пишет «не является внутренней или внешней командой» – Python не установлен.

3. Что делать:

- скачайте Python с официального сайта (<https://www.python.org/>);
- при установке обязательно поставьте галочку «Add Python to PATH»;
- перезагрузите VS Code после установки

Важно: иногда будет достаточно загрузить внутреннее расширение Python.

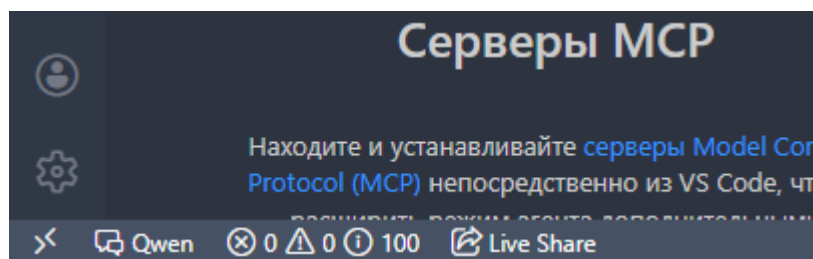


Рисунок 82 – Открытие терминала

Проблема 11. Установил Python, но VS Code всё равно не видит его.

1. Проверьте, что Python добавился в PATH:

- закройте VS Code полностью;
- откройте командную строку Windows (Win+R → `cmd` → Enter);
- напишите `python --version` – если показывает версию, всё хорошо.

2. Если в командной строке работает, а в VS Code нет:
 - перезагрузите компьютер (иногда нужно);
 - в VS Code нажмите Ctrl+Shift+P , введите «Python: Select Interpreter» и выберите установленную версию Python.
3. Если не помогло: переустановите Python, снова убедившись, что галочка «Add to PATH» стоит.

Заключение

В рамках данного методического руководства были рассмотрены современные цифровые инструменты и подходы, позволяющие существенно повысить эффективность учебной и научно-исследовательской деятельности обучающихся. Проведённый анализ показал, что интеграция интеллектуальных и программных средств в образовательный процесс обеспечивает качественно новый уровень организации самостоятельной работы, подготовки академических текстов, обработки информации и структурирования материалов.

Особое значение имеет тот факт, что рассмотренные инструменты не являются изолированными решениями, а могут использоваться совместно, дополняя друг друга в рамках единого образовательного процесса. Например, результаты анализа материалов с помощью ИИ могут интегрироваться в документы Word, храниться в облачных проектах, структурироваться в VS Code и сопровождаться системой версионирования через Git/GitHub. Такой комплексный подход позволяет выстроить современную, гибкую и масштабируемую инфраструктуру персональной учебной среды.

Вы прошли путь от автоматизации в Word через макросы до полноценной работы с ИИ-агентами в Visual Studio Code. Вы научились настраивать системные файлы (AGENTS.md, copilot-instructions.md), создавать HTML-презентации одним запросом, анализировать учебные материалы и готовиться к экзаменам, не покидая редактора. Но это лишь начало.

Что открывается дальше? Современные агенты (Claude Code, Qwen Code, GitHub Copilot, Cursor) обладают возможностями, которые превращают их из простых помощников в автономных исполнителей:

- Skills (навыки) – многоразовые инструкции, которые агент загружает по требованию. Вы можете создать навык «Академический рецензент», и тогда агент автоматически проверит вашу курсовую на соответствие ГОСТ, стилистику и структуру – достаточно будет сказать «примени навык рецензента к файлу курсовая.docx».

- Subagents (под-агенты) – вы делегируете разные части задачи специализированным агентам: один ищет источники в открытых базах данных, второй составляет план, третий пишет текст, четвёртый оформляет список литературы. Всё это происходит параллельно, а вы получаете готовый черновик.

- MCP (Model Context Protocol) – универсальный протокол, позволяющий агенту подключаться к внешним системам: университетской библиотеке, API статистических служб, вашей личной базе заметок. Агент сможет сам находить нужные статьи, скачивать их, извлекать ключевые идеи и вставлять цитаты с правильными ссылками.

Как начать? Уже сейчас вы можете:

1. Для понимания принципов построения пользовательских навыков и вспомогательных агентов рекомендуется ознакомиться с открытым репозиторием учебных материалов: <https://github.com/alexeykrol/coursevibecode>. В данном репозитории представлены реальные примеры настройки Skills и Subagents, демонстрирующие практические сценарии расширения функциональности ИИ-ассистентов. Особое внимание рекомендуется уделить каталогу: `2_lessons/claude-code-book/skills-examples`. В указанной директории содержатся шаблоны навыков, примеры структуры конфигурационных файлов, варианты построения цепочек инструкций и практические кейсы автоматизации типовых задач. Анализ данных примеров позволяет понять принципы проектирования модульных ИИ-инструкций и логику построения расширяемых агентных систем.

2. После ознакомления с примерами рекомендуется перейти к созданию собственного простейшего навыка. Для этого необходимо в рабочем каталоге проекта сформировать директорию: `.qwen/skills/`. Внутри неё создаётся файл: `SKILL.md`. В данный файл помещается инструкция, определяющая поведение пользовательского навыка. В качестве первого тестового примера может использоваться следующая базовая конфигурация:

Проверь текст на орфографические, пунктуационные и стилистические ошибки. Исправь найденные недочёты. Сохрани академический стиль изложения.

После подключения данного навыка ИИ-ассистент сможет использовать его как специализированный модуль обработки текста при соответствующих запросах пользователя. На практике это является первым шагом к построению собственных автоматизированных сценариев взаимодействия с интеллектуальной системой.

3. Для получения доступа к расширенным возможностям облачной разработки и профессиональным ИИ-инструментам обучающимся рекомендуется оформить участие в программе: GitHub Student Developer Pack. Данная программа предоставляет студентам бесплатный доступ к ряду профессиональных сервисов и образовательных ресурсов, включая:

- продвинутые ИИ-ассистенты и агентные системы;
- облачные среды разработки;
- расширенные тарифы хостинговых платформ;
- профессиональные инструменты DevOps и CI/CD;
- специализированные образовательные платформы и технические курсы.

Подключение к данной программе позволяет существенно расширить доступный инструментарий и использовать профессиональные средства разработки без дополнительных финансовых затрат.

Наиболее эффективной стратегией освоения является последовательный подход: сначала изучение готовых решений, затем модификация существующих навыков, далее разработка собственных пользовательских сценариев автоматизации.

Такой подход позволяет постепенно перейти от базового использования ИИ-ассистентов к построению персонализированной интеллектуальной среды, адаптированной под конкретные учебные, исследовательские и профессиональные задачи пользователя.

Существенное расширение функционала достигается за счёт подключения интерпретаторов и аналитических библиотек, позволяющих выполнять обработку данных непосредственно в рабочей среде. Использование Python, Jupyter Notebook, Pandas, NumPy и Matplotlib даёт возможность проводить статистический анализ,

очищать и трансформировать данные, строить графики, диаграммы и аналитические модели, а также автоматически формировать визуализированные результаты исследований для включения в отчётные материалы.

Мир агентов развивается стремительно. То, что вы освоили в этой методичке – прочный фундамент. Дальше – только ваша фантазия и желание экспериментировать. Успехов в учёбе и автоматизации!